



ONBUCH UPSKILLING PROGRAM
Fascicule de cours & d'exercices

2026 – 2027
Terminale TI

PROGRAMMATION · TERMINALE TI

Program- mation.

C

PHP

HTML

CSS

JavaScript

Du langage C à la programmation web : algorithmique, structures de données, PHP côté serveur et HTML / CSS / JavaScript côté client.

AUTEUR – L'ÉQUIPE ONBUCH

index.php – onbuch

```
1 <!DOCTYPE html>
2 <h1>Bonjour, OnBuch</h1>
3 <?php echo date("Y"); ?>
4 <script>
5 console.log("Tle TI");
  _
```



APPLICATION MOBILE
OnBuch — disponible sur Google Play Store

play.google.com

{ } </> ; \$ #

NOM · PRÉNOM _____
CLASSE _____

Avant-propos

Ce fascicule accompagne l'élève de **Terminale TI** (Technologies de l'Information) dans la maîtrise du module **Algorithmique & Programmation** du programme officiel camerounais (MINESEC). Il couvre, pas à pas et par la pratique, la **programmation web dynamique** (HTML, JavaScript, **PHP** et **MySQL**) puis la **programmation procédurale en langage C** (applications monopostes).

Chaque chapitre propose : un **cours clair et illustré**, du **code entièrement commenté** (avec coloration syntaxique), des **méthodes**, des **exercices corrigés** progressifs et un encadré « *L'essentiel* ». Le livre se termine par des **aide-mémoire** et des **sujets type Baccalauréat corrigés**.

À retenir — comment travailler ?

Lis le cours, *tape et exécute* chaque exemple sur ta machine (WampServer / XAMPP pour le web, Code ::Blocks pour le C), puis cherche les exercices seul avant de regarder le corrigé. On ne programme bien qu'en programmant.

L'équipe OnBuch

Sommaire

1	Pages web et formulaires (rappels)	1
1.1	Structure d'une page HTML	1
1.2	Les formulaires HTML	2
1.3	Les tableaux HTML	4
1.4	JavaScript : les bases	4
1.5	JavaScript et la page	6
1.6	Contrôle de saisie d'un formulaire	6
1.7	Exercices	8
1.8	Corrigés	9
2	L'environnement de développement web	14
2.1	L'architecture client/serveur	14
2.2	Les environnements intégrés de développement web	15
2.3	Installer et configurer l'environnement	16
2.4	Démarrer et arrêter les services	16
2.5	Le répertoire racine	17
2.6	Le port d'écoute	17
2.7	Héberger un site	18
2.8	Exercices	19
2.9	Corrigés	20
3	Premiers pas en PHP	22
3.1	Qu'est-ce que PHP ?	22
3.2	Insérer du PHP dans une page	22
3.3	Variables et constantes	23
3.4	Afficher : <code>echo</code> et <code>print</code>	23
3.5	Les opérateurs	23
3.6	Exercices	24
3.7	Corrigés	24
4	Structures de contrôle en PHP	26
4.1	Pourquoi des structures de contrôle ?	26
4.2	Les conditions	26
4.3	L'instruction <code>switch</code>	28
4.4	Opérateurs de comparaison et logiques	29
4.5	Les boucles	30
4.6	Maîtriser les boucles : <code>break</code> et <code>continue</code>	32
4.7	La fonction <code>date()</code>	33
4.8	Exercices	34
4.9	Corrigés	35
5	Tableaux et fonctions en PHP	39
5.1	Les tableaux indexés	39
5.2	Parcourir un tableau indexé	40
5.3	Les tableaux associatifs	41
5.4	Les tableaux à plusieurs dimensions	42
5.5	Fonctions utiles sur les tableaux	43

5.6	Les fonctions	44
5.7	Quelques fonctions PHP utiles	45
5.8	Exemples complets	46
5.9	Exercices	47
5.10	Corrigés	48
6	Formulaires et variables superglobales	53
6.1	Le rôle d'un formulaire	53
6.2	Les variables superglobales	53
6.3	Méthodes GET et POST	54
6.4	Récupérer les valeurs d'un formulaire	55
6.5	Valider et sécuriser les données	56
6.6	Envoyer le formulaire vers la même page	56
6.7	Exemple fil rouge : gestion d'un produit	57
6.8	Téléverser un fichier : <code>\$_FILES</code>	58
6.9	Exercices	59
6.10	Corrigés	60
7	PHP et MySQL : les bases de données	63
7.1	La base de données relationnelle	63
7.2	Le langage SQL	64
7.3	phpMyAdmin	66
7.4	Se connecter à MySQL depuis PHP : <code>mysqli</code>	66
7.5	Exécuter une requête : <code>mysqli_query</code>	67
7.6	Lire les résultats d'un <code>SELECT</code>	67
7.7	Insérer, modifier, supprimer depuis PHP	68
7.8	Fermer la connexion : <code>mysqli_close</code>	70
7.9	Application fil rouge : gestion des élèves	70
7.10	Exercices	71
7.11	Corrigés	72
8	Introduction au langage C	76
8.1	Qu'est-ce que le langage C ?	76
8.2	Structure d'un programme C	76
8.3	Compiler et exécuter : les IDE	77
8.4	Les types de base	78
8.5	Les constantes	78
8.6	Les entrées/sorties : <code>printf</code> et <code>scanf</code>	79
8.7	Les opérateurs	80
8.8	Exemples complets	81
8.9	Exercices	82
8.10	Corrigés	83
9	Structures de contrôle en C	87
9.1	Pourquoi des structures de contrôle ?	87
9.2	Les alternatives	87
9.3	Le <code>switch...case</code>	90
9.4	Les booléens en C	91
9.5	Les boucles	91
9.6	<code>break</code> et <code>continue</code>	93
9.7	Exemples complets et tracés	94
9.8	Exercices	96
9.9	Corrigés	97

10 Les fonctions en C	103
10.1 Pourquoi des fonctions ?	103
10.2 Déclarer une fonction	103
10.3 Appeler une fonction	104
10.4 Où placer les fonctions : le prototype	105
10.5 Variables locales et globales : la portée	106
10.6 Quelques fonctions utiles	107
10.7 La récursivité	108
10.8 Exercices	110
10.9 Corrigés	111
11 Tableaux, tri et enregistrements en C	115
11.1 Les tableaux à une dimension	115
11.2 Algorithmes classiques sur un tableau	117
11.3 Le tri par sélection	119
11.4 Les tableaux à deux dimensions (matrices)	120
11.5 Les enregistrements (struct)	121
11.6 Exemple complet : la fiche élève	123
11.7 Exercices	124
11.8 Corrigés	124
12 Projet guidé : la calculatrice (et plus)	129
12.1 La démarche de projet	129
12.2 Projet 1 — La calculatrice en C	130
12.3 Projet 2 — Gestion des notes d’une classe	133
12.4 Exercices	135
12.5 Corrigés	136
A Aide-mémoire	139
A.1 Aide-mémoire PHP	139
A.2 Aide-mémoire SQL	143
A.3 Aide-mémoire mysqli (PHP + MySQL)	144
A.4 Aide-mémoire langage C	145
A.5 Correspondance PHP ↔ C	148
B Sujets type Baccalauréat TI (corrigés)	149
B.1 Bac Blanc 2025 — Collège F.-X. Vogt	149
B.2 Sujet TI n° 1 — Bibliothèque scolaire	158
B.3 Sujet TI n° 2 — Pharmacie	166
B.4 Sujet TI n° 3 — Cybercafé	175
B.5 Sujet TI n° 4 — Gestion des notes	182
B.6 Sujet TI n° 5 — Boutique en ligne	188

Pages web et formulaires (rappels)

Au programme

Avant de programmer côté serveur en PHP, il faut maîtriser le socle qui s'exécute dans le navigateur de l'élève : le **HTML** (structure de la page), les **formulaires** (saisie de l'utilisateur) et le **JavaScript** (vérification de la saisie). C'est la base de tout site web dynamique.

1.1 Structure d'une page HTML

Définition

HTML (*HyperText Markup Language*) est le langage de **balisage** qui décrit la structure d'une page web. Une balise s'écrit entre chevrons : une balise **ouvrante** `<p>` et une balise **fermante** `</p>` entourent un contenu. Le navigateur lit le HTML et l'**affiche**.

Toute page commence par une déclaration `<!DOCTYPE html>` qui indique qu'il s'agit d'une page HTML moderne. Le document se divise en deux parties : le `<head>` (informations sur la page : titre, encodage) et le `<body>` (le contenu visible).

HTML

```

1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4   <meta charset="utf-8"           <!-- accents : é è à ç -->
5   <title>Lycée Bilingue de Bafoussam</title>
6 </head>
7 <body>
8   <h1>Bienvenue au lycée</h1>    <!-- titre principal -->
9   <h2>Classe de Terminale TI</h2> <!-- sous-titre -->
10  <p>Cette page presente nos eleves.</p> <!-- paragraphe -->
11  <a href="cours.html">Voir les cours</a> <!-- lien -->
12 </body>
13 </html>

```

Squelette minimal d'une page HTML : doctype, en-tête et corps.

Propriété Balises courantes

`<h1>` à `<h6>` : titres (du plus grand au plus petit); `<p>` : paragraphe; `<br>` : saut de ligne;
`<a href="...">` : lien; `` : image; `<ul>` / `<ol>` avec `<li>` : listes;

`<div>` et `<span>` : conteneurs neutres.

⚠ Attention. Certaines balises sont **vides** (sans contenu) et ne se ferment pas : `<br>` , `<img>` , `<input>` , `<meta>` . Les autres doivent **toujours** être refermées : à chaque `<p>` correspond un `</p>` .

1.2 Les formulaires HTML

Un formulaire permet à l'utilisateur (un élève, un parent...) de **saisir** des informations puis de les **envoyer** au serveur. C'est par lui que les données entrent dans une application web.

1.2.1 La balise `<form>`

```
<form action="traitement.php" method="post"> ...champs ...</form>
```

- `action` : l'adresse du fichier (souvent un script PHP) qui va **traiter** les données envoyées ;
- `method` : la façon d'envoyer les données. `post` (les données sont cachées, recommandé pour les formulaires) ou `get` (les données apparaissent dans l'URL).

⚠ Attention. Chaque champ destiné à être envoyé **doit** posséder un attribut `name` . C'est ce nom que le serveur (PHP) utilisera pour récupérer la valeur. Un champ **sans** `name` n'est pas transmis.

1.2.2 Les champs de saisie : `<input>`

La balise `<input>` change de comportement selon son attribut `type` .

type	Rôle
<code>text</code>	saisie d'une ligne de texte (nom, ville...)
<code>password</code>	saisie masquée (mot de passe : ••••)
<code>number</code>	nombre uniquement (âge, note)
<code>email</code>	adresse e-mail (format vérifié par le navigateur)
<code>radio</code>	bouton radio : un seul choix dans un groupe
<code>checkbox</code>	case à cocher : choix multiples
<code>submit</code>	bouton qui envoie le formulaire

HTML

```
<form action="inscription.php" method="post">
  <!-- Champ texte avec une étiquette cliquable -->
  <label for="nom">Nom de l'élève :</label>
  <input type="text" id="nom" name="nom">
  <br>

  <!-- Mot de passe (saisie masquée) -->
  <label for="mdp">Mot de passe :</label>
  <input type="password" id="mdp" name="mdp">
  <br>
```



```

12
13 <!-- Nombre : l'âge -->
14 <label for="age">Âge :</label>
15 <input type="number" id="age" name="age">
16 <br>
17
18 <!-- Adresse e-mail -->
19 <label for="mail">E-mail :</label>
20 <input type="email" id="mail" name="mail">
21 <br>
22
23 <!-- Boutons radio : un SEUL choix (même name) -->
24 Sexe :
25 <input type="radio" name="sexe" value="M"> Masculin
26 <input type="radio" name="sexe" value="F"> Féminin
27 <br>
28
29 <!-- Case à cocher -->
30 <input type="checkbox" name="interne" value="oui"> Élève interne
31 <br>
32
33 <!-- Bouton d'envoi -->
34 <input type="submit" value="S'inscrire">
35 </form>

```

Un formulaire d'inscription complet utilisant plusieurs types de champs.

💡 Remarque. La balise `<label for="nom">` est reliée au champ dont l' `id` vaut `nom`. Cliquer sur l'étiquette place alors le curseur dans le champ : c'est plus pratique, surtout sur téléphone.

1.2.3 Liste déroulante : `<select>`

Pour proposer un choix dans une liste fixe (la série, la classe...), on utilise `<select>` qui contient des `<option>`.

HTML

```

1 <label for="serie">Série :</label>
2 <select id="serie" name="serie">
3   <option value="TI">Terminale TI</option>
4   <option value="C">Terminale C</option>
5   <option value="D">Terminale D</option>
6 </select>

```

La valeur envoyée est celle de l'attribut `value` de l'option choisie.

1.2.4 Zone de texte : `<textarea>`

Pour un texte long (une remarque, une adresse complète), on emploie `<textarea>`, qui s'écrit sur plusieurs lignes.

HTML

```

1 <label for="obs">Observations :</label><br>
2 <textarea id="obs" name="obs" rows="4" cols="40">
3 Écrivez ici...
4 </textarea>

```

1.3 Les tableaux HTML

Définition

Un **tableau** HTML organise des données en lignes et colonnes. Il se construit avec `<table>` (le tableau), `<tr>` (*table row* : une ligne), `<th>` (*table header* : cellule d'en-tête) et `<td>` (*table data* : cellule ordinaire).

HTML

```
1 <table border="1">
2   <!-- Première ligne : les en-têtes -->
3   <tr>
4     <th>Matière</th>
5     <th>Note /20</th>
6   </tr>
7   <!-- Lignes de données -->
8   <tr>
9     <td>Informatique</td>
10    <td>16</td>
11  </tr>
12  <tr>
13    <td>Mathématiques</td>
14    <td>13</td>
15  </tr>
16 </table>
```

Un tableau de notes : une ligne d'en-tête (`<th>`) puis les données (`<td>`).

💡 **Remarque.** L'attribut `border="1"` dessine une bordure simple. Dans un vrai site, on préfère colorer et espacer les cellules avec du **CSS**, mais `border` suffit pour visualiser la structure pendant l'apprentissage.

1.4 JavaScript : les bases

Définition

JavaScript (abrégié **JS**) est un langage de programmation qui s'exécute **dans le navigateur**, donc **côté client**. Il sert à rendre la page *interactive* : réagir à un clic, vérifier un formulaire, modifier le contenu affiché.

On insère du JavaScript dans la balise `<script>`, généralement juste avant `</body>`.

```
<script>  instructions JavaScript...  </script>
```

1.4.1 Variables et types

On déclare une variable avec `let` (modifiable), `const` (constante, non modifiable) ou l'ancien mot-clé `var`. JavaScript devine le type tout seul.

JavaScript

```
<script>
1  let nom = "Awa";           // chaîne de caractères
2  let age = 17;              // nombre
3  const PI = 3.14;           // constante (ne change plus)
4  let estAdmis = true;       // booléen (true ou false)
5  // Afficher dans la console du navigateur
6  console.log(nom, age, estAdmis);
7
8 </script>
```

⚠ Attention. En JavaScript, chaque instruction se termine par un point-virgule ; et les mots-clés s'écrivent en **minuscules**. Let ou LET provoquent une erreur : il faut écrire let .

1.4.2 Opérateurs

Catégorie	Opérateurs	Exemple
Arithmétiques	+ - * / %	age + 1
Comparaison	== != < > <= >=	note >= 10
Égalité stricte	=== !==	type === "number"
Logiques	&& !	x>0 && x<20

💡 Remarque. Le + sert aussi à **coller** deux chaînes : "Bon" + "jour" donne "Bonjour" . C'est la *concaténation*.

1.4.3 Fonctions, condition et boucle

Une **fonction** regroupe des instructions sous un nom, pour les réutiliser. Elle peut **retourner** un résultat avec `return` .

JavaScript

```
<script>
1  // Fonction qui calcule une moyenne de deux notes
2  function moyenne(n1, n2) {
3      return (n1 + n2) / 2;
4  }
5
6
7  let m = moyenne(12, 16); // m vaut 14
8
9  // Condition : if ... else
10 if (m >= 10) {
11     console.log("Admis avec " + m);
12 } else {
13     console.log("Recalé");
14 }
15
16 // Boucle for : compter de 1 à 5
17 for (let i = 1; i <= 5; i = i + 1) {
18     console.log("Élève numéro " + i);
19 }
20 </script>
```

Fonction, condition `if/else` et boucle `for` : les briques de base.

1.5 JavaScript et la page

JavaScript devient vraiment utile quand il **réagit** aux actions de l'utilisateur et **lit** le contenu de la page.

1.5.1 Les événements

Un **événement** est une action de l'utilisateur (clic, envoi de formulaire). On y associe du code grâce à des attributs HTML :

- `onclick` : se déclenche au clic sur un élément (souvent un bouton) ;
- `onsubmit` : se déclenche quand on envoie un formulaire.

1.5.2 Lire un champ et afficher un message

Pour récupérer ce que l'élève a tapé, on retrouve le champ par son `id`, puis on lit sa propriété `.value`. La fonction `alert(...)` affiche, elle, une petite fenêtre d'information.

```
document.getElementById("id").value renvoie le texte saisi dans le champ dont l'attribut id
vaut "id" .
```

HTML

```
1 <input type="text" id="ville" name="ville">
2 <button onclick="direBonjour()">Saluer</button>
3
4 <script>
5   function direBonjour() {
6     // On lit la valeur tapée dans le champ "ville"
7     let v = document.getElementById("ville").value;
8     alert("Bonjour, élève de " + v + " !");
9   }
10 </script>
```

Au clic, la fonction lit le champ `ville` et affiche un message avec `alert`.

Tester du JavaScript. (1) Enregistre le fichier avec l'extension `.html` ; (2) ouvre-le directement dans un navigateur (Chrome, Firefox) ; (3) ouvre la **console** (touche `F12`) pour voir les messages `console.log` et les éventuelles erreurs.

1.6 Contrôle de saisie d'un formulaire

Définition

Le **contrôle de saisie** (ou *validation*) consiste à vérifier, avec JavaScript, que les informations tapées par l'utilisateur sont **correctes avant** de les envoyer au serveur. Cela évite d'enregistrer des données fausses ou vides.

1.6.1 Le principe : bloquer l'envoi

On relie une fonction de vérification à l'événement `onsubmit` du formulaire. **Règle d'or** : si la fonction renvoie `false`, le formulaire **n'est pas envoyé** ; si elle renvoie `true`, l'envoi continue.

```
<form onsubmit="return verifier()" ...> : l'envoi est annulé quand verifier() renvoie false .
```

1.6.2 Les vérifications utiles

- **Champ vide** : comparer la valeur à la chaîne vide (`valeur === ""`);
- **Valeur numérique** : convertir avec `Number(valeur)` et tester avec `isNaN(...)` (*is Not a Number*, « n'est pas un nombre »);
- **Longueur** : la propriété `.length` donne le nombre de caractères (`mdp.length < 6`);
- **Format simple** : chercher un caractère, par exemple le `@` d'un e-mail avec `valeur.indexOf("@")` (vaut `-1` s'il est absent).

1.6.3 Exemple complet

Voici un formulaire d'inscription complet avec sa fonction de validation. La fonction vérifie successivement chaque champ et renvoie `false` dès qu'un problème est détecté, ce qui **bloque** l'envoi.

HTML

```
<!DOCTYPE html>
<html lang="fr">
<head><meta charset="utf-8"><title>Inscription</title></head>
<body>

<h2>Inscription - Lycée de Garoua</h2>

<!-- L'envoi n'a lieu que si verifier() renvoie true -->
<form action="inscription.php" method="post"
      onsubmit="return verifier()">

  <label for="nom">Nom :</label>
  <input type="text" id="nom" name="nom"><br>

  <label for="age">Âge :</label>
  <input type="text" id="age" name="age"><br>

  <label for="mdp">Mot de passe :</label>
  <input type="password" id="mdp" name="mdp"><br>

  <input type="submit" value="S'inscrire">
</form>

<script>
function verifier() {
  // 1) Lire les valeurs tapées par l'élève
  let nom = document.getElementById("nom").value;
  let age = document.getElementById("age").value;
  let mdp = document.getElementById("mdp").value;

  // 2) Le nom ne doit pas être vide
  if (nom === "") {
    alert("Le nom est obligatoire.");
    return false;      // on bloque l'envoi
  }

  // 3) L'âge doit être un nombre strictement positif
```

```

38     if (isNaN(age) || age === "") {
39         alert("L'âge doit être un nombre.");
40         return false;
41     }
42     if (Number(age) <= 0) {
43         alert("L'âge doit être positif.");
44         return false;
45     }
46
47     // 4) Le mot de passe doit faire au moins 6 caractères
48     if (mdp.length < 6) {
49         alert("Mot de passe : 6 caractères minimum.");
50         return false;
51     }
52
53     // 5) Tout est correct : on autorise l'envoi
54     return true;
55 }
56 </script>
57
58 </body>
59 </html>

```

Formulaire + fonction `verifier()` : chaque `return false` empêche l'envoi.

⚠ Attention. La validation JavaScript améliore le confort de l'élève, mais elle s'exécute dans son navigateur : un utilisateur malintentionné peut la contourner. Les données devront **toujours** être revérifiées côté serveur (en PHP, au chapitre sur les formulaires).

1.7 Exercices

► Application

Exercice 1 Page de présentation

★ Écris une page HTML complète (doctype, head, body) qui affiche le titre « *Lycée Technique de Douala* », un sous-titre « *Terminale TI* » et un paragraphe de présentation de ton choix.

Exercice 2 Formulaire d'inscription d'un élève

★ Crée un formulaire (méthode `post`, action `inscrire.php`) avec : un champ texte *Nom*, un champ texte *Prénom*, un champ `number` *Âge*, une liste déroulante *Série* (TI, C, D) et un bouton d'envoi. Chaque champ doit avoir un `name` et un `label`.

Exercice 3 Tableau des notes

★ Construis un tableau HTML à deux colonnes (*Matière*, *Note /20*) et trois lignes de données : Informatique 15, Français 11, Physique 13. Ajoute une ligne d'en-tête avec `<th>`.

Exercice 4 Choix multiples

★★ Réalise un formulaire demandant le *sexe* (deux boutons radio M/F) et les *options choisies* (cases à cocher : Sport, Musique, Club informatique). Termine par un bouton d'envoi.

Exercice 5 Saluer l'élève

★★ Crée un champ texte (`id="prenom"`) et un bouton. Au clic sur le bouton, une fonction JavaScript lit le prénom et affiche, avec `alert`, le message « *Bienvenue, ... !* ».

► Entraînement

Exercice 6 Validation de l'âge

★★ À partir d'un formulaire contenant un champ *Âge*, écris une fonction JavaScript reliée à `onsubmit` qui bloque l'envoi si l'âge n'est pas un nombre ou s'il n'est pas strictement positif.

Exercice 7 Deux mots de passe identiques

★★ Crée un formulaire avec deux champs `password` (*Mot de passe* et *Confirmation*). Écris une fonction qui bloque l'envoi si les deux valeurs sont différentes, ou si le mot de passe fait moins de 6 caractères.

Exercice 8 Calculer une moyenne

★★ Trois champs `number` contiennent trois notes. Au clic sur un bouton, une fonction JavaScript calcule la moyenne et l'affiche avec `alert`, en précisant « *Admis* » si la moyenne est ≥ 10 , sinon « *Recalé* ».

Exercice 9 E-mail valide (format simple)

★★★ Dans un formulaire d'inscription, ajoute un champ *e-mail*. Écris une fonction de validation qui bloque l'envoi si le champ est vide ou s'il ne contient pas le caractère `@` (utilise `indexOf`).

Exercice 10 Inscription complète vérifiée

★★★ Reprends le formulaire de l'exercice 2 et écris une seule fonction `verifier()` qui contrôle : nom non vide, prénom non vide, âge numérique et positif. L'envoi ne doit avoir lieu que si tout est correct.

1.8 Corrigés

► Corrigé de l'exercice 1

HTML

```

1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4   <meta charset="utf-8">
5   <title>Lycée Technique de Douala</title>
6 </head>
7 <body>
8   <h1>Lycée Technique de Douala</h1>
9   <h2>Terminale TI</h2>
10  <p>Notre classe forme les futurs techniciens en informatique
11    de la ville de Douala.</p>
12 </body>
13 </html>

```

► Corrigé de l'exercice 2

HTML

```

1 <form action="inscrire.php" method="post">
2   <label for="nom">Nom :</label>
3   <input type="text" id="nom" name="nom"><br>
4
5   <label for="prenom">Prénom :</label>
6   <input type="text" id="prenom" name="prenom"><br>
7
8   <label for="age">Âge :</label>

```

```

9   <input type="number" id="age" name="age"><br>
10
11   <label for="serie">Série :</label>
12   <select id="serie" name="serie">
13     <option value="TI">Terminale TI</option>
14     <option value="C">Terminale C</option>
15     <option value="D">Terminale D</option>
16   </select><br>
17
18   <input type="submit" value="S'inscrire">
19 </form>

```

► Corrigé de l'exercice 3

HTML

```

1 <table border="1">
2   <tr>
3     <th>Matière</th>
4     <th>Note /20</th>
5   </tr>
6   <tr><td>Informatique</td><td>15</td></tr>
7   <tr><td>Français</td><td>11</td></tr>
8   <tr><td>Physique</td><td>13</td></tr>
9 </table>

```

► Corrigé de l'exercice 4

HTML

```

1 <form action="choix.php" method="post">
2   Sexe :
3   <input type="radio" name="sexe" value="M"> Masculin
4   <input type="radio" name="sexe" value="F"> Féminin
5   <br>
6   Options :
7   <input type="checkbox" name="sport" value="oui"> Sport
8   <input type="checkbox" name="musique" value="oui"> Musique
9   <input type="checkbox" name="club" value="oui"> Club informatique
10  <br>
11  <input type="submit" value="Valider">
12 </form>

```

► Corrigé de l'exercice 5

HTML

```

1 <input type="text" id="prenom" name="prenom">
2 <button onclick="saluer()">Saluer</button>
3
4 <script>
5   function saluer() {
6     // Lecture du prénom tapé
7     let p = document.getElementById("prenom").value;
8     alert("Bienvenue, " + p + " !");
9   }
10 </script>

```


► Corrigé de l'exercice 6

HTML

```
1 <form action="age.php" method="post" onsubmit="return verifierAge()">
2   <label for="age">Âge :</label>
3   <input type="text" id="age" name="age">
4   <input type="submit" value="Envoyer">
5 </form>
6
7 <script>
8   function verifierAge() {
9     let age = document.getElementById("age").value;
10    // Doit être un nombre
11    if (isNaN(age) || age === "") {
12      alert("L'âge doit être un nombre.");
13      return false;
14    }
15    // Doit être strictement positif
16    if (Number(age) <= 0) {
17      alert("L'âge doit être positif.");
18      return false;
19    }
20    return true; // tout est correct : envoi autorisé
21  }
22 </script>
```

► Corrigé de l'exercice 7

HTML

```
1 <form action="compte.php" method="post" onsubmit="return verifierMdp()">
2   <label for="mdp">Mot de passe :</label>
3   <input type="password" id="mdp" name="mdp"><br>
4
5   <label for="conf">Confirmation :</label>
6   <input type="password" id="conf" name="conf"><br>
7
8   <input type="submit" value="Créer le compte">
9 </form>
10
11 <script>
12   function verifierMdp() {
13     let mdp = document.getElementById("mdp").value;
14     let conf = document.getElementById("conf").value;
15
16     // Longueur minimale
17     if (mdp.length < 6) {
18       alert("Le mot de passe doit faire au moins 6 caractères.");
19       return false;
20     }
21     // Les deux saisies doivent être identiques
22     if (mdp !== conf) {
23       alert("Les deux mots de passe sont différents.");
24       return false;
25     }
26     return true;
27   }
28 </script>
```

► Corrigé de l'exercice 8

HTML

```

1 N1 : <input type="number" id="n1"><br>
2 N2 : <input type="number" id="n2"><br>
3 N3 : <input type="number" id="n3"><br>
4 <button onclick="calculer()">Calculer la moyenne</button>
5
6 <script>
7   function calculer() {
8     // On convertit chaque saisie en nombre
9     let n1 = Number(document.getElementById("n1").value);
10    let n2 = Number(document.getElementById("n2").value);
11    let n3 = Number(document.getElementById("n3").value);
12
13    let moy = (n1 + n2 + n3) / 3;
14
15    if (moy >= 10) {
16      alert("Moyenne : " + moy + " - Admis");
17    } else {
18      alert("Moyenne : " + moy + " - Recalé");
19    }
20  }
21 </script>

```

► Corrigé de l'exercice 9

HTML

```

1 <form action="inscrire.php" method="post" onsubmit="return verifierMail()">
2   <label for="mail">E-mail :</label>
3   <input type="text" id="mail" name="mail">
4   <input type="submit" value="Envoyer">
5 </form>
6
7 <script>
8   function verifierMail() {
9     let mail = document.getElementById("mail").value;
10
11    // Champ vide ?
12    if (mail === "") {
13      alert("L'e-mail est obligatoire.");
14      return false;
15    }
16    // Le caractère @ est-il présent ? indexOf vaut -1 sinon
17    if (mail.indexOf("@") === -1) {
18      alert("E-mail invalide : le caractère @ est manquant.");
19      return false;
20    }
21    return true;
22  }
23 </script>

```

► Corrigé de l'exercice 10

HTML

```

1 <form action="inscrire.php" method="post" onsubmit="return verifier()">
2   <label for="nom">Nom :</label>
3   <input type="text" id="nom" name="nom"><br>
4
5   <label for="prenom">Prénom :</label>
6   <input type="text" id="prenom" name="prenom"><br>
7
8   <label for="age">Âge :</label>
9   <input type="text" id="age" name="age"><br>
10
11   <input type="submit" value="S'inscrire">
12 </form>
13
14 <script>
15   function verifier() {
16     let nom = document.getElementById("nom").value;
17     let prenom = document.getElementById("prenom").value;
18     let age = document.getElementById("age").value;
19
20     // Nom et prénom obligatoires
21     if (nom === "") {
22       alert("Le nom est obligatoire.");
23       return false;
24     }
25     if (prenom === "") {
26       alert("Le prénom est obligatoire.");
27       return false;
28     }
29     // Âge : un nombre strictement positif
30     if (isNaN(age) || age === "") {
31       alert("L'âge doit être un nombre.");
32       return false;
33     }
34     if (Number(age) <= 0) {
35       alert("L'âge doit être positif.");
36       return false;
37     }
38     return true;    // formulaire correct : envoi autorisé
39   }
40 </script>

```

★ À retenir

Le **HTML** structure la page, le **formulaire** (`<form>` , champs `<input>` , attribut `name`) recueille la saisie, et le **JavaScript** la **contrôle** dans le navigateur. La règle clé du contrôle de saisie : `onsubmit="return verifier()"` et un `return false` pour **bloquer** l'envoi tant qu'un champ est invalide. Mais ce contrôle reste côté client : on revérifiera toujours côté serveur.

L'environnement de développement web

Au programme

Comprendre l'architecture **client/serveur** et le cycle requête/réponse HTTP, identifier le rôle d'un **serveur web** (Apache, Nginx, IIS), installer et configurer un **environnement intégré** (WampServer, XAMPP, LAMP), repérer le **répertoire racine** et le **port d'écoute**, puis **héberger un site en local** avant d'évoquer la mise en ligne.

2.1 L'architecture client/serveur

Tout site web dynamique repose sur un dialogue entre deux acteurs : un **client** et un **serveur**. Avant d'écrire la moindre ligne de PHP, il faut bien comprendre qui fait quoi.

Définition

Dans le modèle **client/serveur**, le **client** demande un service et le **serveur** le fournit. Sur le Web, le client est le **navigateur** (Chrome, Firefox, Edge...) et le serveur est un **serveur web** qui héberge les pages et répond aux demandes.

2.1.1 Le client : le navigateur

Le navigateur est le logiciel installé sur la machine de l'utilisateur. Son rôle :

- envoyer une **requête** lorsqu'on saisit une adresse (URL) ou qu'on clique sur un lien ;
- recevoir la **réponse** du serveur (du code HTML, des images, du CSS...);
- **afficher** la page à l'écran après l'avoir interprétée.

Le navigateur ne « voit » jamais le code PHP : il ne reçoit que du HTML déjà fabriqué par le serveur.

2.1.2 Le serveur web

Définition

Un **serveur web** est un logiciel (et la machine qui l'héberge) qui reste en permanence à l'écoute des requêtes des clients, recherche la ressource demandée (une page, une image...) et la renvoie au navigateur.

Les serveurs web les plus répandus sont :

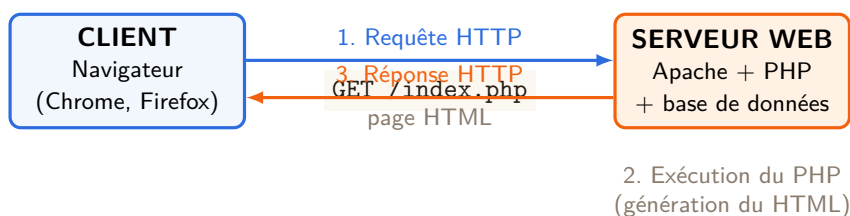
- **Apache** (*Apache HTTP Server*) : libre, gratuit, le plus utilisé pour apprendre et pour de nombreux sites ;
- **Nginx** : réputé pour sa rapidité et sa faible consommation, très utilisé sur les gros sites ;
- **IIS** (*Internet Information Services*) : le serveur web de Microsoft, intégré à Windows Server.

💡 **Remarque.** Le mot « serveur » désigne tantôt le *logiciel* (Apache), tantôt la *machine* (un ordinateur puissant allumé en permanence). Sur ton propre PC, le même ordinateur peut jouer à la fois le rôle de client *et* de serveur : c'est exactement ce qu'on fait quand on installe WampServer pour tester un site en local.

2.1.3 Le cycle requête / réponse HTTP

Le navigateur et le serveur communiquent grâce à un langage commun : le protocole **HTTP** (*HyperText Transfer Protocol*). L'échange suit toujours le même cycle en quatre temps.

- 1) **Requête.** L'utilisateur saisit `http://www.onbuch.cm/index.php`. Le navigateur envoie une **requête HTTP** au serveur (« donne-moi la page `index.php` »).
- 2) **Traitement.** Le serveur web reçoit la requête. Si la page contient du PHP, il fait **exécuter** le code par le préprocesseur PHP (qui peut interroger une base de données).
- 3) **Réponse.** Le serveur renvoie une **réponse HTTP** contenant la page **HTML** finale (le résultat, plus le code PHP).
- 4) **Affichage.** Le navigateur interprète le HTML reçu et affiche la page à l'utilisateur.



Le cycle requête/réponse HTTP : le client demande, le serveur exécute puis répond.

2.2 Les environnements intégrés de développement web

Pour développer un site PHP, il faut au minimum trois logiciels qui fonctionnent ensemble. Plutôt que de les installer un par un, on utilise un **environnement intégré** qui les regroupe en une seule installation.

Définition

Un **environnement intégré de développement web** est une suite logicielle qui réunit, prêts à l'emploi, un **serveur web**, un **SGBD** (système de gestion de base de données) et un **préprocesseur** de langage côté serveur.

2.2.1 Les trois composants

Quel que soit l'environnement choisi, on retrouve toujours les mêmes briques :

Composant	Logiciel typique	Rôle
Serveur web	Apache	Reçoit les requêtes, renvoie les pages
SGBD	MySQL / MariaDB	Stocke et gère les données (base de données)
Préprocesseur	PHP	Exécute le code et fabrique le HTML

💡 **Remarque.** On parle souvent d'**outil AMP** : **A**pache + **M**ySQL + **P**HP. La première lettre indique le système d'exploitation : **WAMP** (Windows), **LAMP** (Linux), **MAMP** (macOS).

2.2.2 Les environnements les plus courants

Environnement	Système	Répertoire racine	Particularité
WampServer	Windows	<code>www</code>	Très répandu dans les lycées
XAMPP	Multi-plateforme	<code>htdocs</code>	Marche sous Windows, Linux, macOS
EasyPHP	Windows	<code>www</code>	Léger, simple à installer
LAMP	Linux	<code>/var/www/html</code>	Installé paquet par paquet

💡 **Remarque.** **LAMP** n'est pas un logiciel unique à télécharger : c'est l'ensemble **Linux** + **Apache** + **MySQL** + **PHP** qu'on installe généralement avec le gestionnaire de paquets (par exemple `sudo apt install apache2 mysql-server php`). WampServer, XAMPP et EasyPHP, eux, s'installent en un seul fichier d'installation.

2.3 Installer et configurer l'environnement

L'installation d'un environnement comme WampServer ou XAMPP se résume à télécharger le programme depuis le site officiel, puis à suivre l'assistant. Une fois installé, le logiciel place une icône dans la **zone de notification** (à côté de l'horloge) qui permet de piloter les services.

Installer WampServer et tester l'installation.

- 1) **Télécharger** WampServer depuis le site officiel (`wampserver.com`) en choisissant la version 32 ou 64 bits qui correspond à ton Windows.
- 2) **Lancer l'installateur** et accepter les options par défaut (dossier d'installation `C:\wamp64`). Laisser installer les composants Apache, MySQL et PHP.
- 3) **Démarrer WampServer** : une icône apparaît près de l'horloge. Elle devient **verte** quand tous les services sont actifs (orange = un service manque, rouge = aucun service).
- 4) **Tester** : ouvrir le navigateur et saisir `http://localhost` . La page d'accueil de WampServer doit s'afficher : l'installation est réussie.
- 5) **Créer une première page** : déposer un fichier `essai.php` dans le dossier `www` , puis ouvrir `http://localhost/essai.php` .

⚠ **Attention.** Sur certains réseaux de lycées ou de cybercafés, un autre logiciel (Skype, un antivirus, ou un autre serveur) peut déjà utiliser le port 80 et empêcher Apache de démarrer (icône orange). Dans ce cas, il faut soit fermer ce logiciel, soit **changer le port d'écoute** d'Apache (voir plus loin).

2.4 Démarrer et arrêter les services

Un **service** est un logiciel qui tourne en arrière-plan. Dans WampServer ou XAMPP, les deux services essentiels sont **Apache** (le serveur web) et **MySQL** (le SGBD).

- **WampServer** : un clic *gauche* sur l'icône ouvre un menu avec *Start All Services*, *Stop All Services* et *Restart All Services*. Le menu *Apache* (et *MySQL*) permet aussi de démarrer ou arrêter chaque service séparément.
- **XAMPP** : le *Control Panel* affiche chaque module avec un bouton *Start / Stop*. Un voyant vert signale qu'un service est démarré.

💡 **Remarque.** Il faut **redémarrer** (*Restart*) Apache à chaque fois qu'on modifie un fichier de configuration (comme `httpd.conf`), sinon le changement n'est pas pris en compte. En revanche, modifier une simple page `.php` ne nécessite aucun redémarrage : il suffit de recharger la page dans le navigateur.

2.5 Le répertoire racine

Définition

Le **répertoire racine** (*document root*) est le dossier dans lequel le serveur web va chercher les pages à servir. Tout fichier placé dans ce dossier devient accessible depuis le navigateur via `http://localhost/`.

Le nom du répertoire racine dépend de l'environnement :

- **WampServer** : `www` (par exemple `C:\wamp64\www`);
- **XAMPP** : `htdocs` (par exemple `C:\xampp\htdocs`);
- **LAMP (Linux)** : `/var/www/html`.

Exemple. Sous WampServer, le fichier `C:\wamp64\www\bulletin.php` est accessible à l'adresse `http://localhost/bulletin.php`. Si on le range dans un sous-dossier `lycee`, l'adresse devient `http://localhost/lycee/bulletin.php`.

⚠ **Attention.** Une page placée **en dehors** du répertoire racine (par exemple sur le Bureau) ne pourra **pas** être servie par Apache : le navigateur ne la trouvera pas à l'adresse `http://localhost/`. Place toujours tes fichiers `.php` dans `www` (ou `htdocs`).

2.6 Le port d'écoute

Définition

Un **port** est un numéro qui identifie un service réseau sur une machine. Le serveur web Apache « écoute » par défaut sur le **port 80**, le port standard du protocole HTTP. C'est pourquoi on peut écrire `http://localhost` sans préciser de numéro.

💡 **Remarque.** Quand aucun port n'est précisé, le navigateur utilise automatiquement le port 80 pour `http`. Écrire `http://localhost` revient donc exactement à écrire `http://localhost:80`.

2.6.1 Identifier et modifier le port

Le port d'écoute d'Apache est défini dans son fichier de configuration principal, `httpd.conf`. On y trouve une ligne `Listen` :

Environnement	Emplacement de <code>httpd.conf</code>
WampServer	<code>C:\wamp64\bin\apache\apache2.x.x\conf</code>
XAMPP	<code>C:\xampp\apache\conf</code>

Dans WampServer, on peut ouvrir directement ce fichier par le menu de l'icône : *Apache* → *httpd.conf*. La ligne à repérer est :

```
# Fichier httpd.conf - port d'ecoute d'Apache
Listen 80
```

*La directive **Listen** fixe le port sur lequel Apache attend les requêtes.*

Pour faire écouter Apache sur un autre port (par exemple le **8080**, souvent utilisé quand le port 80 est déjà occupé), on remplace la valeur :

```
# Apache ecoute desormais sur le port 8080
Listen 8080
```

Changer le port d'écoute d'Apache.

- 1) Ouvrir le fichier `httpd.conf` (menu *Apache*).
- 2) Remplacer la ligne `Listen 80` par `Listen 8080`.
- 3) Si elle existe, modifier aussi la ligne `ServerName localhost:80` en `ServerName localhost:8080`.
- 4) **Enregistrer** le fichier et **redémarrer** Apache.
- 5) Accéder désormais au site avec le port dans l'URL : `http://localhost:8080`.

⚠ Attention. Après avoir changé le port, l'adresse `http://localhost` (port 80) ne fonctionne plus : il faut impérativement préciser le nouveau port, comme dans `http://localhost:8080`. Et il ne faut pas oublier de redémarrer Apache, sinon l'ancien réglage reste actif.

2.7 Héberger un site

2.7.1 En local (sur sa propre machine)

Héberger un site « en local », c'est le faire fonctionner sur son propre ordinateur, sans connexion Internet. C'est la méthode idéale pour développer et tester avant la mise en ligne.

Héberger un site en local.

- 1) **Placer le dossier** du site dans le répertoire racine. Exemple : copier le dossier `notesonline` dans `C:\wamp64\www`.
- 2) **Démarrer les services** : lancer Apache (serveur web) et MySQL (si le site utilise une base de données). L'icône WampServer doit être verte.
- 3) **Ouvrir le navigateur** et saisir l'adresse du site : `http://localhost/notesonline/`. Le serveur cherche alors un fichier `index.php` (ou `index.html`) dans ce dossier.
- 4) **Naviguer** dans le site comme s'il était en ligne, mais visible uniquement depuis cette machine.

💡 Remarque. L'adresse `localhost` (ou son équivalent numérique `127.0.0.1`) désigne toujours « ma propre machine ». Un site en local n'est donc pas accessible depuis Internet : il faut un véritable hébergeur pour le rendre public.

2.7.2 En ligne (mise en production)

Une fois le site terminé et testé en local, on le publie sur Internet pour qu'il soit accessible à tous. Il faut alors deux choses :

- un **hébergeur** : une entreprise qui loue de l'espace sur un serveur web allumé en permanence et connecté à Internet (par exemple OVH, Ionos, ou un hébergeur local camerounais comme Camtel/Blue) ;
- un **nom de domaine** : l'adresse facile à retenir du site (par exemple `www.onbuch.cm`), à louer auprès d'un bureau d'enregistrement. Le suffixe `.cm` est réservé aux sites du Cameroun.

Exemple. Un lycée de Yaoundé développe son site de publication des résultats. Il le teste d'abord en local sous WampServer (`http://localhost/resultats/`). Une fois prêt, il loue le nom de domaine `lycee-bilingue.cm` et un hébergement, puis transfère ses fichiers `.php` sur le serveur de l'hébergeur (généralement par **FTP**). Le site est alors accessible depuis n'importe quel ordinateur du monde.

💡 **Remarque.** Pour transférer les fichiers vers l'hébergeur, on utilise le plus souvent le protocole **FTP** (*File Transfer Protocol*) avec un logiciel comme FileZilla, ou bien l'interface d'administration fournie par l'hébergeur.

2.8 Exercices

► Application

Exercice 1 Client ou serveur ?

★ Pour chaque élément, indique s'il joue le rôle de **client** ou de **serveur** : a) Mozilla Firefox ; b) Apache ; c) le navigateur d'un téléphone ; d) Nginx ; e) Google Chrome.

Exercice 2 Vrai ou faux ?

★ Réponds par vrai ou faux et justifie : a) le navigateur exécute le code PHP ; b) Apache est un serveur web ; c) HTTP est le protocole utilisé entre le navigateur et le serveur web ; d) le client envoie la réponse et le serveur la requête ; e) un site en local est visible depuis Internet.

Exercice 3 Associer composant et rôle

★ Associe chaque composant d'un environnement WAMP à son rôle :

- 1) Apache a) exécute le code et fabrique le HTML
- 2) MySQL b) stocke et gère les données
- 3) PHP c) reçoit les requêtes et renvoie les pages

Exercice 4 Le bon répertoire racine

★ Donne le nom du répertoire racine pour : a) WampServer ; b) XAMPP ; c) LAMP sous Linux. Où faut-il placer un fichier `accueil.php` sous WampServer pour qu'il soit accessible à `http://localhost/accueil.php` ?

Exercice 5 Décoder une URL

★★ On donne l'adresse `http://localhost:8080/ecole/notes.php`. Indique : a) le port d'écoute utilisé ; b) le sous-dossier du site dans le répertoire racine ; c) le nom du fichier demandé ; d) pourquoi le port `8080` apparaît ici alors qu'il n'apparaît pas dans `http://localhost/`.

► Entraînement

Exercice 6 Changer le port d'écoute

★★ Le port 80 est déjà occupé par un autre logiciel et Apache refuse de démarrer (icône orange). Décris la procédure complète pour faire écouter Apache sur le port 8081, en précisant le fichier modifié, la ligne à changer et l'adresse à utiliser ensuite.

Exercice 7 Procédure d'hébergement local

★★ Un camarade t'envoie un dossier `cantine` contenant son site PHP. Décris, étape par étape, ce que tu dois faire pour l'exécuter sur ta machine avec WampServer, depuis la copie du dossier jusqu'à l'affichage dans le navigateur.

Exercice 8 Le cycle requête/réponse

★★ Un élève saisit `http://www.onbuch.cm/resultats.php` dans son navigateur. Décris, dans l'ordre, les quatre étapes du cycle requête/réponse HTTP jusqu'à l'affichage de la page, en précisant à quel moment le code PHP est exécuté.

Exercice 9 De local à en ligne

★★★ Le club informatique d'un lycée de Douala a terminé son site sous XAMPP (`http://localhost/club/`).
a) Pourquoi ce site n'est-il pas encore visible par les élèves chez eux ? b) Cite les **deux** éléments à acquérir pour le mettre en ligne. c) Par quel moyen transfère-t-on les fichiers vers l'hébergeur ?

2.9 Corrigés

► Corrigé de l'exercice 1

Le **client** est le logiciel qui demande et affiche les pages (un navigateur) ; le **serveur** est le logiciel qui héberge et renvoie les pages.

- a) Mozilla Firefox → **client** (navigateur) ;
- b) Apache → **serveur** (serveur web) ;
- c) navigateur d'un téléphone → **client** ;
- d) Nginx → **serveur** (serveur web) ;
- e) Google Chrome → **client** (navigateur).

► Corrigé de l'exercice 2

- a) **Faux** : le PHP s'exécute sur le *serveur*, pas dans le navigateur.
- b) **Vrai** : Apache est bien un serveur web.
- c) **Vrai** : HTTP est le protocole d'échange entre client et serveur web.
- d) **Faux** : c'est le client qui envoie la *requête* et le serveur qui renvoie la *réponse*.
- e) **Faux** : un site en local (`localhost`) n'est visible que sur la machine elle-même, pas depuis Internet.

► Corrigé de l'exercice 3

- 1 → c) Apache reçoit les requêtes et renvoie les pages ;
- 2 → b) MySQL stocke et gère les données ;
- 3 → a) PHP exécute le code et fabrique le HTML.

► Corrigé de l'exercice 4

Le répertoire racine est : a) `www` pour WampServer ; b) `htdocs` pour XAMPP ; c) `/var/www/html` pour LAMP sous Linux. Sous WampServer, le fichier `accueil.php` doit être placé directement dans le dossier `www` (c'est-à-dire `C:\wamp64\www\accueil.php`) pour être accessible à `http://localhost/accueil.php` .

► Corrigé de l'exercice 5

Dans `http://localhost:8080/ecole/notes.php` :

- a) le port d'écoute est **8080** (la valeur après les deux-points) ;
- b) le sous-dossier du site est `ecole` (placé dans le répertoire racine) ;

- c) le fichier demandé est `notes.php` ;
- d) le port `8080` apparaît parce qu'Apache a été configuré sur ce port (au lieu du 80 par défaut). Dans `http://localhost/`, le port n'est pas écrit car le navigateur utilise automatiquement le port 80, valeur par défaut de HTTP.

► Corrigé de l'exercice 6

Procédure pour faire écouter Apache sur le port 8081 :

- 1) ouvrir le fichier de configuration `httpd.conf` d'Apache (menu *Apache* → *httpd.conf* dans Wamp-Server) ;
- 2) remplacer la ligne `Listen 80` par `Listen 8081` (et, si présente, `ServerName localhost:80` par `ServerName localhost:8081`) ;
- 3) enregistrer le fichier ;
- 4) redémarrer Apache (*Restart Services*) pour appliquer le changement ;
- 5) accéder au site avec le nouveau port : `http://localhost:8081/` .

► Corrigé de l'exercice 7

Pour exécuter le dossier `cantine` sous WampServer :

- 1) copier le dossier `cantine` dans le répertoire racine `C:\wamp64\www` ;
- 2) démarrer WampServer et lancer les services Apache (et MySQL si le site utilise une base de données) : l'icône doit devenir verte ;
- 3) ouvrir le navigateur et saisir `http://localhost/cantine/` ;
- 4) le serveur affiche le fichier `index.php` du dossier ; on peut alors naviguer dans le site.

► Corrigé de l'exercice 8

Cycle requête/réponse pour `http://www.onbuch.cm/resultats.php` :

- 1) **Requête** : le navigateur (client) envoie une requête HTTP au serveur pour demander la page `resultats.php` ;
- 2) **Traitement** : le serveur web reçoit la requête et fait **exécuter le code PHP** de la page (c'est à ce moment que le PHP s'exécute, éventuellement en interrogeant une base de données) ;
- 3) **Réponse** : le serveur renvoie au navigateur la page **HTML** produite par le PHP ;
- 4) **Affichage** : le navigateur interprète le HTML reçu et affiche la page à l'élève.

► Corrigé de l'exercice 9

- a) Le site n'est visible que sur la machine du club car `localhost` désigne cette seule machine ; il n'est pas accessible depuis Internet.
- b) Pour le mettre en ligne, il faut acquérir : un **hébergeur** (un serveur connecté à Internet en permanence) et un **nom de domaine** (par exemple `club-douala.cm`).
- c) On transfère les fichiers vers l'hébergeur par **FTP** (avec un logiciel comme FileZilla) ou via l'interface d'administration de l'hébergeur.

★ À retenir

Sur le Web, le **client** (navigateur) envoie une **requête HTTP** et le **serveur web** (Apache) renvoie une **réponse** : la page HTML. Un environnement intégré (WampServer, XAMPP, LAMP) réunit **Apache + MySQL + PHP**. On place ses pages dans le **répertoire racine** (`www` ou `htdocs`), Apache écoute par défaut sur le **port 80** (modifiable via `Listen` dans `httpd.conf`), et on teste tout en local sur `http://localhost/` avant la mise en ligne (hébergeur + nom de domaine).

Premiers pas en PHP

Au programme

Insérer du PHP dans une page HTML, comprendre où s'exécute un script, déclarer variables et constantes, utiliser les opérateurs et afficher des données avec `echo` et `print`. C'est le socle de toute application web dynamique.

3.1 Qu'est-ce que PHP ?

Définition

PHP (*PHP : Hypertext Preprocessor*) est un langage de script **exécuté côté serveur**. Le serveur interprète le code PHP, puis envoie au navigateur du client une page **HTML** ordinaire : le visiteur ne voit jamais le code PHP.

Remarque. Contrairement à JavaScript qui s'exécute dans le navigateur (côté *client*), PHP s'exécute sur le *serveur*. C'est pour cela qu'on peut, avec PHP, se connecter à une base de données ou lire des fichiers du serveur en toute sécurité.

3.2 Insérer du PHP dans une page

Un script PHP se place entre les balises `<?php` et `?>`. Le fichier doit porter l'extension `.php`.

```
<?php  instructions PHP...  ?>
```

PHP

```

1 <!DOCTYPE html>
2 <html lang="fr">
3 <head><meta charset="utf-8"><title>Ma première page</title></head>
4 <body>
5   <h1>Bonjour</h1>
6   <?php
7     // Ceci est un commentaire sur une ligne
8     echo "<p>Bienvenue sur OnBuch, il est " . date("H:i") . " !</p>";
9   ?>
10 </body>
11 </html>
```

Le code PHP est noyé dans le HTML ; seul son résultat est envoyé au client.

⚠ Attention. N'oublie jamais le point-virgule ; à la fin de chaque instruction PHP, ni la balise fermante `?>`. Un oubli provoque une erreur de syntaxe (*Parse error*).

3.3 Variables et constantes

Définition

Une **variable** est un espace mémoire nommé qui stocke une valeur modifiable. En PHP, le nom d'une variable commence **toujours** par le symbole `$`.

PHP

```
<?php
1  $nom = "Awa";           // chaîne de caractères
2  $age = 17;              // entier
3  $moyenne = 14.5;        // nombre décimal
4  $estAdmis = true;       // booléen
5  define("ETABLISSEMENT", "Lycée de Maroua"); // constante
6  echo "Élève : $nom ($age ans) - " . ETABLISSEMENT;
7
8  ?>
```

Propriété Règles de nommage

Un nom de variable commence par `$` suivi d'une lettre ou d'un `_`, puis de lettres, chiffres ou `_`. PHP distingue les majuscules des minuscules : `$nom` et `$Nom` sont deux variables différentes.

3.4 Afficher : echo et print

- `echo` affiche une ou plusieurs valeurs (séparées par des virgules);
- `print` affiche une seule valeur et renvoie `1` ;
- l'opérateur de concaténation `.` colle deux chaînes.

Exemple. `echo "Total : ", 3 + 4;` affiche `Total : 7`, tandis que `echo "3 + 4";` affiche littéralement `3 + 4`.

3.5 Les opérateurs

Catégorie	Opérateurs	Exemple
Arithmétiques	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code> <code>**</code>	<code>\$x % 2</code> (reste)
Comparaison	<code>==</code> <code>!=</code> <code><</code> <code>></code> <code><=</code> <code>>=</code> <code>===</code>	<code>\$a === \$b</code>
Logiques	<code>&&</code> <code> </code> <code>!</code>	<code>\$x > 0 && \$x < 10</code>
Affectation	<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>.=</code>	<code>\$s .= "!"</code>

Tester un script PHP en local. (1) Lancer le serveur (WampServer/XAMPP); (2) placer le fichier `.php` dans le dossier `www` (ou `htdocs`); (3) ouvrir `http://localhost/monfichier.php` dans le navigateur.

3.6 Exercices

► Application

Exercice 1 Première page dynamique

★ Écris un script PHP qui affiche dans une page HTML le message « *Nous sommes le ...* » suivi de la date du jour.

Exercice 2 Calculs et affichage

★ On donne `$a = 7` et `$b = 3`. Affiche, chacune sur une ligne, la somme, la différence, le produit, le quotient et le reste de la division de `$a` par `$b`.

Exercice 3 Bulletin

★★ Trois notes sont stockées dans `$n1`, `$n2`, `$n3`. Calcule et affiche la moyenne avec deux décimales (fonction `round`).

3.7 Corrigés

► Corrigé de l'exercice 1

PHP

```
<?php
1 echo "<p>Nous sommes le " . date("d/m/Y") . "</p>";
2 ?>
```

► Corrigé de l'exercice 2

PHP

```
<?php
1 $a = 7; $b = 3;
2 echo "Somme : ", $a + $b, "<br>";
3 echo "Différence : ", $a - $b, "<br>";
4 echo "Produit : ", $a * $b, "<br>";
5 echo "Quotient : ", $a / $b, "<br>";
6 echo "Reste : ", $a % $b;
7 ?>
```

► Corrigé de l'exercice 3

PHP

```
<?php
1 $n1 = 12; $n2 = 15; $n3 = 9;
2 $moyenne = ($n1 + $n2 + $n3) / 3;
3 echo "Moyenne : " . round($moyenne, 2);
4 ?>
```

★ À retenir

PHP s'exécute **côté serveur** entre `<?php` et `?>`. Les variables commencent par `$`, chaque instruction finit par `;`, et `echo` sert à renvoyer du HTML au navigateur.

Structures de contrôle en PHP

Au programme

Donner de l'intelligence à un script : prendre des décisions avec `if`, `else`, `elseif`, l'opérateur ternaire et `switch` ; répéter des traitements avec `while`, `do...while`, `for` et `foreach` ; maîtriser `break`, `continue` et la fonction `date()`. Ce sont les outils qui transforment une page statique en application web dynamique.

4.1 Pourquoi des structures de contrôle ?

Jusqu'ici, nos scripts exécutaient leurs instructions **les unes après les autres**, de haut en bas. Mais un vrai programme doit **choisir** (afficher « Admis » ou « Recalé » selon la moyenne) et **répéter** (afficher la table de multiplication de 7). Les **structures de contrôle** pilotent cet ordre d'exécution.

Définition

On distingue deux grandes familles de structures de contrôle :

- les structures **conditionnelles** (ou de *sélection*) : `if`, `elseif`, `else`, `switch` ;
- les structures **répétitives** (ou *boucles*) : `while`, `do...while`, `for`, `foreach`.

4.2 Les conditions

4.2.1 La condition simple : `if`

Définition

L'instruction `if` (« si ») exécute un bloc d'instructions **uniquement si** une condition est vraie. La condition est une expression qui vaut `true` (vrai) ou `false` (faux).

```
if (condition) { instructions si vrai }
```

PHP

```
<?php
$moyenne = 12.5;
if ($moyenne >= 10) {
    echo "Félicitations, vous êtes admis !";
}
```



```
?>
```

Si la moyenne atteint 10, le message s'affiche ; sinon, rien ne se passe.

⚠ Attention. La condition se place entre parenthèses () et le bloc entre accolades { }. Attention à utiliser == (comparaison) et non = (affectation) : if (\$x = 5) affecte 5 à \$x au lieu de tester, une erreur très fréquente.

4.2.2 Choisir entre deux cas : if...else

Définition

else (« sinon ») fournit un bloc à exécuter **quand la condition est fausse**. Exactement un des deux blocs est exécuté.

PHP

```
<?php
1 $moyenne = 8;
2 if ($moyenne >= 10) {
3     echo "Résultat : ADMIS";
4 } else {
5     echo "Résultat : RECALÉ";
6 }
7 ?>
```

4.2.3 Choisir parmi plusieurs cas : if...elseif...else

Quand il y a plus de deux possibilités, on enchaîne les conditions avec elseif. PHP teste les conditions **dans l'ordre** et s'arrête à la première qui est vraie.

PHP

```
<?php
1 // Mention au baccalauréat selon la moyenne
2 $moyenne = 14.5;
3 if ($moyenne >= 16) {
4     echo "Mention : Très Bien";
5 } elseif ($moyenne >= 14) {
6     echo "Mention : Bien";
7 } elseif ($moyenne >= 12) {
8     echo "Mention : Assez Bien";
9 } elseif ($moyenne >= 10) {
10    echo "Mention : Passable";
11 } else {
12    echo "Ajourné";
13 }
14 ?>
```

Pour 14,5 : le test >= 16 échoue, >= 14 réussit, on affiche « Bien ».

⚠ Attention. L'ordre des tests compte. Si on plaçait \$moyenne >= 10 en premier, toutes les bonnes moyennes afficheraient « Passable », car 14.5 >= 10 est déjà vrai. On va donc **du plus exigeant au moins exigeant**.

4.2.4 L'opérateur ternaire

Définition

L'opérateur **ternaire** `?` : est un raccourci pour un `if...else` qui choisit **entre deux valeurs**. On le lit : « *condition ? valeur si vrai : valeur si faux* ».

```
condition ? valeurSiVrai : valeurSiFaux
```

PHP

```
<?php
1  $moyenne = 11;
2  $resultat = ($moyenne >= 10) ? "Admis" : "Recalé";
3  echo "Décision : $resultat";    // affiche : Décision : Admis
?>
```

💡 **Remarque.** Le ternaire est pratique pour une affectation courte, mais pour plusieurs cas ou des blocs longs, le `if...elseif...else` reste plus lisible.

4.3 L'instruction switch

Définition

`switch` compare une **même variable** à plusieurs valeurs précises (*case*). C'est plus lisible qu'une longue suite de `elseif` lorsqu'on teste l'égalité avec des valeurs fixes.

```
switch ($var) { case valeur1: ... break; case valeur2: ... break; default: ... }
```

PHP

```
<?php
1  // Afficher la filière selon un code saisi
2  $code = "TI";
3  switch ($code) {
4      case "A4":
5          echo "Série A4 (littéraire)";
6          break;
7      case "C":
8          echo "Série C (mathématiques)";
9          break;
10     case "TI":
11         echo "Série TI (technologies de l'information)";
12         break;
13     default:
14         echo "Série inconnue";
15 }
?>
```

Sans le `break`, l'exécution « tomberait » dans le case suivant.

⚠ Attention. Le mot-clé `break` est **indispensable** à la fin de chaque `case` : il arrête le `switch`. Sans lui, PHP continue d'exécuter les `case` suivants (effet de « cascade »). Le bloc `default` est optionnel et joue le rôle du `else`.

Exemple. On peut volontairement **regrouper** des cas en omettant le `break` :

PHP

```
<?php
1 $jour = "samedi";
2 switch ($jour) {
3     case "samedi":
4     case "dimanche":
5         echo "C'est le week-end !";
6         break;
7     default:
8         echo "C'est un jour de cours.";
9 }
10
11 ?>
```

4.4 Opérateurs de comparaison et logiques

Une condition repose sur des **opérateurs** qui renvoient `true` ou `false`.

Opérateur	Signification	Exemple vrai
<code>==</code>	égal (valeur)	<code>5 == "5"</code>
<code>===</code>	identique (valeur + type)	<code>5 === 5</code>
<code>!=</code>	différent	<code>4 != 5</code>
<code><</code>	inférieur	<code>3 < 10</code>
<code>></code>	supérieur	<code>10 > 3</code>
<code><=</code>	inférieur ou égal	<code>10 <= 10</code>
<code>>=</code>	supérieur ou égal	<code>12 >= 10</code>

⚠ Attention. `==` compare seulement les valeurs : `5 == "5"` est **vrai** (PHP convertit la chaîne en nombre). `===` compare **valeur ET type** : `5 === "5"` est **faux** (entier contre chaîne). En cas de doute, préfère `===`, plus sûr.

Les **opérateurs logiques** combinent plusieurs conditions :

Opérateur	Nom	Vrai quand...
<code>&&</code>	ET	les deux conditions sont vraies
<code> </code>	OU	au moins une condition est vraie
<code>!</code>	NON	la condition est fausse (inverse)

PHP

```
<?php
1 $age = 17;
2 $moyenne = 11;
3 // Admis ET majeur ?
```

```

1  if ($moyenne >= 10 && $age >= 18) {
2      echo "Admis et majeur";
3  }
4
5  // Boursier si moyenne très haute OU situation sociale
6  $boursier = ($moyenne >= 16) || ($age < 16);
7
8  // Inverser une condition avec !
9  if (!($moyenne >= 10)) {
10     echo "Doit repasser l'examen";
11 }
12
13 ?>

```

4.5 Les boucles

Définition

Une **boucle** répète un bloc d'instructions tant qu'une condition reste vraie. Le bloc répété s'appelle le **corps** de la boucle, et chaque répétition une **itération**.

4.5.1 La boucle while

`while` (« tant que ») teste la condition **avant** chaque itération. Si elle est fausse dès le départ, le corps n'est jamais exécuté.

```
while (condition) { instructions }
```

PHP

```

1 <?php
2 // Compter de 1 à 5
3 $i = 1;
4 while ($i <= 5) {
5     echo $i . " "; // affiche : 1 2 3 4 5
6     $i = $i + 1;    // ou $i++ : ne pas oublier !
7 }
8 ?>

```

⚠ Attention. Il faut **toujours** faire évoluer la variable de la condition (ici `$i++`) à l'intérieur du corps. Sinon la condition reste vraie pour toujours : c'est une **boucle infinie** qui bloque le serveur.

4.5.2 La boucle do...while

`do...while` teste la condition **après** chaque itération : le corps est donc exécuté **au moins une fois**, même si la condition est fausse au départ.

```
do { instructions } while (condition);
```

PHP

```

1 <?php
2 $i = 10;

```

```

1 do {
2     echo "Exécuté une fois, $i = $i"; // s'affiche bien
3 } while ($i < 5);                     // la condition est pourtant fausse
4 ?>

```

Le point-virgule après while(...) est obligatoire ici.

4.5.3 La boucle for

Définition

La boucle `for` regroupe en une seule ligne les trois étapes d'une répétition comptée : l'**initialisation**, la **condition** et l'**incrément**. C'est la boucle idéale quand on connaît le nombre de tours à l'avance.

```
for (initialisation; condition; incrément) { instructions }
```

PHP

```

1 <?php
2 // Table de multiplication de 7
3 $n = 7;
4 for ($i = 1; $i <= 10; $i++) {
5     echo "$n x $i = " . ($n * $i) . "<br>";
6 }
7 ?>

```

Affiche : 7 x 1 = 7, 7 x 2 = 14, ... jusqu'à 7 x 10 = 70.

Exemple. Somme des entiers de 1 à n avec une boucle `for` :

PHP

```

1 <?php
2 $n = 100;
3 $somme = 0;
4 for ($i = 1; $i <= $n; $i++) {
5     $somme = $somme + $i; // ou $somme += $i;
6 }
7 echo "Somme de 1 à $n = $somme"; // 5050
8 ?>

```

Exemple. Compter à rebours en faisant décroître le compteur :

PHP

```

1 <?php
2 for ($i = 5; $i >= 1; $i--) {
3     echo $i . "... "; // affiche : 5... 4... 3... 2... 1...
4 }
5 echo "Décollage !";
6 ?>

```

4.5.4 La boucle foreach (parcours de tableau)

Définition

foreach parcourt **tous les éléments d'un tableau**, un par un, sans se soucier des indices. C'est la boucle naturelle pour traiter une liste (élèves, notes, produits...).

```
foreach ($tableau as $valeur) { ... }    ou
foreach ($tableau as $cle => $valeur) { ... }
```

PHP

```
<?php
1 // Liste des élèves d'une classe de Tle TI
2 $eleves = ["Awa", "Bello", "Chimène", "Ngonu"];
3 foreach ($eleves as $eleve) {
4     echo "Élève : $eleve <br>";
5 }
6 ?>
```

Avec la forme `$cle => $valeur`, on récupère aussi l'indice (ou la clé pour un tableau associatif) :

PHP

```
<?php
1 $notes = ["Maths" => 14, "PHP" => 16, "Réseaux" => 11];
2 foreach ($notes as $matiere => $note) {
3     echo "$matiere : $note / 20 <br>";
4 }
5 ?>
```

Les tableaux seront étudiés en détail au chapitre suivant.

4.6 Maîtriser les boucles : break et continue

Définition

À l'intérieur d'une boucle :

- **break** sort immédiatement de la boucle ;
- **continue** saute le reste du tour et passe directement à l'itération suivante.

PHP

```
<?php
1 // S'arrêter dès qu'on trouve un élève recalé
2 $moyennes = [12, 15, 8, 14, 16];
3 foreach ($moyennes as $m) {
4     if ($m < 10) {
5         echo "Un élève recalé trouvé (moyenne $m), on arrête.";
6         break;           // on quitte la boucle
7     }
8     echo "OK ($m) ";
9 }
10
```

```
1 | ?>
```

PHP

```
<?php
// Afficher seulement les nombres pairs de 1 à 10
for ($i = 1; $i <= 10; $i++) {
    if ($i % 2 != 0) {
        continue; // impair : on saute ce tour
    }
    echo $i . " "; // affiche : 2 4 6 8 10
}
?>
```

4.7 La fonction date()

Définition

La fonction `date($format)` renvoie la date et l'heure **du serveur**, mises en forme selon une chaîne de **format**. Chaque lettre du format représente un élément de la date.

Lettre	Signification	Exemple
d	jour (2 chiffres)	26
m	mois (2 chiffres)	06
Y	année (4 chiffres)	2026
H	heure (00 à 23)	14
i	minutes (00 à 59)	05
N	jour de la semaine (1=lundi)	5

PHP

```
<?php
// Afficher la date et l'heure du jour
echo "Nous sommes le " . date("d/m/Y") . "<br>";
echo "Il est " . date("H:i") . "<br>";
?>
```

Exemple. Salutation selon l'heure. On combine `date("H")` (l'heure sous forme de nombre) et un `if...elseif...else` :

PHP

```
<?php
1 $heure = (int) date("H"); // ex. 14 ; (int) force un entier
2
3 if ($heure < 12) {
4     echo "Bonjour et bonne matinée !";
5 } elseif ($heure < 18) {
6     echo "Bon après-midi !";
7 } else {
8     echo "Bonne soirée !";
9 }
10 ?>
```

Exemple. Afficher le nom du jour grâce à `date("N")` et un tableau :

PHP

```
<?php
1 $jours = [1 => "Lundi", "Mardi", "Mercredi", "Jeudi",
2           "Vendredi", "Samedi", "Dimanche"];
3 $numero = (int) date("N"); // 1 = lundi ... 7 = dimanche
4 echo "Aujourd'hui, c'est " . $jours[$numero];
5 ?>
```

Construire une condition correcte. (1) Identifier la donnée à tester (moyenne, âge, quantité...); (2) choisir le bon opérateur de comparaison; (3) ordonner les cas du plus précis au plus général; (4) vérifier qu'un seul bloc s'exécute pour chaque valeur d'entrée.

4.8 Exercices

► Application

Exercice 1 Pair ou impair

★ Une variable `$nombre` contient un entier. Écris un script qui affiche « pair » si le nombre est pair, « impair » sinon. (Indice : un nombre est pair si `$nombre % 2 == 0`.)

Exercice 2 Admission

★ La variable `$moyenne` contient la moyenne d'un élève. Affiche « Admis » si elle vaut au moins 10, sinon « Recalé », en utilisant l'opérateur ternaire.

Exercice 3 Le plus grand de trois nombres

★★ Trois notes sont rangées dans `$a`, `$b` et `$c`. Affiche la plus grande des trois à l'aide de conditions.

Exercice 4 Compte à rebours

★ À l'aide d'une boucle `for`, affiche un compte à rebours de 10 jusqu'à 1, puis le mot « Partez ! ».

Exercice 5 Table de multiplication

★★ La variable `$n` contient un entier. Affiche sa table de multiplication (de `$n × 1` à `$n × 10`) à l'aide d'une boucle.

► Entraînement

Exercice 6 Prix avec remise selon la quantité

★★ Dans une librairie de Yaoundé, un cahier coûte 500 FCFA. Si le client achète 10 cahiers ou plus, il bénéficie de 10 % de remise ; à partir de 50 cahiers, la remise est de 20 %. La quantité est dans `$qte`. Calcule et affiche le prix total à payer en francs CFA.

Exercice 7 Mention au baccalauréat

★★ La variable `$moyenne` contient une moyenne sur 20. Affiche la mention : « Très Bien » (≥ 16), « Bien » (≥ 14), « Assez Bien » (≥ 12), « Passable » (≥ 10) ou « Ajourné » sinon.

Exercice 8 Table de multiplication en HTML

★★ Reprends la table de multiplication de `$n`, mais affiche-la dans un véritable **tableau HTML** (balises `<table>`, `<tr>`, `<td>`).

Exercice 9 Somme et moyenne de n notes

★★★ On dispose d'un tableau `$notes` contenant les notes d'un élève (par exemple `[12, 15, 8, 14, 16]`). Calcule et affiche la **somme** et la **moyenne** de ces notes, ainsi que le verdict « Admis » ou « Recalé ».

Exercice 10 FizzBuzz scolaire

★★★ Pour les nombres de 1 à 30, affiche : « Récéré » si le nombre est divisible par 3, « Pause » s'il est divisible par 5, « Récéré-Pause » s'il est divisible par 3 ET par 5, et le nombre lui-même sinon.

Exercice 11 Année bissextile

★★★ La variable `$annee` contient une année. Affiche si elle est bissextile. (Règle : une année est bissextile si elle est divisible par 4 **et** non divisible par 100, **ou** si elle est divisible par 400.)

4.9 Corrigés

► Corrigé de l'exercice 1

PHP

```
<?php
$nombre = 7;
if ($nombre % 2 == 0) {
    echo "$nombre est pair";
} else {
    echo "$nombre est impair";
}
?>
```

► Corrigé de l'exercice 2

PHP

```
<?php
$moyenne = 11.5;
$resultat = ($moyenne >= 10) ? "Admis" : "Recalé";
echo $resultat;
?>
```

► Corrigé de l'exercice 3

PHP

```

1 <?php
2 $a = 12; $b = 17; $c = 9;
3 $max = $a;           // on suppose que $a est le plus grand
4 if ($b > $max) {
5     $max = $b;
6 }
7 if ($c > $max) {
8     $max = $c;
9 }
10 echo "La plus grande note est : $max";
11 ?>

```

► Corrigé de l'exercice 4

PHP

```

1 <?php
2 for ($i = 10; $i >= 1; $i--) {
3     echo $i . " ";
4 }
5 echo "Partez !";
6 ?>

```

► Corrigé de l'exercice 5

PHP

```

1 <?php
2 $n = 8;
3 for ($i = 1; $i <= 10; $i++) {
4     echo "$n x $i = " . ($n * $i) . "<br>";
5 }
6 ?>

```

► Corrigé de l'exercice 6

PHP

```

1 <?php
2 $prixUnitaire = 500;    // FCFA
3 $qte = 60;
4 if ($qte >= 50) {
5     $remise = 0.20;
6 } elseif ($qte >= 10) {
7     $remise = 0.10;
8 } else {
9     $remise = 0;
10 }
11 $total = $prixUnitaire * $qte * (1 - $remise);
12 echo "Total à payer : " . $total . " FCFA";
13 ?>

```

► Corrigé de l'exercice 7

PHP

```

1 <?php
2     $moyenne = 14.5;
3     if ($moyenne >= 16) {
4         echo "Très Bien";
5     } elseif ($moyenne >= 14) {
6         echo "Bien";
7     } elseif ($moyenne >= 12) {
8         echo "Assez Bien";
9     } elseif ($moyenne >= 10) {
10        echo "Passable";
11    } else {
12        echo "Ajourné";
13    }
14 ?>

```

► Corrigé de l'exercice 8

PHP

```

1 <?php
2     $n = 6;
3     echo "<table border='1'>";
4     for ($i = 1; $i <= 10; $i++) {
5         echo "<tr>";
6         echo "<td>$n x $i</td>";
7         echo "<td>" . ($n * $i) . "</td>";
8         echo "</tr>";
9     }
10    echo "</table>";
11 ?>

```

► Corrigé de l'exercice 9

PHP

```

1 <?php
2     $notes = [12, 15, 8, 14, 16];
3     $somme = 0;
4     foreach ($notes as $note) {
5         $somme += $note;
6     }
7     $moyenne = $somme / count($notes); // count() compte les éléments
8     echo "Somme : $somme <br>";
9     echo "Moyenne : " . round($moyenne, 2) . " / 20 <br>";
10    echo ($moyenne >= 10) ? "Admis" : "Recalé";
11 ?>

```

La fonction count() renvoie le nombre d'éléments d'un tableau.

► Corrigé de l'exercice 10

PHP

```

1 <?php
2     for ($i = 1; $i <= 30; $i++) {
3         if ($i % 3 == 0 && $i % 5 == 0) {
4             echo "Récré-Pause <br>";
5         }
6     }

```

```

1 } elseif ($i % 3 == 0) {
2     echo "Récréé <br>";
3 } elseif ($i % 5 == 0) {
4     echo "Pause <br>";
5 } else {
6     echo $i . " <br>";
7 }
8 }
9 }
10 }
11 }
12 }
13 ?>

```

On teste d'abord le cas « divisible par 3 ET 5 », sinon il ne serait jamais atteint.

► Corrigé de l'exercice 11

PHP

```

1 <?php
2 $annee = 2024;
3 if (($annee % 4 == 0 && $annee % 100 != 0) || ($annee % 400 == 0)) {
4     echo "$annee est bissextile";
5 } else {
6     echo "$annee n'est pas bissextile";
7 }
8 }
9 ?>

```

★ À retenir

Un script intelligent **décide** (if, elseif, else, ternaire, switch) et **répète** (while, do...while, for, foreach). Ordonne tes conditions du plus précis au plus général, n'oublie jamais le break dans un switch ni de faire évoluer le compteur d'une boucle, et choisis for quand tu connais le nombre de tours, while sinon.

Tableaux et fonctions en PHP

Au programme

Ranger plusieurs valeurs dans un même **tableau** (indexé ou associatif), le parcourir avec `for` et `foreach`, exploiter les fonctions prêtes à l'emploi (`count`, `sort`, `in_array` ...), puis écrire ses propres **fonctions** pour organiser et réutiliser son code.

5.1 Les tableaux indexés

Jusqu'ici, une variable ne contenait qu'une seule valeur. Pour gérer les notes d'une classe de 40 élèves, il faudrait 40 variables ! Le **tableau** règle ce problème : il stocke plusieurs valeurs sous un seul nom.

Définition

Un **tableau** (*array*) est une variable capable de contenir **plusieurs valeurs**. Dans un tableau **indexé**, chaque valeur est repérée par un numéro de position appelé **indice**, qui commence à 0.

5.1.1 Créer un tableau

PHP offre deux écritures équivalentes : la fonction `array()` ou les crochets `[]` (plus courte, recommandée).

PHP

```
<?php
1 // Écriture classique
2 $eleves = array("Awa", "Bello", "Chimene", "Daniel");
3 // Écriture courte (identique)
4 $villes = ["Yaoundé", "Douala", "Maroua", "Bafoussam"];
5
6 // Un tableau vide, que l'on remplira ensuite
7 $notes = [];
8
9 ?>
```

```
$tableau = [valeur0, valeur1, valeur2, ...];
```

5.1.2 Accéder à une valeur par son indice

On écrit le nom du tableau suivi de l'indice entre crochets. Le **premier** élément porte l'indice `0`, le deuxième `1`, etc.

PHP

```
<?php
1 $eleves = ["Awa", "Bello", "Chimene", "Daniel"];
2 echo $eleves[0]; // affiche : Awa
3 echo $eleves[2]; // affiche : Chimene
4
5 // On peut aussi modifier une case
6 $eleves[1] = "Bello Ngono";
7 echo $eleves[1]; // affiche : Bello Ngono
8
9 ?>
```

⚠ Attention. Un tableau de 4 éléments a pour indices `0`, `1`, `2` et `3`. Le dernier indice est donc $4 - 1 = 3$. Accéder à `$eleves[4]` renvoie une valeur `NULL` et déclenche un avertissement.

5.1.3 Ajouter un élément

Pour ajouter une valeur à la fin d'un tableau, on utilise des crochets **vides** `[]` : PHP choisit automatiquement le bon indice.

PHP

```
<?php
1 $notes = []; // tableau vide
2 $notes[] = 12; // va à l'indice 0
3 $notes[] = 15.5; // va à l'indice 1
4 $notes[] = 9; // va à l'indice 2
5 echo $notes[1]; // affiche : 15.5
6
7 ?>
```

💡 Remarque. `$notes[] = 12;` signifie « ajoute 12 à la fin ». On verra plus loin la fonction `array_push` qui fait la même chose et permet d'ajouter plusieurs valeurs d'un coup.

5.2 Parcourir un tableau indexé

5.2.1 Avec la boucle for

Comme les indices vont de `0` à `count($t) - 1`, la boucle `for` est naturelle. La fonction `count` donne le **nombre d'éléments**.

PHP

```
<?php
1 $eleves = ["Awa", "Bello", "Chimene", "Daniel"];
2 $n = count($eleves); // n vaut 4
3
4 for ($i = 0; $i < $n; $i++) {
5     echo ($i + 1) . ". " . $eleves[$i] . "<br>";
6 }
7
8 ?>
```

Exemple. Le code ci-dessus affiche :

1. Awa 2. Bello 3. Chimene 4. Daniel (chacun sur sa ligne grâce à `<br>`).

5.2.2 Avec la boucle `foreach`

`foreach` parcourt **toutes** les valeurs sans se soucier des indices : c'est la façon la plus simple et la plus sûre de lire un tableau.

```
foreach ($tableau as $valeur) { ...}
```

PHP

```
<?php
1 $villes = ["Yaoundé", "Douala", "Maroua"];
2
3
4 foreach ($villes as $ville) {
5     echo "Ville : " . $ville . "<br>";
6 }
7 ?>
```

 **Remarque.** Avec `foreach`, pas besoin de gérer un compteur ni de connaître la taille du tableau : on est sûr de ne sortir ni avant le début ni après la fin.

5.3 Les tableaux associatifs

Dans un tableau indexé, les clés sont des numéros. Dans un tableau **associatif**, on remplace ces numéros par des **clés** de notre choix (le plus souvent des mots), associées chacune à une valeur.

Définition

Un **tableau associatif** relie des **clés** (*keys*) à des **valeurs** (*values*) grâce à la flèche `=>`. La clé sert d'« étiquette » pour retrouver la valeur.

PHP

```
<?php
1 // clé => valeur
2 $eleve = [
3     "nom"    => "Awa Mballa",
4     "classe" => "Tle TI",
5     "moyenne" => 14.5,
6     "ville"  => "Garoua"
7 ];
8
9
10 echo $eleve["nom"]; // affiche : Awa Mballa
11 echo $eleve["moyenne"]; // affiche : 14.5
12 ?>
```

```
$t = ["cle1" => valeur1, "cle2" => valeur2, ...];
Accès : $t["cle1"]
```

5.3.1 Parcourir un tableau associatif

Pour obtenir à la fois la clé et la valeur, on utilise la forme complète de `foreach` avec `=>`.

```
foreach ($tableau as $cle => $valeur) { ...}
```

PHP

```
<?php
1 $eleve = [
2     "nom"    => "Awa Mballa",
3     "classe" => "Tle TI",
4     "moyenne" => 14.5
5 ];
6
7
8 foreach ($eleve as $cle => $valeur) {
9     echo $cle . " : " . $valeur . "<br>";
10 }
11 // Affiche :
12 // nom : Awa Mballa
13 // classe : Tle TI
14 // moyenne : 14.5
15 ?>
```

Exemple. On peut bâtir un **répertoire** reliant le nom d'un élève à sa moyenne, puis le parcourir pour afficher chaque résultat :

PHP

```
<?php
1 $moyennes = [
2     "Awa"    => 14.5,
3     "Bello"  => 9.0,
4     "Chimene" => 16.25,
5     "Daniel" => 11.75
6 ];
7
8
9 foreach ($moyennes as $nom => $moy) {
10     echo $nom . " a obtenu " . $moy . "/20<br>";
11 }
12 ?>
```

5.4 Les tableaux à plusieurs dimensions

Une case d'un tableau peut elle-même contenir... un autre tableau ! On obtient un **tableau multidimensionnel**, très pratique pour représenter une **liste de fiches** (une ligne par élève, plusieurs colonnes par ligne).

Définition

Un **tableau à deux dimensions** est un tableau dont chaque valeur est elle-même un tableau. On accède à un élément avec **deux** indices ou clés : `$t[i][j]`.

PHP

```

1 <?php
2 // Une fiche par élève (tableau de tableaux associatifs)
3 $classe = [
4     ["nom" => "Awa",      "moyenne" => 14.5],
5     ["nom" => "Bello",    "moyenne" => 9.0],
6     ["nom" => "Chimene",  "moyenne" => 16.25]
7 ];
8
9 // Accès : 1re fiche, clé "nom"
10 echo $classe[0]["nom"];      // affiche : Awa
11 echo $classe[2]["moyenne"];  // affiche : 16.25
12
13 // Parcours complet
14 foreach ($classe as $fiche) {
15     echo $fiche["nom"] . " : " . $fiche["moyenne"] . "/20<br>";
16 }
17 ?>

```

💡 Remarque. Cette structure « tableau de fiches » est exactement ce que renverra une requête de base de données MySQL (une ligne = une fiche). La maîtriser ici prépare le chapitre sur PHP/MySQL.

5.5 Fonctions utiles sur les tableaux

PHP fournit de nombreuses fonctions toutes prêtes. Voici les plus courantes au programme.

Fonction	Rôle	Exemple
<code>count(\$t)</code>	nombre d'éléments	<code>count([4,5,6])</code> → 3
<code>sort(\$t)</code>	trie en ordre croissant	<code>[3,1,2]</code> → <code>[1,2,3]</code>
<code>rsort(\$t)</code>	trie en ordre décroissant	<code>[3,1,2]</code> → <code>[3,2,1]</code>
<code>in_array(\$v,\$t)</code>	\$v est-il dans \$t ?	renvoie <code>true</code> / <code>false</code>
<code>array_push(\$t,\$v)</code>	ajoute \$v à la fin	comme <code>\$t[] = \$v;</code>
<code>implode(\$sep,\$t)</code>	colle en une chaîne	<code>implode(", ", \$t)</code>
<code>explode(\$sep,\$s)</code>	découpe une chaîne en tableau	<code>explode(",",\$s)</code>

PHP

```

1 <?php
2 $notes = [12, 8, 15, 9, 17];
3
4 echo count($notes);          // 5
5
6 sort($notes);                // $notes devient [8, 9, 12, 15, 17]
7 rsort($notes);               // $notes devient [17, 15, 12, 9, 8]
8
9 if (in_array(15, $notes)) {
10     echo "La note 15 existe.";
11 }
12
13 array_push($notes, 20);       // ajoute 20 à la fin
14
15 // implode : tableau -> chaîne

```

```

16 $eleves = ["Awa", "Bello", "Chimene"];
17 echo implode(" - ", $eleves); // affiche : Awa - Bello - Chimene
18
19 // explode : chaîne -> tableau
20 $liste = "Yaoundé;Douala;Maroua";
21 $villes = explode(";", $liste); // ["Yaoundé", "Douala", "Maroua"]
22 echo $villes[1]; // affiche : Douala
23 ?>

```

⚠ Attention. `sort` et `rsort` **modifient directement** le tableau passé en argument (on ne récupère pas le résultat avec un `=`). Après `sort($notes);`, c'est `$notes` lui-même qui est trié.

5.6 Les fonctions

Quand un même calcul revient souvent (calculer une moyenne, décider d'une admission...), on l'enferme dans une **fonction** que l'on appelle ensuite autant de fois que nécessaire. C'est la clé d'un code clair et réutilisable.

Définition

Une **fonction** est un bloc d'instructions portant un **nom**. On la **déclare** une fois, puis on l'**appelle** par son nom. Elle peut recevoir des **paramètres** (données d'entrée) et **renvoyer** un résultat avec `return`.

5.6.1 Déclarer et appeler une fonction

```
function nom($param1, $param2) { ... return $resultat; }
```

PHP

```

1 <?php
2 // Déclaration : on définit la fonction
3 function carre($x) {
4     return $x * $x;
5 }
6
7 // Appel : on l'utilise
8 echo carre(5); // affiche : 25
9 $r = carre(9); // $r vaut 81
10 echo $r;
11 ?>

```

💡 Remarque. `return` **renvoie** une valeur *et arrête* la fonction : les instructions placées après ne sont pas exécutées. Une fonction sans `return` peut tout de même travailler (par exemple afficher quelque chose avec `echo`).

5.6.2 Paramètres par défaut

On peut donner une valeur **par défaut** à un paramètre : si l'appel ne la précise pas, cette valeur est utilisée.

PHP

```
<?php
// La devise vaut "FCFA" si on ne la précise pas
function prix($montant, $devise = "FCFA") {
    return $montant . " " . $devise;
}

echo prix(2500);           // affiche : 2500 FCFA
echo prix(10, "euros");   // affiche : 10 euros
?>
```

5.6.3 Portée des variables

Définition

La **portée** (*scope*) d'une variable est la zone du programme où elle existe. Une variable créée **dans** une fonction est **locale** : elle n'existe pas en dehors. Une variable créée en dehors est **globale** et n'est, par défaut, **pas visible** à l'intérieur d'une fonction.

PHP

```
<?php
$tva = 0.1925; // variable globale

function montantTTC($ht) {
    global $tva; // on rend $tva visible ici
    return $ht * (1 + $tva);
}

echo montantTTC(1000); // affiche : 1192.5
?>
```

⚠ Attention. Sans le mot-clé `global`, la variable `$tva` serait `NULL` à l'intérieur de la fonction. Mieux encore : on évite `global` en **passant** la donnée en paramètre, ce qui rend la fonction plus claire.

5.7 Quelques fonctions PHP utiles

5.7.1 Sur les chaînes de caractères

Fonction	Rôle	Exemple
<code>strlen(\$s)</code>	longueur (nb de caractères)	<code>strlen("Awa")</code> → 3
<code>strtoupper(\$s)</code>	tout en majuscules	→ "AWA"
<code>strtolower(\$s)</code>	tout en minuscules	→ "awa"
<code>substr(\$s,\$d,\$l)</code>	extraît une portion	<code>substr("Yaoundé",0,3)</code> → "Yao"
<code>str_replace(\$a,\$b,\$s)</code>	remplace \$a par \$b	remplace dans \$s

PHP

```
<?php
$nom = "Awa Mballa";
```

```

1 echo strlen($nom);           // 10 (l'espace compte)
2 echo strtoupper($nom);       // AWA MBALLA
3 echo strtolower($nom);       // awa mballa
4 echo substr($nom, 0, 3);     // Awa (3 caractères dès l'indice 0)
5 echo str_replace("a", "@", $nom); // Aw@ Mb@ll@
6
7 ?>

```

5.7.2 Sur les nombres

Fonction	Rôle	Exemple
<code>round(\$x,\$d)</code>	arrondi à \$d décimales	<code>round(14.567,2)</code> → 14.57
<code>abs(\$x)</code>	valeur absolue	<code>abs(-5)</code> → 5
<code>max(\$a,\$b,...)</code>	le plus grand	<code>max(8,15,9)</code> → 15
<code>min(\$a,\$b,...)</code>	le plus petit	<code>min(8,15,9)</code> → 8
<code>rand(\$a,\$b)</code>	entier aléatoire \$a..\$b	<code>rand(1,6)</code> (dé)

PHP

```

1 <?php
2 echo round(14.567, 2); // 14.57
3 echo abs(-8);          // 8
4 echo max(8, 15, 9);    // 15
5 echo min(8, 15, 9);    // 8
6 echo rand(1, 6);       // un entier au hasard entre 1 et 6
7
8 // max et min acceptent aussi un tableau
9 $notes = [12, 8, 17, 9];
10 echo max($notes);      // 17
11
12 ?>

```

5.8 Exemples complets

Exemple. Moyenne d'un tableau de notes. On parcourt le tableau pour cumuler la somme, puis on divise par le nombre de notes.

PHP

```

1 <?php
2 $notes = [12, 8, 15, 9, 17];
3 $somme = 0;
4
5 foreach ($notes as $note) {
6     $somme = $somme + $note;
7 }
8
9 $moyenne = $somme / count($notes);
10 echo "Moyenne de la classe : " . round($moyenne, 2) . "/20";
11 // affiche : Moyenne de la classe : 12.2/20
12 ?>

```

Exemple. Fonction `estAdmis`. Elle reçoit une moyenne et renvoie `true` si l'élève est admis ($\text{moyenne} \geq 10$), `false` sinon.

PHP

```
1 <?php
2 function estAdmis($moyenne) {
3     return $moyenne >= 10;
4 }
5
6 $moy = 11.5;
7 if (estAdmis($moy)) {
8     echo "Admis";
9 } else {
10    echo "Non admis";
11 }
12 ?>
```

Exemple. Fonction qui calcule une moyenne. On regroupe le calcul précédent dans une fonction réutilisable, qui reçoit un tableau et renvoie la moyenne.

PHP

```
1 <?php
2 function moyenne($notes) {
3     $somme = 0;
4     foreach ($notes as $n) {
5         $somme = $somme + $n;
6     }
7     return $somme / count($notes);
8 }
9
10 $classeA = [12, 8, 15, 9, 17];
11 $classeB = [10, 11, 14];
12
13 echo round(moyenne($classeA), 2); // 12.2
14 echo round(moyenne($classeB), 2); // 11.67
15 ?>
```

💡 Remarque. Une fois `moyenne` écrite, on la réutilise pour n'importe quelle classe sans réécrire la boucle. C'est tout l'intérêt des fonctions : **écrire une fois, réutiliser partout.**

5.9 Exercices

► Application

Exercice 1 Afficher un tableau

★ On donne `$villes = ["Yaoundé", "Douala", "Maroua", "Bafoussam"]`. Affiche chaque ville précédée de son numéro de position (1, 2, 3, 4), une par ligne.

Exercice 2 Max, min, somme et moyenne

★ Soit `$notes = [12, 8, 15, 9, 17, 6]`. Affiche la plus grande note, la plus petite, la somme totale et la moyenne arrondie à deux décimales.

Exercice 3 Rechercher un élève

★ On dispose de `$classe = ["Awa", "Bello", "Chimene", "Daniel"]`. Écris un script qui indique si l'élève « Chimene » figure dans la liste, puis si « Eric » y figure.

Exercice 4 Compter les admis

★★ Soit le tableau associatif suivant :

PHP

```
$moyennes = ["Awa"=>14, "Bello"=>8, "Chimene"=>16, "Daniel"=>9, "Eric"=>11];
```

Compte combien d'élèves sont admis (moyenne ≥ 10) et affiche le résultat.

Exercice 5 Fonction mention

★★ Écris une fonction `mention($m)` qui renvoie : « Excellent » si $m \geq 16$, « Bien » si $14 \leq m < 16$, « Assez bien » si $12 \leq m < 14$, « Passable » si $10 \leq m < 12$ et « Insuffisant » sinon. Teste-la sur quelques moyennes.

► Entraînement

Exercice 6 Inverser un tableau

★★ Sans utiliser la fonction toute prête `array_reverse`, écris un script qui construit un nouveau tableau contenant les éléments de `$t = [10, 20, 30, 40]` dans l'ordre inverse, puis l'affiche.

Exercice 7 Bulletin de la classe

★★ Soit le tableau de fiches :

PHP

```
$classe = [{"nom"=>"Awa", "moy"=>14}, {"nom"=>"Bello", "moy"=>8}, {"nom"=>"Chimene", "moy"=>16}];
```

Affiche pour chaque élève son nom, sa moyenne et sa mention (réutilise la fonction `mention` de l'exercice 5).

Exercice 8 Fréquences des notes

★★★ On donne `$notes = [12, 8, 12, 15, 8, 12, 9]`. Construis un tableau associatif qui compte combien de fois chaque note apparaît, puis affiche le résultat sous la forme `12 -> 3 fois`.

Exercice 9 Initiales

★★★ Écris une fonction `initiales($nomComple)` qui, à partir d'une chaîne de noms séparés par des espaces, renvoie les initiales en majuscules. Par exemple, `"Awa Mballa Ngono"` doit donner `"A.M.N."`. Indice : utilise `explode`, puis `substr` et `strtoupper`.

5.10 Corrigés

► Corrigé de l'exercice 1

PHP

```
<?php
$villes = ["Yaoundé", "Douala", "Maroua", "Bafoussam"];

for ($i = 0; $i < count($villes); $i++) {
    echo ($i + 1) . ". " . $villes[$i] . "<br>";
}
```

```

1 }
2 // On pouvait aussi utiliser foreach avec un compteur séparé.
3 ?>

```

► Corrigé de l'exercice 2

PHP

```

1 <?php
2     $notes = [12, 8, 15, 9, 17, 6];
3
4     $somme = 0;
5     foreach ($notes as $n) {
6         $somme = $somme + $n;
7     }
8
9     echo "Maximum : " . max($notes) . "<br>";
10    echo "Minimum : " . min($notes) . "<br>";
11    echo "Somme : " . $somme . "<br>";
12    echo "Moyenne : " . round($somme / count($notes), 2);
13 ?>

```

► Corrigé de l'exercice 3

PHP

```

1 <?php
2     $classe = ["Awa", "Bello", "Chimene", "Daniel"];
3
4     if (in_array("Chimene", $classe)) {
5         echo "Chimene est dans la classe.<br>";
6     } else {
7         echo "Chimene est absente.<br>";
8     }
9
10    if (in_array("Eric", $classe)) {
11        echo "Eric est dans la classe.";
12    } else {
13        echo "Eric n'est pas dans la classe.";
14    }
15 ?>

```

► Corrigé de l'exercice 4

PHP

```

1 <?php
2     $moyennes = [
3         "Awa"      => 14,
4         "Bello"    => 8,
5         "Chimene"  => 16,
6         "Daniel"   => 9,
7         "Eric"     => 11
8     ];
9
10    $nbAdmis = 0;
11    foreach ($moyennes as $nom => $moy) {
12        if ($moy >= 10) {

```

```

13     $nbAdmis = $nbAdmis + 1;
14 }
15 }
16
17 echo "Nombre d'admis : " . $nbAdmis . " sur " . count($moyennes);
18 // affiche : Nombre d'admis : 3 sur 5
19 ?>

```

► Corrigé de l'exercice 5

PHP

```

1 <?php
2 function mention($m) {
3     if ($m >= 16) {
4         return "Excellent";
5     } elseif ($m >= 14) {
6         return "Bien";
7     } elseif ($m >= 12) {
8         return "Assez bien";
9     } elseif ($m >= 10) {
10        return "Passable";
11    } else {
12        return "Insuffisant";
13    }
14 }
15
16 echo mention(17); // Excellent
17 echo mention(13); // Assez bien
18 echo mention(7); // Insuffisant
19 ?>

```

💡 Remarque. On teste les seuils du plus grand au plus petit. Dès qu'une condition est vraie, `return` renvoie la mention et arrête la fonction : inutile de re-tester $m < 16$, c'est déjà garanti par l'ordre des `elseif`.

► Corrigé de l'exercice 6

PHP

```

1 <?php
2 $t = [10, 20, 30, 40];
3 $inverse = [];
4
5 for ($i = count($t) - 1; $i >= 0; $i--) {
6     $inverse[] = $t[$i]; // on ajoute du dernier vers le premier
7 }
8
9 // Affichage
10 foreach ($inverse as $valeur) {
11     echo $valeur . " "; // affiche : 40 30 20 10
12 }
13 ?>

```

► Corrigé de l'exercice 7

PHP

```

1 <?php
2 function mention($m) {
3     if ($m >= 16) return "Excellent";
4     elseif ($m >= 14) return "Bien";
5     elseif ($m >= 12) return "Assez bien";
6     elseif ($m >= 10) return "Passable";
7     else return "Insuffisant";
8 }
9
10 $classe = [
11     ["nom" => "Awa", "moy" => 14],
12     ["nom" => "Bello", "moy" => 8],
13     ["nom" => "Chimene", "moy" => 16]
14 ];
15
16 foreach ($classe as $eleve) {
17     echo $eleve["nom"] . " : " . $eleve["moy"]
18         . "/20 (" . mention($eleve["moy"]) . ")<br>";
19 }
20 // Awa : 14/20 (Bien)
21 // Bello : 8/20 (Insuffisant)
22 // Chimene : 16/20 (Excellent)
23 ?>

```

► Corrigé de l'exercice 8

PHP

```

1 <?php
2 $notes = [12, 8, 12, 15, 8, 12, 9];
3 $freq = []; // tableau associatif note => nombre
4
5 foreach ($notes as $n) {
6     if (in_array($n, array_keys($freq))) {
7         $freq[$n] = $freq[$n] + 1; // déjà vue : on incrémente
8     } else {
9         $freq[$n] = 1; // première fois
10    }
11 }
12
13 foreach ($freq as $note => $nombre) {
14     echo $note . " -> " . $nombre . " fois<br>";
15 }
16 // 12 -> 3 fois
17 // 8 -> 2 fois
18 // 15 -> 1 fois
19 // 9 -> 1 fois
20 ?>

```

► Corrigé de l'exercice 9

PHP

```

1 <?php
2 function initiales($nomComplet) {
3     $mots = explode(" ", $nomComplet); // découpe sur les espaces

```

```
1 $resultat = "";
2
3 foreach ($mots as $mot) {
4     $premiere = substr($mot, 0, 1); // 1re lettre du mot
5     $resultat = $resultat . strtoupper($premiere) . ".";
6 }
7
8 return $resultat;
9 }
10
11 echo initiales("Awa Mballa Ngono"); // affiche : A.M.N.
12 ?>
```

★ À retenir

Un **tableau** regroupe plusieurs valeurs : indices numériques (départ à 0) ou clés => valeurs. On le parcourt avec `foreach ( $t as $cle => $val )` et on l'exploite avec `count`, `sort`, `in_array` ... Une **fonction** (`function nom($p) { return ...; }`) enferme un calcul pour l'écrire une fois et le réutiliser partout.

Formulaires et variables superglobales

Au programme

Récupérer et traiter les données d'un formulaire HTML dans un script PHP : les variables **superglobales** (`$_GET`, `$_POST` ...), le choix entre les méthodes **GET** et **POST**, la validation et la sécurisation des saisies, l'envoi vers la même page et le téléversement (*upload*) de fichiers. C'est le cœur d'une application web *interactive*.

6.1 Le rôle d'un formulaire

Jusqu'ici, nos scripts affichaient des données fixées dans le code. Un site *dynamique* doit au contraire **recevoir des informations de l'utilisateur** : un nom, un mot de passe, le prix d'un produit... Ces informations sont saisies dans un **formulaire HTML**, puis envoyées au serveur, où un script PHP les récupère et les traite.

Définition

Un **formulaire** est une zone d'une page HTML (balise `<form>`) contenant des champs de saisie (`<input>`, `<textarea>`, `<select>` ...). Quand l'utilisateur valide, le navigateur envoie les données vers le fichier indiqué par l'attribut `action`, selon la `method` choisie (`get` ou `post`).

HTML

```
<form action="traitement.php" method="post">
  <input type="text" name="nom">
  <input type="submit" value="Envoyer">
</form>
```

Chaque champ porte un attribut `name` : c'est la clé qui permettra à PHP de retrouver la valeur saisie.

⚠ Attention. La donnée envoyée est repérée par l'attribut `name` du champ, *pas* par son `id`. Si un champ n'a pas de `name`, sa valeur n'est jamais transmise au serveur.

6.2 Les variables superglobales

Définition

Une variable **superglobale** est un tableau associatif fourni automatiquement par PHP, accessible **partout** dans le script (même à l'intérieur d'une fonction) sans avoir à le déclarer. Son

nom commence par `$_` et s'écrit en majuscules.

Voici les superglobales au programme de Terminale TI.

Superglobale	Contenu
<code>\$_GET</code>	Données envoyées par la méthode GET (visibles dans l'URL)
<code>\$_POST</code>	Données envoyées par la méthode POST (cachées dans la requête)
<code>\$_REQUEST</code>	Réunion de <code>\$_GET</code> , <code>\$_POST</code> et des cookies
<code>\$_FILES</code>	Informations sur les fichiers téléversés (<i>upload</i>)
<code>\$_SERVER</code>	Informations sur le serveur et la requête (URL, méthode...)
<code>\$_SESSION</code>	Données conservées d'une page à l'autre pour un visiteur

Remarque. Ces tableaux sont **associatifs** : on accède à une valeur par sa clé, qui est le `name` du champ. Par exemple, `$_POST['nom']` contient la valeur du champ `name="nom"` envoyé en POST.

Exemple. Quelques cases utiles de `$_SERVER` : `$_SERVER['REQUEST_METHOD']` vaut `"GET"` ou `"POST"` ; `$_SERVER['PHP_SELF']` contient le nom du fichier courant.

6.3 Méthodes GET et POST

Le navigateur peut transmettre les données de deux façons, choisies par l'attribut `method` du formulaire.

6.3.1 La méthode GET

Les données sont ajoutées **à la fin de l'URL**, après un point d'interrogation, sous forme `cle=valeur` séparées par des `&` :

```
http://localhost/page.php?ville=Douala&annee=2026
```

Elles sont récupérées dans `$_GET['ville']` et `$_GET['annee']`. Comme tout est visible dans l'URL, on peut placer le lien dans un favori ou le partager.

6.3.2 La méthode POST

Les données sont placées dans le **corps de la requête**, invisibles dans l'URL. Elles sont récupérées dans `$_POST`. C'est la méthode à utiliser pour les mots de passe, les longs textes et tout ce qui modifie le serveur (création, suppression...).

6.3.3 Tableau comparatif

Critère	GET	POST
Visibilité	Visible dans l'URL	Cachée dans la requête
Récupération	<code>\$_GET</code>	<code>\$_POST</code>
Taille	Limitée (URL courte)	Très grande (fichiers OK)
Favori/partage	Possible	Impossible
Sécurité	Faible (mot de passe nu)	Meilleure (non affichée)
Usage typique	Recherche, filtre, lien	Connexion, enregistrement

Quelle méthode choisir ? Utilise GET pour une *lecture* qu'on peut refaire sans risque (recherche, pagination, filtre); utilise POST dès qu'il y a un *secret* (mot de passe), une *grande* quantité de données, un *fichier*, ou une action qui *modifie* les données (ajout, suppression).

⚠ Attention. GET n'est jamais sécurisé : la valeur s'affiche en clair dans la barre d'adresse et reste dans l'historique. Ne fais **jamais** transiter un mot de passe en GET.

6.4 Récupérer les valeurs d'un formulaire

Voyons un cas complet en deux fichiers : la page HTML qui contient le formulaire, et le script PHP qui reçoit les données.

6.4.1 Le formulaire : formulaire.html

HTML

```

1 <!DOCTYPE html>
2 <html lang="fr">
3 <head><meta charset="utf-8"><title>Inscription</title></head>
4 <body>
5   <h1>Inscription au club informatique</h1>
6   <form action="traitement.php" method="post">
7     <p>Nom : <input type="text" name="nom"></p>
8     <p>Classe : <input type="text" name="classe"></p>
9     <p>Ville : <input type="text" name="ville"></p>
10    <p><input type="submit" value="Valider"></p>
11  </form>
12 </body>
13 </html>

```

6.4.2 Le traitement : traitement.php

PHP

```

1 <?php
2   // On lit chaque champ par son attribut name
3   $nom    = $_POST['nom'];
4   $classe = $_POST['classe'];
5   $ville  = $_POST['ville'];
6
7   echo "<h1>Bienvenue $nom !</h1>";
8   echo "<p>Classe : $classe - Ville : $ville</p>";
9 ?>

```

Le champ `name="nom"` du formulaire devient `$_POST['nom']` dans le script.

💡 Remarque. Si le formulaire utilisait `method="get"`, le code serait identique en remplaçant `$_POST` par `$_GET`, et les valeurs apparaîtraient dans l'URL `traitement.php?nom=...&classe=...`.

6.5 Valider et sécuriser les données

Ne jamais faire confiance aux données reçues : un champ peut être vide, absent, ou contenir du code malveillant. Quatre fonctions sont au programme.

6.5.1 isset() et empty()

- `isset($_POST['nom'])` renvoie `true` si la clé existe (le formulaire a bien été envoyé) ;
- `empty($_POST['nom'])` renvoie `true` si la valeur est vide ("", 0 , non définie).

PHP

```
<?php
1 // A-t-on bien reçu le formulaire ?
2 if (isset($_POST['nom']) && !empty($_POST['nom'])) {
3     echo "Bonjour " . $_POST['nom'];
4 } else {
5     echo "Le nom est obligatoire.";
6 }
7 ?>
```

6.5.2 trim() et htmlspecialchars()

- `trim($texte)` supprime les espaces inutiles en début et fin de chaîne ;
- `htmlspecialchars($texte)` transforme les caractères dangereux (< , > , & , ") en entités HTML, ce qui empêche l'injection de code (*faible XSS*).

PHP

```
<?php
1 // Nettoyage standard d'un champ texte
2 $nom = trim($_POST['nom']); // retire les espaces
3 $nom = htmlspecialchars($nom); // neutralise le HTML
4 echo "Nom sécurisé : $nom";
5 ?>
```

⚠ Attention. Sans `htmlspecialchars`, un visiteur pourrait saisir `<script>...</script>` et faire exécuter son code dans la page : c'est une attaque classique. Affiche **toujours** les données utilisateur à travers cette fonction.

6.6 Envoyer le formulaire vers la même page

On peut faire pointer l'action du formulaire vers le fichier lui-même : il suffit de laisser `action` vide. Le même fichier affiche alors le formulaire *et* traite la réponse. On distingue les deux cas en testant la méthode de la requête.

PHP

```
<?php
1 $message = "";
2 // Le formulaire a-t-il été soumis ?
3 if ($_SERVER['REQUEST_METHOD'] === 'POST') {
4     $nom = trim($_POST['nom']);
5     if ($nom !== "") {
6         $message = "Merci, " . htmlspecialchars($nom) . " !";
7     } else {
8         $message = "Veuillez saisir votre nom.";
9     }
10 }
```

```

10     }
11 }
12 ?>
13 <form action="" method="post">
14     <input type="text" name="nom">
15     <input type="submit" value="OK">
16 </form>
17 <p><?php echo $message; ?></p>

```

action="" renvoie vers la page courante : un seul fichier suffit.

6.7 Exemple fil rouge : gestion d'un produit

Construisons un mini-écran de **gestion de stock** pour une boutique de Yaoundé. L'utilisateur saisit un produit (nom, prix unitaire en FCFA, quantité); la page PHP récupère, valide, puis affiche un récapitulatif avec le **montant total**.

6.7.1 Le formulaire (HTML)

HTML

```

1 <!DOCTYPE html>
2 <html lang="fr">
3 <head><meta charset="utf-8"><title>Gestion de produit</title></head>
4 <body>
5     <h1>Ajouter un produit</h1>
6     <form action="produit.php" method="post">
7         <p>Nom du produit : <input type="text" name="produit"></p>
8         <p>Prix unitaire (FCFA) : <input type="number" name="prix"></p>
9         <p>Quantité : <input type="number" name="quantite"></p>
10        <p><input type="submit" value="Enregistrer"></p>
11    </form>
12 </body>
13 </html>

```

6.7.2 Le traitement (PHP) : produit.php

PHP

```

1 <?php
2 // 1. Vérifier que tous les champs sont présents
3 if (isset($_POST['produit'], $_POST['prix'], $_POST['quantite'])) {
4
5     // 2. Nettoyer le nom du produit
6     $produit = htmlspecialchars(trim($_POST['produit']));
7     // 3. Convertir prix et quantité en nombres entiers
8     $prix = (int) $_POST['prix'];
9     $quantite = (int) $_POST['quantite'];
10
11     // 4. Valider : champ non vide et valeurs positives
12     if ($produit === "" || $prix <= 0 || $quantite <= 0) {
13         echo "<p>Erreur : saisie incomplète ou valeurs invalides.</p>";
14     } else {
15         // 5. Calculer le montant total
16         $total = $prix * $quantite;
17
18         // 6. Afficher le récapitulatif

```

```

19     echo "<h2>Récapitulatif</h2>";
20     echo "<ul>";
21     echo "<li>Produit : $produit</li>";
22     echo "<li>Prix unitaire : $prix FCFA</li>";
23     echo "<li>Quantité : $quantite</li>";
24     echo "<li><strong>Total : $total FCFA</strong></li>";
25     echo "</ul>";
26 }
27 } else {
28     echo "<p>Veuillez remplir le formulaire.</p>";
29 }
30 ?>

```

Saisir « Cahier », 350 et 12 affiche un total de 4 200 FCFA.

Remarque. La conversion (int) \$_POST['prix'] (appelée *transtypage*) garantit qu'on manipule bien un nombre avant de multiplier. Sans elle, une saisie comme "12abc" pourrait fausser le calcul.

6.8 Téléverser un fichier : \$_FILES

Pour envoyer un fichier (photo, PDF), le formulaire doit respecter trois règles : méthode `post`, attribut `enctype="multipart/form-data"`, et un champ `<input type="file">`.

HTML

```

<form action="upload.php" method="post" enctype="multipart/form-data">
  <input type="file" name="photo">
  <input type="submit" value="Envoyer">
</form>

```

Côté PHP, le fichier reçu est décrit dans `$_FILES['photo']`, un tableau contenant notamment :

Clé	Signification
<code>['name']</code>	Nom d'origine du fichier (ex. <code>recu.pdf</code>)
<code>['tmp_name']</code>	Emplacement temporaire sur le serveur
<code>['size']</code>	Taille en octets
<code>['error']</code>	Code d'erreur (0 si tout va bien)

Le fichier arrive d'abord dans un dossier **temporaire** ; on le déplace ensuite vers son emplacement définitif avec `move_uploaded_file()`.

PHP

```

<?php
1  if (isset($_FILES['photo']) && $_FILES['photo']['error'] === 0) {
2      $source      = $_FILES['photo']['tmp_name'];
3      $destination = "uploads/" . $_FILES['photo']['name'];
4
5      // Déplacer le fichier temporaire vers le dossier uploads/
6      if (move_uploaded_file($source, $destination)) {
7          echo "Fichier reçu : " . $_FILES['photo']['name'];
8      } else {
9          echo "Échec de l'enregistrement.";
10     }

```



```

11     }
12   } else {
13     echo "Aucun fichier valide reçu.";
14   }
15 }?>

```

⚠ **Attention.** `move_uploaded_file` est la **seule** façon sûre de récupérer un fichier téléversé : elle vérifie que le fichier provient bien d'un envoi HTTP. Le dossier de destination (`uploads/`) doit exister et être accessible en écriture.

6.9 Exercices

► Application

Exercice 1 Lire un champ

★ Un formulaire en `method="post"` contient un champ `name="prenom"`. Écris la ligne PHP qui récupère sa valeur dans une variable `$prenom`.

Exercice 2 GET ou POST ?

★ Pour chacune de ces situations, indique la méthode la plus adaptée : (a) recherche d'un cours par mot-clé ; (b) saisie d'un mot de passe ; (c) envoi d'une photo de profil ; (d) filtre des résultats par ville.

Exercice 3 Formulaire de contact

★ Écris le formulaire HTML d'une page *Contact* (champs : `nom`, `email`, et un `<textarea name="message">`), qui envoie en POST vers `contact.php`. Écris ensuite `contact.php` qui affiche « Message de ... » suivi du contenu reçu.

Exercice 4 Total d'une commande

★ Une page reçoit en POST un `prix` et une `quantite`. Écris le script qui calcule et affiche le total en FCFA.

Exercice 5 Champs obligatoires

★★ Un formulaire d'inscription envoie `nom` et `classe`. Écris le script qui refuse l'enregistrement (message d'erreur) si l'un des deux champs est vide, et affiche sinon « Inscription de ... en classe de ... ».

► Entraînement

Exercice 6 Basculer GET en POST

★★ On donne ce formulaire de recherche en GET : `<form action="recherche.php" method="get">` avec un champ `name="motcle"`, traité par `echo $_GET['motcle'];`. Réécris le formulaire *et* le traitement en méthode POST.

Exercice 7 Page de connexion

★★ Écris une page `connexion.php` (formulaire + traitement dans le même fichier) qui demande un `login` et un `motdepasse`. Si `login` vaut `"admin"` et `motdepasse` vaut `"onbuch2026"`, affiche « Connexion réussie » ; sinon « Identifiants incorrects ».

Exercice 8 Facture de fournitures

★★ Un formulaire reçoit le nom d'un article, son prix unitaire (FCFA) et la quantité. Affiche un récapitulatif sécurisé (`htmlspecialchars`) et le total. Refuse les prix ou quantités négatifs.

Exercice 9 Conversion FCFA → euros

★★ Une page reçoit en POST un montant en FCFA (`name="montant"`). Affiche sa valeur en euros sachant que 1 € = 656 FCFA. Arrondis à deux décimales.

Exercice 10 Calculatrice de moyenne

★★★ Un formulaire envoie trois notes (`note1` , `note2` , `note3`) en POST. Vérifie que les trois champs sont remplis, convertis-les en nombres, puis affiche la moyenne arrondie à deux décimales et la mention (*Admis* si moyenne ≥ 10 , sinon *Ajourné*).

6.10 Corrigés

► Corrigé de l'exercice 1

PHP

```
<?php
    $prenom = $_POST['prenom'];
?>
```

► Corrigé de l'exercice 2

(a) GET (recherche, action sans risque) ; (b) POST (un mot de passe ne doit jamais apparaître dans l'URL) ; (c) POST (envoi de fichier obligatoirement en POST) ; (d) GET (filtre que l'on peut mettre en favori).

► Corrigé de l'exercice 3

HTML

```
<form action="contact.php" method="post">
    <p>Nom : <input type="text" name="nom"></p>
    <p>Email : <input type="text" name="email"></p>
    <p>Message :<br><textarea name="message"></textarea></p>
    <p><input type="submit" value="Envoyer"></p>
</form>
```

PHP

```
<?php
    $nom      = htmlspecialchars(trim($_POST['nom']));
    $email    = htmlspecialchars(trim($_POST['email']));
    $message  = htmlspecialchars(trim($_POST['message']));

    echo "<h2>Message de $nom ($email)</h2>";
    echo "<p>$message</p>";
?>
```

► Corrigé de l'exercice 4

PHP

```
<?php
    $prix      = (int) $_POST['prix'];
    $quantite  = (int) $_POST['quantite'];
    $total     = $prix * $quantite;
    echo "Total : $total FCFA";
?>
```

► Corrigé de l'exercice 5

PHP

```

1 <?php
2 $nom      = trim($_POST['nom']);
3 $classe   = trim($_POST['classe']);
4
5 if ($nom === "" || $classe === "") {
6     echo "Erreur : le nom et la classe sont obligatoires.";
7 } else {
8     $nom      = htmlspecialchars($nom);
9     $classe   = htmlspecialchars($classe);
10    echo "Inscription de $nom en classe de $classe.";
11 }
12 ?>

```

► Corrigé de l'exercice 6

On remplace `method="get"` par `method="post"` et `$_GET` par `$_POST`.

HTML

```

1 <form action="recherche.php" method="post">
2   <input type="text" name="motcle">
3   <input type="submit" value="Rechercher">
4 </form>

```

PHP

```

1 <?php
2 echo htmlspecialchars($_POST['motcle']);
3 ?>

```

► Corrigé de l'exercice 7

PHP

```

1 <?php
2 $message = "";
3 if ($_SERVER['REQUEST_METHOD'] === 'POST') {
4     $login = trim($_POST['login']);
5     $mdp    = trim($_POST['motdepasse']);
6     if ($login === "admin" && $mdp === "onbuch2026") {
7         $message = "Connexion réussie";
8     } else {
9         $message = "Identifiants incorrects";
10    }
11 }
12 ?>
13 <form action="" method="post">
14   <p>Login : <input type="text" name="login"></p>
15   <p>Mot de passe : <input type="password" name="motdepasse"></p>
16   <p><input type="submit" value="Se connecter"></p>
17 </form>
18 <p><?php echo $message; ?></p>

```

► Corrigé de l'exercice 10

PHP

```
1 <?php
2     if (isset($_POST['note1'], $_POST['note2'], $_POST['note3'])
3         && $_POST['note1'] != "" && $_POST['note2'] != ""
4         && $_POST['note3'] != "") {
5
6         $n1 = (float) $_POST['note1'];
7         $n2 = (float) $_POST['note2'];
8         $n3 = (float) $_POST['note3'];
9
10        $moyenne = ($n1 + $n2 + $n3) / 3;
11        $mention = ($moyenne >= 10) ? "Admis" : "Ajourné";
12
13        echo "Moyenne : " . round($moyenne, 2) . " - " . $mention;
14    } else {
15        echo "Veuillez remplir les trois notes.";
16    }
17 ?>
```

★ À retenir

Un formulaire envoie ses champs (name) vers PHP, qui les lit dans une **superglobale** : `$_POST` (caché) ou `$_GET` (dans l'URL). Toujours **valider** (`isset`, `empty`) et **sécuriser** (`trim`, `htmlspecialchars`) avant d'utiliser une donnée reçue. Un upload exige `enctype="multipart/form-data"` et `move_uploaded_file()`.

PHP et MySQL : les bases de données

Au programme

Stocker durablement des données dans une base relationnelle MySQL, écrire les requêtes SQL essentielles (`CREATE`, `INSERT`, `SELECT`, `UPDATE`, `DELETE`), puis interfacer cette base depuis PHP avec l'extension `mysqli` : se connecter, exécuter une requête, lire et afficher les résultats. C'est le cœur de toute application web de gestion.

7.1 La base de données relationnelle

Jusqu'ici, dès que le script PHP se termine, les données disparaissent. Pour **conserver** les notes d'un élève, la liste des produits d'une boutique de Douala ou les inscriptions à un concours, on a besoin d'une **base de données**. Le logiciel qui la gère s'appelle un **SGBD** (Système de Gestion de Base de Données) ; ici, ce sera **MySQL**.

Définition

Une **base de données relationnelle** organise l'information dans des **tables**. Une table ressemble à un tableau à deux dimensions :

- une **ligne** (ou *enregistrement*) représente un élément concret (un élève, un produit...) ;
- une **colonne** (ou *champ*) représente une **propriété** commune à tous les éléments (le nom, la classe, la moyenne...).

Exemple. La table `eleves` du Lycée de Maroua :

id	nom	classe	moyenne
1	Awa Bello	Tle TI	14.50
2	Jean Mballa	Tle TI	9.75
3	Sandrine Eto	Tle TI	16.00

Trois enregistrements (lignes) et quatre champs (colonnes).

Définition

La **clé primaire** (*primary key*) est un champ dont la valeur est **unique** pour chaque ligne : elle identifie un enregistrement sans ambiguïté. On utilise très souvent un champ `id` entier qui **s'incrémente automatiquement** (`AUTO_INCREMENT`).

💡 **Remarque.** Deux élèves peuvent porter le même nom, mais jamais le même `id`. La clé primaire sert justement à distinguer puis à retrouver précisément une ligne, par exemple pour la modifier ou la supprimer.

7.2 Le langage SQL

On dialogue avec MySQL grâce au langage **SQL** (*Structured Query Language*). Une instruction SQL s'appelle une **requête**. Par convention, on écrit les mots-clés en **MAJUSCULES** et on termine chaque requête par un point-virgule `;`.

7.2.1 Créer une table : CREATE TABLE

```
CREATE TABLE nom_table ( champ type contraintes, ... );
```

SQL

```
CREATE TABLE eleves (
1  id          INT AUTO_INCREMENT PRIMARY KEY,  -- clé primaire auto-incrémentée
2  nom         VARCHAR(60) NOT NULL,           -- texte court, obligatoire
3  classe     VARCHAR(20) NOT NULL,
4  moyenne    DECIMAL(4,2) DEFAULT 0          -- nombre : 2 décimales (ex. 14.50)
5 );
```

💡 **Remarque.** Les principaux types : `INT` (entier), `VARCHAR(n)` (chaîne d'au plus `n` caractères), `DECIMAL(t,d)` (nombre décimal), `TEXT` (texte long), `DATE` (une date). `NOT NULL` interdit une valeur vide.

7.2.2 Insérer des données : INSERT INTO

```
INSERT INTO table ( champs ) VALUES ( valeurs );
```

SQL

```
INSERT INTO eleves (nom, classe, moyenne)
1 VALUES ('Awa Bello', 'Tle II', 14.50);
2
3 -- On ne précise pas id : MySQL le génère tout seul (1, 2, 3, ...)
4 INSERT INTO eleves (nom, classe, moyenne)
5 VALUES ('Jean Mballa', 'Tle II', 9.75);
```

⚠ **Attention.** Les valeurs de type texte (`VARCHAR`, `TEXT`, `DATE`) se mettent entre apostrophes simples `'...'`. Les nombres, eux, s'écrivent sans apostrophes.

7.2.3 Lire des données : SELECT

C'est la requête la plus utilisée : elle **interroge** la table et renvoie les lignes demandées.

```
SELECT champs FROM table WHERE condition ORDER BY champ;
```

SQL

```
SELECT * FROM eleves;           -- toutes les colonnes, toutes les lignes
SELECT nom, moyenne FROM eleves; -- seulement deux colonnes
SELECT * FROM eleves WHERE moyenne >= 10; -- les admis
SELECT * FROM eleves WHERE classe = 'Tle TI'; -- filtre par classe
SELECT * FROM eleves ORDER BY moyenne DESC; -- du plus fort au plus faible
```

💡 **Remarque.** L'étoile `*` signifie « toutes les colonnes ». `ORDER BY ... ASC` trie en ordre croissant (par défaut), `DESC` en ordre décroissant. Le mot `WHERE` introduit une **condition** de filtrage.

7.2.4 Modifier des données : UPDATE

```
UPDATE table SET champ = valeur WHERE condition;
```

SQL

```
UPDATE eleves SET moyenne = 15.25 WHERE id = 1;
UPDATE eleves SET classe = 'Tle TI2' WHERE classe = 'Tle TI';
```

⚠ **Attention.** Ne jamais oublier le `WHERE` ! Sans condition, `UPDATE eleves SET moyenne = 0;` mettrait la moyenne de **tous** les élèves à zéro.

7.2.5 Supprimer des données : DELETE

```
DELETE FROM table WHERE condition;
```

SQL

```
DELETE FROM eleves WHERE id = 2; -- supprime l'élève n°2
DELETE FROM eleves WHERE moyenne < 5; -- supprime plusieurs lignes
```

⚠ **Attention.** Comme pour `UPDATE`, sans `WHERE` la requête `DELETE FROM eleves;` vide **toute** la table. On supprime presque toujours par clé primaire (`WHERE id = ...`).

7.3 phpMyAdmin

Définition

phpMyAdmin est une application web qui permet d'administrer une base MySQL à la souris, depuis le navigateur, sans écrire (au début) de code. Livré avec WampServer et XAMPP, on l'ouvre à l'adresse `http://localhost/phpmyadmin`.

On y crée une base, on y ajoute des tables et des lignes via des formulaires, et surtout on dispose d'un onglet **SQL** où l'on tape directement les requêtes vues plus haut pour les tester. C'est l'outil idéal pour préparer sa base **avant** de l'interfacer avec PHP.

Préparer sa base avec phpMyAdmin. (1) Ouvrir `http://localhost/phpmyadmin` ; (2) cliquer sur *Nouvelle base de données*, la nommer `lycee` ; (3) onglet *SQL* : coller le `CREATE TABLE eleves` puis *Exécuter* ; (4) ajouter quelques lignes avec `INSERT INTO`.

7.4 Se connecter à MySQL depuis PHP : `mysqli`

Pour piloter MySQL depuis un script, PHP fournit l'extension **mysqli** (*MySQL improved*). On l'utilise ici en **style procédural** : de simples fonctions `mysqli_xxx()`.

Définition

`mysqli_connect($hote, $utilisateur, $mot_de_passe, $base)` ouvre une connexion au serveur MySQL et renvoie un **identifiant de connexion**.

En local (WampServer/XAMPP), l'hôte est `localhost`, l'utilisateur est `root` et le mot de passe est en général **vide**.

PHP

```

1 <?php
2 // connexion.php - on se connecte une seule fois, puis on inclut ce fichier
3 $hote = "localhost";
4 $user = "root";
5 $pass = ""; // mot de passe vide en local
6 $base = "lycee";
7
8 // Tentative de connexion
9 $cnx = mysqli_connect($hote, $user, $pass, $base);
10
11 // Gestion de l'erreur : si la connexion échoue, on arrête tout
12 if (!$cnx) {
13     die("Échec de la connexion : " . mysqli_connect_error());
14 }
15
16 // (Optionnel) accents corrects : on travaille en UTF-8
17 mysqli_set_charset($cnx, "utf8");
18 ?>

```

Le script `connexion.php`, réutilisé par toutes les pages.

Remarque. La fonction `die("...")` (synonyme de `exit`) affiche un message puis **stoppe** immédiatement le script. `mysqli_connect_error()` renvoie la raison exacte de l'échec (mauvais

mot de passe, base inexistante...).

7.5 Exécuter une requête : `mysqli_query`

Définition

`mysqli_query($cnx, $requete)` envoie une requête SQL au serveur. Pour un `SELECT`, elle renvoie un **jeu de résultats**; pour `INSERT` / `UPDATE` / `DELETE`, elle renvoie `true` en cas de succès, `false` en cas d'erreur.

PHP

```
<?php
1  require "connexion.php";    // on récupère $cnx
2
3
4  $sql = "SELECT * FROM eleves";
5  $res = mysqli_query($cnx, $sql);
6
7
8  if (!$res) {
9      die("Erreur de requête : " . mysqli_error($cnx));
10 }
11 ?>
```

Remarque. `require "connexion.php";` insère le fichier de connexion; on récupère ainsi la variable `$cnx` sans réécrire le code à chaque page. `mysqli_error($cnx)` donne le détail d'une erreur de requête.

7.6 Lire les résultats d'un `SELECT`

Le jeu de résultats contient plusieurs lignes. On les parcourt **une par une** avec une boucle `while` et la fonction `mysqli_fetch_assoc` : elle renvoie la ligne suivante sous forme de **tableau associatif** (`$ligne["nom"]`, `$ligne["moyenne"]` ...), puis `false` quand il n'y a plus de ligne.

```
while ($ligne = mysqli_fetch_assoc($res)) { ... }
```

PHP

```
<?php
1  require "connexion.php";
2  $res = mysqli_query($cnx, "SELECT * FROM eleves ORDER BY nom");
3  ?>
4  <table border="1">
5      <tr><th>Nom</th><th>Classe</th><th>Moyenne</th></tr>
6  <?php
7      // On parcourt chaque ligne du résultat
8      while ($ligne = mysqli_fetch_assoc($res)) {
9          echo "<tr>";
10         echo "<td>" . $ligne["nom"] . "</td>";
11         echo "<td>" . $ligne["classe"] . "</td>";
12         echo "<td>" . $ligne["moyenne"] . "</td>";
13     }
```

```

14     echo "</tr>";
15 }
16 ?>
17 </table>

```

Affichage de la table `eleves` dans un tableau HTML.

7.6.1 Les autres fonctions de lecture

- `mysqli_fetch_assoc($res)` : ligne en tableau **associatif** (`$ligne["nom"]`) — la plus lisible ;
- `mysqli_fetch_row($res)` : ligne en tableau **numéroté** (`$ligne[0]` , `$ligne[1]` ...) ;
- `mysqli_fetch_array($res)` : les deux à la fois (par index **et** par nom) ;
- `mysqli_num_rows($res)` : le **nombre** de lignes renvoyées.

PHP

```

1 <?php
2     require "connexion.php";
3     $res = mysqli_query($cnx, "SELECT * FROM eleves WHERE moyenne >= 10");
4
5     // Compter avant d'afficher
6     $nb = mysqli_num_rows($res);
7     echo "<p>Nombre d'admis : $nb</p>";
8
9     if ($nb == 0) {
10         echo "<p>Aucun élève admis pour le moment.</p>";
11     } else {
12         while ($e = mysqli_fetch_assoc($res)) {
13             echo "<p>" . $e["nom"] . " - " . $e["moyenne"] . "</p>";
14         }
15     }
16 ?>

```

7.7 Insérer, modifier, supprimer depuis PHP

Les requêtes `INSERT` , `UPDATE` et `DELETE` se lancent aussi avec `mysqli_query` . Le plus souvent, leurs valeurs proviennent d'un **formulaire** (`$_POST` ou `$_GET`).

7.7.1 Ajouter (INSERT) à partir d'un formulaire

HTML

```

1 <!-- ajouter.php : le formulaire -->
2 <form method="post" action="ajouter.php">
3     Nom :      <input type="text"   name="nom"><br>
4     Classe :   <input type="text"   name="classe"><br>
5     Moyenne :  <input type="number" name="moyenne" step="0.01"><br>
6     <input type="submit" value="Ajouter">
7 </form>

```

PHP

```

1 <?php
2     require "connexion.php";
3

```

```

4 // Le formulaire a-t-il été envoyé ?
5 if (isset($_POST["nom"])) {
6     // On récupère les données saisies
7     $nom      = $_POST["nom"];
8     $classe   = $_POST["classe"];
9     $moyenne  = $_POST["moyenne"];
10
11     // On construit puis on exécute la requête INSERT
12     $sql = "INSERT INTO eleves (nom, classe, moyenne)
13           VALUES ('$nom', '$classe', $moyenne)";
14
15     if (mysqli_query($cnx, $sql)) {
16         echo "<p>Élève ajouté avec succès.</p>";
17     } else {
18         echo "<p>Erreur : " . mysqli_error($cnx) . "</p>";
19     }
20 }
21 ?>

```

⚠ Attention. Dans la requête, le `$nom` et la `$classe` sont du texte : ils restent entre apostrophes `'$nom'`. La `$moyenne` est un nombre : pas d'apostrophes.

7.7.2 Modifier (UPDATE)

PHP

```

1 <?php
2     require "connexion.php";
3
4     $id      = $_POST["id"];      // id de l'élève à modifier
5     $moyenne = $_POST["moyenne"]; // nouvelle moyenne
6
7     $sql = "UPDATE eleves SET moyenne = $moyenne WHERE id = $id";
8
9     if (mysqli_query($cnx, $sql)) {
10         echo "<p>Moyenne mise à jour.</p>";
11     } else {
12         echo "<p>Erreur : " . mysqli_error($cnx) . "</p>";
13     }
14 ?>

```

7.7.3 Supprimer (DELETE)

On supprime une ligne précise grâce à sa clé primaire. L'`id` est souvent passé dans l'URL (`supprimer.php?id=2`), donc lu dans `$_GET`.

PHP

```

1 <?php
2     require "connexion.php";
3
4     $id = $_GET["id"];      // ex. supprimer.php?id=2
5
6     $sql = "DELETE FROM eleves WHERE id = $id";
7
8     if (mysqli_query($cnx, $sql)) {

```

```

9      echo "<p>Élève supprimé.</p>";
10    } else {
11      echo "<p>Erreur : " . mysqli_error($cnx) . "</p>";
12    }
13  ?>

```

7.8 Fermer la connexion : mysqli_close

À la fin du script, on libère la connexion au serveur.

PHP

```

1  <?php
2      mysqli_close($cnx);    // bonne pratique en fin de page
3  ?>

```

 **Remarque.** PHP ferme automatiquement la connexion à la fin de l'exécution de la page ; `mysqli_close` reste une bonne habitude, surtout pour les longs scripts.

7.9 Application fil rouge : gestion des élèves

Réunissons tout dans une **mini-application** de gestion des élèves : on affiche la liste, on ajoute un élève, on en supprime un. Trois fichiers suffisent.

7.9.1 1. connexion.php

PHP

```

1  <?php
2      $cnx = mysqli_connect("localhost", "root", "", "lycee");
3      if (!$cnx) {
4          die("Connexion impossible : " . mysqli_connect_error());
5      }
6      mysqli_set_charset($cnx, "utf8");
7  ?>

```

7.9.2 2. liste.php — afficher, ajouter, supprimer

PHP

```

1  <?php require "connexion.php"; ?>
2  <!DOCTYPE html>
3  <html lang="fr">
4  <head><meta charset="utf-8"><title>Gestion des élèves</title></head>
5  <body>
6      <h1>Liste des élèves - Lycée de Maroua</h1>
7
8  <?php
9      // --- AJOUT (formulaire envoyé en POST) ---
10     if (isset($_POST["nom"])) {
11         $nom      = $_POST["nom"];
12         $classe   = $_POST["classe"];
13         $moyenne  = $_POST["moyenne"];
14         $sql = "INSERT INTO eleves (nom, classe, moyenne)

```

```

18         VALUES ('$nom', '$classe', $moyenne)";
19     mysqli_query($cnx, $sql);
20 }
21
22 // --- SUPPRESSION (id passé dans l'URL) ---
23 if (isset($_GET["suppr"])) {
24     $id = $_GET["suppr"];
25     mysqli_query($cnx, "DELETE FROM eleves WHERE id = $id");
26 }
27 ?>
28
29 <!-- Formulaire d'ajout -->
30 <form method="post" action="liste.php">
31     <input type="text" name="nom" placeholder="Nom">
32     <input type="text" name="classe" placeholder="Classe">
33     <input type="number" name="moyenne" step="0.01" placeholder="Moyenne">
34     <input type="submit" value="Ajouter">
35 </form>
36
37 <!-- Tableau des élèves -->
38 <table border="1" cellpadding="6">
39     <tr><th>Id</th><th>Nom</th><th>Classe</th><th>Moyenne</th><th>Action</th></tr>
40 <?php
41     $res = mysqli_query($cnx, "SELECT * FROM eleves ORDER BY moyenne DESC");
42     while ($e = mysqli_fetch_assoc($res)) {
43         echo "<tr>";
44         echo "<td>" . $e["id"] . "</td>";
45         echo "<td>" . $e["nom"] . "</td>";
46         echo "<td>" . $e["classe"] . "</td>";
47         echo "<td>" . $e["moyenne"] . "</td>";
48         // Lien de suppression : on passe l'id dans l'URL
49         echo "<td><a href='liste.php?suppr=" . $e["id"] . "'>Supprimer</a></td>";
50         echo "</tr>";
51     }
52 ?>
53 </table>
54
55 <?php mysqli_close($cnx); ?>
56 </body>
57 </html>

```

Une seule page gère l'affichage, l'ajout et la suppression : c'est une mini-application CRUD complète.

💡 Remarque. **CRUD** résume les quatre opérations de base d'une application de gestion : *Create* (INSERT), *Read* (SELECT), *Update* (UPDATE) et *Delete* (DELETE). Tu sais désormais les réaliser toutes les quatre.

⚠ Attention. Ces exemples sont volontairement simples pour le programme TI. Dans une vraie application, on **n'insère jamais** directement `$_POST` dans une requête (risque d'*injection SQL*) : on nettoie les données. Retiens l'idée ; les techniques de sécurisation se voient en études supérieures.

7.10 Exercices

► Application

Exercice 1 Créer une table

★ Écris la requête SQL qui crée une table `produits` avec les champs : `id` (entier, clé primaire auto-incrémentée), `designation` (texte, 50 caractères), `prix` (nombre décimal), `stock` (entier).

Exercice 2 Insérer des lignes

★ Insère dans la table `produits` deux articles d'une boutique de Yaoundé : un « Cahier 200 pages » à 350 FCFA (stock 120) et un « Stylo bleu Bic » à 100 FCFA (stock 500).

Exercice 3 Sélectionner les admis

★ Écris la requête qui affiche le nom et la moyenne de tous les élèves **admis** (moyenne supérieure ou égale à 10), triés de la meilleure moyenne à la moins bonne.

Exercice 4 Mettre à jour

★ Écris la requête qui augmente de 50 le stock du produit dont l'`id` vaut 1.

Exercice 5 Supprimer

★ Écris la requête qui supprime tous les produits en rupture de stock (`stock = 0`).

Exercice 6 Afficher une table en HTML

★★ Écris le script PHP qui se connecte à la base `lycee`, lit la table `produits` et affiche `designation`, `prix` et `stock` dans un tableau HTML.

Exercice 7 Compter les enregistrements

★★ Écris le script PHP qui affiche le **nombre** d'élèves de la classe `'Tle TI'` contenus dans la table `eleves`.

► Entraînement

Exercice 8 Recherche par nom

★★ Un formulaire envoie en POST un champ `recherche`. Écris le script PHP qui affiche tous les élèves dont le nom **contient** le texte saisi (on utilisera `LIKE` et le caractère `%`).

Exercice 9 Moyenne de la classe

★★ Écris une requête SQL qui calcule la **moyenne générale** (moyenne des moyennes) de tous les élèves de la table `eleves` (fonction `AVG`).

Exercice 10 Formulaire de modification

★★★ Écris un script `modifier.php` qui reçoit en POST un `id` et une nouvelle `moyenne`, met à jour l'élève correspondant, puis affiche un message de confirmation.

7.11 Corrigés

► Corrigé de l'exercice 1

```
SQL
CREATE TABLE produits (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  designation VARCHAR(50) NOT NULL,
  prix        DECIMAL(8,2) DEFAULT 0,
  stock       INT DEFAULT 0
);
```

► Corrigé de l'exercice 2

SQL

```
1 INSERT INTO produits (designation, prix, stock)
2 VALUES ('Cahier 200 pages', 350, 120);
3
4 INSERT INTO produits (designation, prix, stock)
5 VALUES ('Stylo bleu Bic', 100, 500);
```

► Corrigé de l'exercice 3

SQL

```
1 SELECT nom, moyenne FROM eleves
2 WHERE moyenne >= 10
3 ORDER BY moyenne DESC;
```

► Corrigé de l'exercice 4

SQL

```
1 UPDATE produits SET stock = stock + 50 WHERE id = 1;
```

► Corrigé de l'exercice 5

SQL

```
1 DELETE FROM produits WHERE stock = 0;
```

► Corrigé de l'exercice 6

PHP

```
1 <?php
2 $cnx = mysqli_connect("localhost", "root", "", "lycee");
3 if (!$cnx) {
4     die("Connexion impossible : " . mysqli_connect_error());
5 }
6 $res = mysqli_query($cnx, "SELECT * FROM produits");
7 ?>
8 <table border="1">
9     <tr><th>Désignation</th><th>Prix (FCFA)</th><th>Stock</th></tr>
10 <?php
11     while ($p = mysqli_fetch_assoc($res)) {
12         echo "<tr>";
13         echo "<td>" . $p["designation"] . "</td>";
14         echo "<td>" . $p["prix"] . "</td>";
15         echo "<td>" . $p["stock"] . "</td>";
16         echo "</tr>";
17     }
18     mysqli_close($cnx);
19 ?>
20 </table>
```

► Corrigé de l'exercice 7

PHP

```

1 <?php
2 $cnx = mysqli_connect("localhost", "root", "", "lycee");
3 $res = mysqli_query($cnx, "SELECT * FROM eleves WHERE classe = 'Tle TI'");
4 $nb = mysqli_num_rows($res);
5 echo "<p>La classe de Tle TI compte $nb élève(s).</p>";
6 mysqli_close($cnx);
7 ?>

```

► Corrigé de l'exercice 8

On colle le texte saisi entre deux % : LIKE '%mot%' cherche le mot n'importe où dans le nom.

PHP

```

1 <?php
2 $cnx = mysqli_connect("localhost", "root", "", "lycee");
3 $mot = $_POST["recherche"];
4
5 $sql = "SELECT * FROM eleves WHERE nom LIKE '%$mot%'";
6 $res = mysqli_query($cnx, $sql);
7
8 if (mysqli_num_rows($res) == 0) {
9     echo "<p>Aucun élève trouvé.</p>";
10 } else {
11     while ($e = mysqli_fetch_assoc($res)) {
12         echo "<p>" . $e["nom"] . " (" . $e["classe"] . ")</p>";
13     }
14 }
15 mysqli_close($cnx);
16 ?>

```

► Corrigé de l'exercice 9

SQL

```

1 SELECT AVG(moyenne) AS moyenne_generale FROM eleves;

```

► Corrigé de l'exercice 10

PHP

```

1 <?php
2 // modifier.php
3 $cnx = mysqli_connect("localhost", "root", "", "lycee");
4 if (!$cnx) {
5     die("Connexion impossible : " . mysqli_connect_error());
6 }
7
8 if (isset($_POST["id"])) {
9     $id = $_POST["id"];
10     $moyenne = $_POST["moyenne"];
11
12     $sql = "UPDATE eleves SET moyenne = $moyenne WHERE id = $id";
13
14     if (mysqli_query($cnx, $sql)) {
15         echo "<p>Élève n°$id mis à jour : nouvelle moyenne = $moyenne.</p>";
16     } else {

```



```

17         echo "<p>Erreur : " . mysqli_error($cnx) . "</p>";
18     }
19 }
20 mysqli_close($cnx);
21 ?>

```

★ À retenir

Pour interfacer une base MySQL en PHP procédural : **(1)** se connecter avec `mysqli_connect` (et die + `mysqli_connect_error` en cas d'échec) ; **(2)** exécuter la requête avec `mysqli_query` ; **(3)** pour un `SELECT`, parcourir le résultat avec `while ($l = mysqli_fetch_assoc($res))` ; **(4)** fermer avec `mysqli_close`. En SQL, n'oublie **jamais** le `WHERE` dans un `UPDATE` ou un `DELETE`.

Introduction au langage C

Au programme

Découvrir le langage C, un langage **compilé** servant à créer des applications monopostes (*stand alone*). Écrire, compiler et exécuter un premier programme, manipuler les types de base, les constantes, les entrées/sorties (`printf`, `scanf`) et les opérateurs. C'est le socle de toute programmation procédurale.

8.1 Qu'est-ce que le langage C ?

Définition

Le **langage C** est un langage de programmation **compilé** et **procédural**. Un programme C est traduit *une fois pour toutes* en langage machine par un outil appelé **compilateur** ; on obtient alors un fichier **exécutable** que l'ordinateur lance directement, sans avoir besoin du code source ni du compilateur.

Le C sert à écrire des **applications monopostes** (en anglais *stand alone*) : des logiciels qui fonctionnent seuls sur une machine, sans serveur ni navigateur. C'est par exemple le cas d'une calculatrice, d'un logiciel de gestion d'une boutique à Douala, ou d'un programme de calcul de moyennes pour un lycée.

💡 Remarque. Contrairement à PHP qui est *interprété* (le serveur lit et exécute le code à chaque visite), le C est *compilé* : on traduit le code source en exécutable **avant** de l'utiliser. Un programme compilé est donc plus rapide et peut être distribué sans dévoiler le code source.

Propriété Du code source à l'exécutable

Le fichier source porte l'extension `.c`. Le compilateur (par exemple `gcc`) le transforme en un fichier exécutable (`.exe` sous Windows). On résume la chaîne ainsi : **écrire** → **compiler** → **exécuter**.

8.2 Structure d'un programme C

Tout programme C s'organise autour d'une fonction obligatoire appelée `main` (« principale »). C'est par elle que l'exécution commence.

```
#include <stdio.h>
int main() { instructions... return 0; }
```

Voici le tout premier programme : il affiche un message à l'écran.

```
C
#include <stdio.h> /* bibliotheque des entrees/sorties standard */
int main()         /* fonction principale : point de depart */
{
    printf("Bonjour le monde !\n"); /* affiche le message */
    return 0;        /* indique au systeme que tout s'est bien passe */
}
```

Premier programme C : il affiche « Bonjour le monde ! ».

Analysons chaque ligne :

- `#include <stdio.h>` charge la bibliothèque *standard input/output*, indispensable pour utiliser `printf` et `scanf` ;
- `int main()` déclare la fonction principale ; les accolades `{` et `}` délimitent son contenu (le *corps*) ;
- `printf(...)` affiche du texte à l'écran ;
- `return 0;` termine le programme en signalant qu'il s'est bien déroulé.

⚠ Attention. Chaque instruction se termine par un point-virgule `;`. Le C distingue les majuscules des minuscules : `main` et `Main` ne sont pas la même chose. Les commentaires s'écrivent entre `/*` et `*/`, ou après `//` jusqu'à la fin de la ligne.

8.3 Compiler et exécuter : les IDE

Pour écrire et compiler du C, on utilise un **IDE** (environnement de développement intégré) qui réunit l'éditeur de texte et le compilateur. Les plus répandus dans les lycées camerounais sont **Code ::Blocks** et **Dev-C++**.

Écrire, compiler, exécuter avec Code ::Blocks.

- 1) **Écrire** : créer un nouveau fichier source `.c` et taper le programme ;
- 2) **Compiler** : cliquer sur *Build* (icône engrenage) ; le compilateur vérifie la syntaxe et produit l'exécutable ;
- 3) **Exécuter** : cliquer sur *Run* (flèche verte) ; une fenêtre console s'ouvre et affiche le résultat. On peut aussi tout faire d'un coup avec *Build and Run* (engrenage + flèche).

8.3.1 Gérer les erreurs de compilation

Quand le compilateur rencontre une faute (point-virgule oublié, accolade non fermée, nom mal orthographié), il refuse de produire l'exécutable et affiche un message d'**erreur** indiquant le *numéro de la ligne* fautive.

Exemple. Si l'on oublie le `;` après `printf("Bonjour")`, gcc affiche un message du type « *expected ';' before 'return'* » : il faut corriger la ligne indiquée, puis recompiler.

💡 **Remarque.** On distingue les **erreurs** (*errors*), qui empêchent la compilation, des **avertissements** (*warnings*), qui n'empêchent pas la compilation mais signalent un risque. On corrige toujours les erreurs en premier, en partant de la **première** ligne signalée.

8.4 Les types de base

Une **variable** est un espace mémoire nommé qui stocke une valeur. En C, on doit **déclarer** chaque variable en précisant son **type** avant de l'utiliser.

Type	Contenu	Exemple
<code>int</code>	nombre entier	<code>int age = 17;</code>
<code>float</code>	nombre décimal (simple)	<code>float prix = 250.0;</code>
<code>double</code>	nombre décimal (précis)	<code>double pi = 3.14159;</code>
<code>char</code>	un seul caractère	<code>char lettre = 'A';</code>

type nomVariable = valeur ;

Exemple : `int n = 5;` ou simplement `int n;` (sans valeur).

C

```

1 #include <stdio.h>
2
3 int main()
4 {
5     int age = 17;           /* un entier */
6     float taille = 1.75;    /* un decimal */
7     double moyenne = 14.85; /* un decimal plus precis */
8     char initiale = 'A';    /* un caractere, entre apostrophes */
9
10    printf("Age : %d ans\n", age);
11    printf("Taille : %.2f m\n", taille);
12    printf("Moyenne : %.2f / 20\n", moyenne);
13    printf("Initiale : %c\n", initiale);
14    return 0;
15 }
```

Déclaration et affichage des quatre types de base.

⚠ **Attention.** Un `char` se note entre apostrophes simples : `'A'`. Une chaîne de caractères (plusieurs lettres) se note entre guillemets doubles : `"Awa"`. Ne confonds pas `'A'` (un caractère) et `"A"` (une chaîne).

8.5 Les constantes

Une **constante** est une valeur qui ne change jamais pendant l'exécution. En C, on la définit de deux façons.

- Avec la directive `#define`, placée **avant** le `main` :
- Avec le mot-clé `const`, dans une déclaration de variable :

```
C
1 #include <stdio.h>
2 #define PI 3.14159          /* constante par #define (pas de point-virgule) */
3 #define ETABLISSEMENT "Lycee de Maroua"
4
5 int main()
6 {
7     const int TVA = 19;      /* constante par const */
8     printf("%s\n", ETABLISSEMENT);
9     printf("PI = %f, TVA = %d %%\n", PI, TVA);
10    return 0;
11 }
```

Deux manières de définir une constante : `#define` et `const`.

Remarque. Par convention, on écrit les noms de constantes en **MAJUSCULES**. La directive `#define` ne se termine **pas** par un point-virgule.

8.6 Les entrées/sorties : printf et scanf

8.6.1 Afficher avec printf

La fonction `printf` affiche du texte. Pour insérer la valeur d'une variable, on utilise un **format** (commençant par `%`) à l'endroit voulu, puis on donne la variable après la virgule.

Format	Pour afficher...
<code>%d</code>	un entier (<code>int</code>)
<code>%f</code>	un décimal (<code>float</code> / <code>double</code>)
<code>%c</code>	un caractère (<code>char</code>)
<code>%s</code>	une chaîne de caractères
<code>\n</code>	un retour à la ligne

```
C
1 #include <stdio.h>
2
3 int main()
4 {
5     int n = 25;
6     float p = 1500.5;
7     printf("Quantite : %d articles\n", n);
8     printf("Prix : %.2f FCFA\n", p);    /* %.2f : 2 chiffres apres la virgule */
9     return 0;
10 }
```

`%d` pour un entier, `%.2f` pour un décimal à deux chiffres.

8.6.2 Saisir avec scanf

La fonction `scanf` permet à l'utilisateur de **saisir** une valeur au clavier. On indique le format, puis l'adresse de la variable avec le symbole `&` (« esperluette ») devant son nom.

`scanf("%d", &variable);` — le `&` est obligatoire.

C

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int age;
6     printf("Quel est ton age ? ");
7     scanf("%d", &age);    /* le & est indispensable */
8     printf("Tu as %d ans.\n", age);
9     return 0;
10 }
```

Saisie d'un entier au clavier avec `scanf` et le `&`.

⚠ Attention. Dans un `scanf`, on met **toujours** le symbole `&` devant le nom de la variable (sauf pour les chaînes `%s`). Oublier le `&` est l'erreur la plus fréquente du débutant : le programme se compile mais **plante** à l'exécution.

💡 Remarque. Le code `\n` dans un texte signifie « passer à la ligne suivante ». Ce n'est pas affiché tel quel : il déplace le curseur à la ligne. Sans `\n`, tous les affichages restent collés sur la même ligne.

8.7 Les opérateurs

8.7.1 Arithmétiques, comparaison, logiques

Catégorie	Opérateurs	Exemple
Arithmétiques	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code>	<code>17 % 5</code> vaut <code>2</code> (reste)
Comparaison	<code>==</code> <code>!=</code> <code><</code> <code>></code> <code><=</code> <code>>=</code>	<code>a == b</code> (égalité)
Logiques	<code>&&</code> <code> </code> <code>!</code>	<code>x > 0 && x < 20</code>
Affectation	<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code>	<code>s += 1</code> (ajoute 1 à <code>s</code>)
Incrément.	<code>++</code> <code>--</code>	<code>n++</code> (ajoute 1 à <code>n</code>)

⚠ Attention. Attention à ne pas confondre l'**affectation** `=` (« reçoit ») et la **comparaison** `==` (« est égal à »). `a = 5` range 5 dans `a` ; `a == 5` teste si `a` vaut 5.

8.7.2 Affectation composée et incrémentation

Les opérateurs composés sont des raccourcis très utilisés :

- `s += 3` équivaut à `s = s + 3` ; de même `-=`, `*=`, `/=` ;
- `n++` ajoute 1 à `n` (incrément) ; `n--` retire 1 (décrément).

C

```
1 #include <stdio.h>
2
3 int main()
```

```

1 {
2     int total = 10;
3     total += 5;    /* total vaut maintenant 15 */
4     total -= 2;    /* total vaut 13 */
5     total++;       /* total vaut 14 */
6     printf("Total = %d\n", total);
7
8     int reste = 17 % 5; /* reste de 17 divise par 5 : 2 */
9     printf("17 modulo 5 = %d\n", reste);
10    return 0;
11 }

```

Affectations composées, incrémentation et opérateur modulo `%`.

8.8 Exemples complets

8.8.1 Somme de deux entiers saisis

```

C
1 #include <stdio.h>
2
3 int main()
4 {
5     int a, b, somme;
6     printf("Entrez deux entiers : ");
7     scanf("%d %d", &a, &b); /* deux valeurs separees par un espace */
8     somme = a + b;
9     printf("La somme est %d\n", somme);
10    return 0;
11 }

```

Saisie de deux entiers et affichage de leur somme.

8.8.2 Convertir une note sur 20 en pourcentage

```

C
1 #include <stdio.h>
2
3 int main()
4 {
5     float note, pourcent;
6     printf("Entrez la note sur 20 : ");
7     scanf("%f", &note);
8     pourcent = note / 20.0 * 100.0; /* regle de trois */
9     printf("Soit %.1f %%\n", pourcent);
10    return 0;
11 }

```

Conversion d'une note sur 20 en pourcentage.

8.8.3 Périmètre et aire d'un rectangle

```
C
#include <stdio.h>

int main()
{
    float longueur, largeur;
    printf("Longueur (m) : ");
    scanf("%f", &longueur);
    printf("Largeur (m) : ");
    scanf("%f", &largeur);

    float perimetre = 2 * (longueur + largeur);
    float aire = longueur * largeur;
    printf("Perimetre = %.2f m\n", perimetre);
    printf("Aire = %.2f m2\n", aire);
    return 0;
}
```

Calcul du périmètre et de l'aire d'un terrain rectangulaire.

8.9 Exercices

► Application

Exercice 1 Message d'accueil

★ Écris un programme C qui affiche, chacun sur une ligne, ton nom, ton lycée et ta ville (exemple : Awa, Lycée de Maroua, Maroua).

Exercice 2 Somme et produit

★ Demande à l'utilisateur de saisir deux entiers, puis affiche leur somme et leur produit.

Exercice 3 Moyenne de trois notes

★ Demande trois notes sur 20 (type `float`) et affiche leur moyenne avec deux décimales.

Exercice 4 Âge de l'élève

★ Demande l'âge de l'utilisateur (entier) et affiche le message « *Tu as ... ans.* ».

► Entraînement

Exercice 5 Conversion FCFA → euros

★★ Sachant que 1 euro vaut 656 FCFA, demande un montant en FCFA et affiche sa valeur en euros (deux décimales).

Exercice 6 Échanger deux variables

★★ Demande deux entiers `a` et `b`, échange leurs valeurs à l'aide d'une variable temporaire, puis affiche-les après l'échange.

Exercice 7 Quotient et reste

★★ Demande deux entiers et affiche le **quotient** (/) et le **reste** (%) de la division du premier par le second.

Exercice 8 Périmètre d'un cercle

★★ En utilisant une constante `PI` définie par `#define`, demande le rayon d'un cercle et affiche son périmètre ($2 \times \pi \times r$) et son aire ($\pi \times r^2$).

Exercice 9 Prix TTC

★★★ Un commerçant de Bafoussam applique une TVA de 19,25 %. Demande le prix hors taxe (HT) d'un article, puis affiche le prix toutes taxes comprises (TTC) avec deux décimales.

Exercice 10 Compteur

★★★ Déclare un entier `n` valant 100. À l'aide des opérateurs composés et de l'incrémement, applique : ajoute 50, retire 10, multiplie par 2, ajoute 1. Affiche le résultat après chaque opération.

8.10 Corrigés

► Corrigé de l'exercice 1

```
C
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("Awa\n");
6     printf("Lycee de Maroua\n");
7     printf("Maroua\n");
8     return 0;
9 }
```

► Corrigé de l'exercice 2

```
C
1 #include <stdio.h>
2
3 int main()
4 {
5     int a, b;
6     printf("Entrez deux entiers : ");
7     scanf("%d %d", &a, &b);
8     printf("Somme = %d\n", a + b);
9     printf("Produit = %d\n", a * b);
10    return 0;
11 }
```

► Corrigé de l'exercice 3

```
C
1 #include <stdio.h>
2
3 int main()
4 {
5     float n1, n2, n3, moyenne;
6     printf("Entrez trois notes : ");
7     scanf("%f %f %f", &n1, &n2, &n3);
8     moyenne = (n1 + n2 + n3) / 3.0;
9     printf("Moyenne = %.2f / 20\n", moyenne);
10    return 0;
11 }
```

► Corrigé de l'exercice 4

```
C
1 #include <stdio.h>
2
3 int main()
4 {
5     int age;
6     printf("Quel est ton age ? ");
7     scanf("%d", &age);
8     printf("Tu as %d ans.\n", age);
9     return 0;
10 }
```

► Corrigé de l'exercice 5

```
C
1 #include <stdio.h>
2
3 int main()
4 {
5     float fcfa, euros;
6     printf("Montant en FCFA : ");
7     scanf("%f", &fcfa);
8     euros = fcfa / 656.0;
9     printf("Soit %.2f euros\n", euros);
10    return 0;
11 }
```

► Corrigé de l'exercice 6

```
C
1 #include <stdio.h>
2
3 int main()
4 {
5     int a, b, temp;
6     printf("Entrez a et b : ");
7     scanf("%d %d", &a, &b);
8
9     temp = a;    /* on garde a dans temp */
10    a = b;       /* a recoit b */
11    b = temp;    /* b recoit l'ancien a */
12
13    printf("Après échange : a = %d, b = %d\n", a, b);
14    return 0;
15 }
```

► Corrigé de l'exercice 7

```
C
1 #include <stdio.h>
2
3 int main()
4 {
```

```

1  int a, b;
2  printf("Entrez deux entiers : ");
3  scanf("%d %d", &a, &b);
4  printf("Quotient = %d\n", a / b);
5  printf("Reste = %d\n", a % b);
6  return 0;
7  }

```

► Corrigé de l'exercice 8

```

C
1  #include <stdio.h>
2  #define PI 3.14159
3
4  int main()
5  {
6      float r, perimetre, aire;
7      printf("Rayon du cercle : ");
8      scanf("%f", &r);
9      perimetre = 2 * PI * r;
10     aire = PI * r * r;
11     printf("Perimetre = %.2f\n", perimetre);
12     printf("Aire = %.2f\n", aire);
13     return 0;
14 }

```

► Corrigé de l'exercice 9

```

C
1  #include <stdio.h>
2
3  int main()
4  {
5      float ht, ttc;
6      printf("Prix HT (FCFA) : ");
7      scanf("%f", &ht);
8      ttc = ht * 1.1925; /* +19,25 % de TVA */
9      printf("Prix TTC = %.2f FCFA\n", ttc);
10     return 0;
11 }

```

► Corrigé de l'exercice 10

```

C
1  #include <stdio.h>
2
3  int main()
4  {
5      int n = 100;
6      n += 50; printf("n = %d\n", n); /* 150 */
7      n -= 10; printf("n = %d\n", n); /* 140 */
8      n *= 2; printf("n = %d\n", n); /* 280 */
9      n++; printf("n = %d\n", n); /* 281 */
10     return 0;
11 }

```

★ À retenir

Un programme C est **compilé** en exécutable. Il commence toujours par `#include <stdio.h>` et la fonction `int main() { ... return 0; }`. On déclare les variables avec leur type (`int`, `float`, `double`, `char`), on affiche avec `printf` (`%d`, `%f`, `%c`, `%s`, `\n`) et on saisit avec `scanf` sans jamais oublier le `&`.

Structures de contrôle en C

Au programme

Faire *décider et répéter* un programme C. On étudie les alternatives (`if`, `if...else`, `switch`), la valeur des booléens (0 = faux, tout le reste = vrai) et les trois boucles (`for`, `while`, `do...while`), puis `break` et `continue`. C'est le cœur de tout algorithme : sans elles, un programme ne ferait qu'exécuter ses lignes une à une.

9.1 Pourquoi des structures de contrôle ?

Jusqu'ici, un programme C s'exécute **ligne après ligne**, du haut vers le bas. Mais un vrai programme doit *choisir* (« si la moyenne est ≥ 10 , l'élève est admis, *sinon* il redouble ») et *répéter* (« afficher les 53 élèves de la classe »). Les **structures de contrôle** modifient cet ordre linéaire : elles dirigent le flot d'exécution.

Définition

Une **structure de contrôle** est une instruction qui commande l'enchaînement des autres instructions. On distingue deux grandes familles : les **structures alternatives** (faire un choix) et les **structures itératives** ou **boucles** (répéter un traitement).

Remarque. Tous les exemples de ce chapitre sont des programmes C complets et compilables. On suppose en tête de fichier la ligne `#include <stdio.h>`, qui donne accès à `printf` (afficher) et `scanf` (lire au clavier).

9.2 Les alternatives

9.2.1 Le if simple

L'instruction `if` exécute un bloc **seulement si** une condition est vraie.

```
if (condition) { instructions }
```

C

```
#include <stdio.h>

int main(void) {
    int age;
```

```

1 printf("Quel age as-tu ? ");
2 scanf("%d", &age);
3
4 if (age >= 18) {
5     printf("Tu es majeur.\n"); // exécuté si age >= 18
6 }
7
8 printf("Fin du programme.\n"); // toujours exécuté
9 return 0;
10 }

```

Le bloc entre accolades n'est exécuté que si la condition est vraie.

⚠ Attention. La condition est entre parenthèses, et on n'écrit **jamais** de point-virgule juste après le `)`. Écrire `if (age >= 18);` crée un bloc vide : le test ne sert alors à rien et le bloc suivant s'exécute toujours.

9.2.2 Le if...else

Le `else` fournit une issue *de secours* : il s'exécute quand la condition est fausse.

```
if (condition) { ... } else { ... }
```

C

```

1 #include <stdio.h>
2
3 int main(void) {
4     float moyenne;
5     printf("Saisis la moyenne de l'eleve : ");
6     scanf("%f", &moyenne);
7
8     if (moyenne >= 10) {
9         printf("Admis.\n");
10    } else {
11        printf("Redouble.\n");
12    }
13    return 0;
14 }

```

Exemple. Premier exemple classique : déterminer si un entier est **pair ou impair**. Un nombre est pair si le reste de sa division par 2 vaut 0 (opérateur `%`).

C

```

1 #include <stdio.h>
2
3 int main(void) {
4     int n;
5     printf("Entre un entier : ");
6     scanf("%d", &n);
7
8     if (n % 2 == 0) {
9         printf("%d est pair.\n", n);

```

```

10 } else {
11     printf("%d est impair.\n", n);
12 }
13 return 0;
14 }

```

pair / impair — le test $n \% 2 == 0$ vaut le cœur de l'algorithme.

Trace pour $n = 7$: $7 \% 2$ vaut 1, donc $1 == 0$ est faux → on exécute le `else` → affichage « 7 est impair. ».

9.2.3 Le if...else if...else

Pour trier parmi **plusieurs cas exclusifs**, on enchaîne des `else if`. Le programme évalue les conditions de haut en bas et s'arrête à la **première** qui est vraie.

```

C
#include <stdio.h>

int main(void) {
    float moy;
    printf("Moyenne de l'eleve (sur 20) : ");
    scanf("%f", &moy);

    if (moy >= 16) {
        printf("Mention Tres Bien.\n");
    } else if (moy >= 14) {
        printf("Mention Bien.\n");
    } else if (moy >= 12) {
        printf("Mention Assez Bien.\n");
    } else if (moy >= 10) {
        printf("Mention Passable.\n");
    } else {
        printf("Ajourne.\n");
    }
    return 0;
}

```

Une moyenne de 14,5 affiche « Mention Bien » : la 2e condition est la première vraie.

💡 Remarque. L'ordre des tests compte. Comme on s'arrête à la première condition vraie, il est inutile d'écrire `else if (moy >= 14 && moy < 16)` : si on est arrivé à ce `else if`, c'est que $moy \geq 16$ était déjà faux.

9.2.4 L'imbrication

On peut placer un `if` à l'intérieur d'un autre. C'est utile quand un second test ne se pose que si le premier est validé.

```

C
#include <stdio.h>

int main(void) {
    int age;
    float moyenne;
    printf("Age : ");    scanf("%d", &age);
}

```

```

7 printf("Moyenne : "); scanf("%f", &moyenne);
8
9 if (moyenne >= 10) { // condition extérieure
10     if (age <= 20) { // condition intérieure
11         printf("Admis et peut concourir.\n");
12     } else {
13         printf("Admis mais hors limite d'age.\n");
14     }
15 } else {
16     printf("Non admis.\n");
17 }
18 return 0;
19 }

```

⚠ **Attention.** Au-delà de deux niveaux d'imbrication, le code devient illisible. On préfère alors des conditions combinées (`&&` , `||`) ou un `switch` .

9.3 Le switch...case

Quand on compare **une même variable** à plusieurs valeurs *constantes*, le `switch` est plus clair qu'une longue chaîne de `else if` .

```
switch (expression) { case v1: ...break; case v2: ...break; default: ...}
```

```

C
#include <stdio.h>

int main(void) {
    int jour;
    printf("Numero du jour (1 a 7) : ");
    scanf("%d", &jour);

    switch (jour) {
        case 1: printf("Lundi\n"); break;
        case 2: printf("Mardi\n"); break;
        case 3: printf("Mercredi\n"); break;
        case 4: printf("Jeudi\n"); break;
        case 5: printf("Vendredi\n"); break;
        case 6:
        case 7: printf("Week-end\n"); break; // 6 et 7 partagent le meme bloc
        default: printf("Numero invalide\n");
    }
    return 0;
}

```

switch des jours — les cas 6 et 7 « tombent » sur le même traitement.

⚠ **Attention.** Le `break` est **indispensable** : sans lui, l'exécution continue dans le `case` suivant (effet « cascade » ou *fall-through*). On l'utilise volontairement ici pour 6 et 7 ; ailleurs, l'oublier est un bug fréquent. Le `default` (facultatif) traite tous les autres cas.

Remarque. Un `switch` ne fonctionne qu'avec des valeurs **entières** (ou des caractères, qui sont des entiers en C) et **constantes**. On ne peut pas écrire `case moy >= 10:` ni `case 3.5:`.

9.4 Les booléens en C

Le C n'a pas, à l'origine, de type « booléen » dédié. Il utilise les **entiers**.

Propriété *Vrai et faux en C*

La valeur `0` signifie **faux**. **Toute** autre valeur (`1`, `42`, `-3...`) signifie **vrai**. Une comparaison comme `a < b` produit `1` si elle est vraie, `0` si elle est fausse.

C

```
#include <stdio.h>

int main(void) {
    int x = 7;
    printf("%d\n", x > 3);    // affiche 1 (vrai)
    printf("%d\n", x > 10);   // affiche 0 (faux)

    if (x) {                  // x vaut 7, donc "vrai"
        printf("x est non nul, donc considere comme vrai.\n");
    }
    return 0;
}
```

Attention. Ne confonds pas `=` (affectation) et `==` (comparaison). `if (x = 0)` affecte `0` à `x` puis teste `0` (faux) : c'est presque toujours une erreur. On voulait `if (x == 0)`.

Remarque. Les opérateurs logiques sont `&&` (ET), `||` (OU) et `!` (NON). Par exemple `(moy >= 10 && moy < 12)` est vrai uniquement si les deux conditions le sont. Depuis la norme C99, on peut aussi inclure `<stdbool.h>` pour écrire `bool`, `true` et `false` — mais ce ne sont que des noms pour `1` et `0`.

9.5 Les boucles

Une **boucle** répète un bloc d'instructions tant qu'une condition reste vraie. Le C en propose trois.

9.5.1 La boucle `for`

Idéale quand on connaît à l'avance le nombre de répétitions. Elle regroupe sur une ligne l'*initialisation*, la *condition* et l'*incrément*.

```
for (initialisation ; condition ; incrément) { instructions }
```

C

```
#include <stdio.h>

int main(void) {
```

```

1  int i;
2  // Affiche la table de multiplication de 7
3  for (i = 1; i <= 10; i++) {
4      printf("7 x %d = %d\n", i, 7 * i);
5  }
6  return 0;
7  }

```

Table de multiplication — *i* va de 1 à 10, le corps s'exécute 10 fois.

Fonctionnement. (1) `i = 1` (une seule fois); (2) on teste `i <= 10` ; si vrai, on exécute le corps ; (3) on fait `i++` ; (4) retour au test. Quand `i` atteint 11, `11 <= 10` est faux : la boucle s'arrête.

9.5.2 La boucle while

Idéale quand on **ne sait pas** combien de fois répéter, mais qu'on connaît la *condition d'arrêt*. La condition est testée **avant** chaque tour.

```
while (condition) { instructions }
```

```

1  #include <stdio.h>
2
3  int main(void) {
4      int n = 10;
5      // Compte à rebours de 10 jusqu'à 1
6      while (n >= 1) {
7          printf("%d ", n);
8          n--; // sans cette ligne, boucle infinie !
9      }
10     printf("Decollage !\n");
11     return 0;
12 }

```

Compte à rebours : 10 9 8 7 6 5 4 3 2 1 Decollage !

⚠ Attention. Dans une boucle `while`, il faut que **quelque chose dans le corps fasse évoluer la condition** vers le faux (ici `n--`). Sinon la boucle ne s'arrête jamais : c'est une boucle infinie.

9.5.3 La boucle do...while

Variante du `while` où la condition est testée **après** le corps. Le bloc est donc exécuté **au moins une fois**. Pratique pour redemander une saisie.

```
do { instructions } while (condition);
```

```

1  #include <stdio.h>
2
3  int main(void) {
4      int note;

```

```

1 // Redemande tant que la note n'est pas entre 0 et 20
2 do {
3     printf("Entre une note (0 a 20) : ");
4     scanf("%d", &note);
5 } while (note < 0 || note > 20);
6
7 printf("Note valide : %d\n", note);
8 return 0;
9 }

```

do...while — la saisie est toujours demandée au moins une fois.

⚠ Attention. Le `do...while` se termine par un **point-virgule** après le `while(...)`. C'est le seul cas où l'on en met un après une parenthèse de condition.

Choisir la bonne boucle.

- 1) Je connais le **nombre exact** de tours (parcourir 1 à 10, n élèves déjà connus) → `for`.
- 2) Je m'arrête sur une **condition** dont je ne connais pas le nombre de tours (« tant que l'utilisateur ne tape pas 0 ») → `while`.
- 3) Comme le cas précédent, mais le corps doit s'exécuter **au moins une fois** (menu, contrôle de saisie) → `do...while`.

Les trois sont interchangeables : tout `for` peut se réécrire en `while`. On choisit celle qui rend l'intention la plus *lisible*.

9.6 break et continue

Ces deux mots-clés modifient le déroulement d'une boucle.

- `break` : **sort immédiatement** de la boucle (ou du `switch`).
- `continue` : **saute la fin du tour** et passe directement au tour suivant.

```

1 #include <stdio.h>
2
3 int main(void) {
4     int i;
5     for (i = 1; i <= 10; i++) {
6         if (i == 7) {
7             break;           // on quitte la boucle dès que i vaut 7
8         }
9         printf("%d ", i);
10    }
11    printf("\n");           // affiche : 1 2 3 4 5 6
12    return 0;
13 }

```

```

1 #include <stdio.h>
2
3 int main(void) {
4     int i;

```

```

1  for (i = 1; i <= 10; i++) {
2      if (i % 2 == 0) {
3          continue;    // on saute les nombres pairs
4      }
5      printf("%d ", i);
6  }
7  printf("\n");        // affiche : 1 3 5 7 9
8  return 0;
9  }

```

9.7 Exemples complets et tracés

9.7.1 Plus grand de deux, puis de trois nombres

```

C
1  #include <stdio.h>
2
3  int main(void) {
4      int a, b, c;
5      printf("Trois entiers : ");
6      scanf("%d %d %d", &a, &b, &c);
7
8      int max = a;        // on suppose a le plus grand
9      if (b > max) max = b; // b est-il plus grand ?
10     if (c > max) max = c; // c est-il plus grand ?
11
12     printf("Le plus grand est %d\n", max);
13     return 0;
14 }

```

Recherche du maximum — on part d'un candidat, on le corrige.

Trace pour `a=4, b=9, c=2` : `max=4` ; `9>4` vrai → `max=9` ; `2>9` faux → `max` reste 9. Résultat : 9.

9.7.2 Valeur absolue

```

C
1  #include <stdio.h>
2
3  int main(void) {
4      int x;
5      printf("Entre un entier : ");
6      scanf("%d", &x);
7
8      if (x < 0) {
9          x = -x;        // on rend x positif
10     }
11     printf("Valeur absolue : %d\n", x);
12     return 0;
13 }

```

9.7.3 Somme et moyenne de n notes

```

1 #include <stdio.h>
2
3 int main(void) {
4     int n, i;
5     float note, somme = 0;
6
7     printf("Combien de notes ? ");
8     scanf("%d", &n);
9
10    for (i = 1; i <= n; i++) {
11        printf("Note %d : ", i);
12        scanf("%f", &note);
13        somme = somme + note;    // accumulation
14    }
15
16    printf("Somme : %.2f\n", somme);
17    printf("Moyenne : %.2f\n", somme / n);
18    return 0;
19 }

```

Le « total » `somme` est mis à 0 puis augmenté à chaque tour.

9.7.4 Factorielle par boucle

La factorielle $n! = 1 \times 2 \times \dots \times n$ se calcule par accumulation multiplicative.

```

1 #include <stdio.h>
2
3 int main(void) {
4     int n, i;
5     long fact = 1;    // long : les factorielles grandissent vite
6
7     printf("Entre n : ");
8     scanf("%d", &n);
9
10    for (i = 2; i <= n; i++) {
11        fact = fact * i;
12    }
13    printf("%d! = %ld\n", n, fact);
14    return 0;
15 }

```

Trace pour `n=4` : `fact=1` ; `i=2` → `fact=2` ; `i=3` → `fact=6` ; `i=4` → `fact=24` . Résultat : 24.

9.7.5 PGCD par soustractions successives et par Euclide

```

1 #include <stdio.h>
2
3 int main(void) {
4     int a, b;
5     printf("Deux entiers positifs : ");
6     scanf("%d %d", &a, &b);
7
8 }

```

```

7 // Methode des soustractions successives
8 while (a != b) {
9     if (a > b) {
10         a = a - b;
11     } else {
12         b = b - a;
13     }
14 }
15 printf("PGCD = %d\n", a);
16 return 0;
17 }

```

```

C
1 #include <stdio.h>
2
3 int main(void) {
4     int a, b, r;
5     printf("Deux entiers positifs : ");
6     scanf("%d %d", &a, &b);
7
8     // Methode d'Euclide (par restes) : plus rapide
9     while (b != 0) {
10         r = a % b;           // reste de a divise par b
11         a = b;
12         b = r;
13     }
14     printf("PGCD = %d\n", a);
15     return 0;
16 }

```

PGCD(48, 36) : Euclide donne $48\%36=12$, puis $36\%12=0 \rightarrow \text{PGCD} = 12$.

9.8 Exercices

► Application

Exercice 1 Signe d'un nombre

★ Écris un programme qui lit un entier et affiche s'il est *strictement positif*, *strictement négatif* ou *nul* (utilise `if...else if...else`).

Exercice 2 Pair ou impair

★ Lis un entier au clavier et indique s'il est pair ou impair.

Exercice 3 Plus grand de trois

★ Lis trois entiers et affiche le plus grand des trois.

Exercice 4 Mention selon la moyenne

★★ Lis une moyenne entière sur 20. À l'aide d'un `switch` portant sur la **tranche** (`(int)(moy) / 2` ou un calcul de ton choix), affiche la mention : Très Bien (≥ 16), Bien (≥ 14), Assez Bien (≥ 12), Passable (≥ 10), Ajourné sinon.

Exercice 5 Somme des n premiers entiers

★★ Lis un entier `n` et calcule $1 + 2 + \dots + n$ avec une boucle `for`.

Exercice 6 Table de multiplication

★ Lis un entier `m` et affiche sa table de multiplication de 1 à 10.

► Entraînement

Exercice 7 Nombre de chiffres d'un entier

★★ Lis un entier positif et compte combien de chiffres il possède (ex. 4520 en a 4). *Indice* : divise par 10 jusqu'à atteindre 0.

Exercice 8 Contrôle de saisie

★★ À l'aide d'un `do...while`, demande l'âge d'un élève et redemande tant qu'il n'est pas compris entre 6 et 25.

Exercice 9 Compte à rebours

★ Lis un entier `n` et affiche le compte à rebours de `n` jusqu'à 1, puis « Termine! ».

Exercice 10 Factorielle

★★ Lis `n` et affiche `n!` en utilisant une boucle `while`.

Exercice 11 Vérifier si un nombre est premier

★★★ Lis un entier `n` (≥ 2) et détermine s'il est premier (divisible seulement par 1 et lui-même). *Indice* : teste les diviseurs de 2 à $n - 1$ et utilise `break` dès qu'un diviseur est trouvé.

Exercice 12 Suite de Fibonacci

★★★ La suite de Fibonacci est définie par $F_0 = 0$, $F_1 = 1$ et $F_n = F_{n-1} + F_{n-2}$. Lis `n` et affiche les `n` premiers termes (0, 1, 1, 2, 3, 5, 8...) à l'aide d'une boucle.

Exercice 13 PGCD d'Euclide

★★ Lis deux entiers positifs et affiche leur PGCD par la méthode des restes.

9.9 Corrigés

► Corrigé de l'exercice 1

```
C
#include <stdio.h>

int main(void) {
    int n;
    printf("Entre un entier : ");
    scanf("%d", &n);

    if (n > 0) {
        printf("Strictement positif.\n");
    } else if (n < 0) {
        printf("Strictement negatif.\n");
    } else {
        printf("Nul.\n");
    }
    return 0;
}
```

► Corrigé de l'exercice 2

```
C
1 #include <stdio.h>
2
3 int main(void) {
4     int n;
5     printf("Entre un entier : ");
6     scanf("%d", &n);
7
8     if (n % 2 == 0) {
9         printf("%d est pair.\n", n);
10    } else {
11        printf("%d est impair.\n", n);
12    }
13    return 0;
14 }
```

► Corrigé de l'exercice 3

```
C
1 #include <stdio.h>
2
3 int main(void) {
4     int a, b, c;
5     printf("Trois entiers : ");
6     scanf("%d %d %d", &a, &b, &c);
7
8     int max = a;
9     if (b > max) max = b;
10    if (c > max) max = c;
11
12    printf("Le plus grand est %d\n", max);
13    return 0;
14 }
```

► Corrigé de l'exercice 4

On ramène la moyenne à un *numéro de tranche* entier, puis on bascule dessus.

```
C
1 #include <stdio.h>
2
3 int main(void) {
4     int moy;
5     printf("Moyenne entiere (0 a 20) : ");
6     scanf("%d", &moy);
7
8     switch (moy / 2) {          // 16..20 -> 8,9,10 ; 14,15 -> 7 ; etc.
9         case 10:
10        case 9:
11        case 8: printf("Tres Bien\n"); break; // moy >= 16
12        case 7: printf("Bien\n"); break; // moy 14-15
13        case 6: printf("Assez Bien\n"); break; // moy 12-13
14        case 5: printf("Passable\n"); break; // moy 10-11
15        default: printf("Ajourne\n"); // moy < 10
16    }
17    return 0;
18 }
```



```
18 }
```

💡 **Remarque.** Comme `moy` est un entier, `moy / 2` est une division **entière** : `16/2 = 8`, `17/2 = 8`, `15/2 = 7` ... Chaque tranche de mention correspond ainsi à une ou plusieurs valeurs de `moy / 2`.

► Corrigé de l'exercice 5

```
C
1 #include <stdio.h>
2
3 int main(void) {
4     int n, i, somme = 0;
5     printf("Entre n : ");
6     scanf("%d", &n);
7
8     for (i = 1; i <= n; i++) {
9         somme = somme + i;
10    }
11    printf("1 + 2 + ... + %d = %d\n", n, somme);
12    return 0;
13 }
```

► Corrigé de l'exercice 6

```
C
1 #include <stdio.h>
2
3 int main(void) {
4     int m, i;
5     printf("Entre un entier : ");
6     scanf("%d", &m);
7
8     for (i = 1; i <= 10; i++) {
9         printf("%d x %d = %d\n", m, i, m * i);
10    }
11    return 0;
12 }
```

► Corrigé de l'exercice 7

```
C
1 #include <stdio.h>
2
3 int main(void) {
4     int n, chiffres = 0;
5     printf("Entre un entier positif : ");
6     scanf("%d", &n);
7
8     if (n == 0) {
9         chiffres = 1;          // le nombre 0 a un chiffre
10    } else {
11        while (n > 0) {
```

```

12         n = n / 10;           // on retire le dernier chiffre
13         chiffres++;
14     }
15 }
16 printf("Nombre de chiffres : %d\n", chiffres);
17 return 0;
18 }

```

► Corrigé de l'exercice 8

```

C
1  #include <stdio.h>
2
3  int main(void) {
4      int age;
5      do {
6          printf("Age de l'eleve (6 a 25) : ");
7          scanf("%d", &age);
8      } while (age < 6 || age > 25);
9
10     printf("Age valide : %d\n", age);
11     return 0;
12 }

```

► Corrigé de l'exercice 9

```

C
1  #include <stdio.h>
2
3  int main(void) {
4      int n;
5      printf("Entre n : ");
6      scanf("%d", &n);
7
8      while (n >= 1) {
9          printf("%d ", n);
10         n--;
11     }
12     printf("Termine !\n");
13     return 0;
14 }

```

► Corrigé de l'exercice 10

```

C
1  #include <stdio.h>
2
3  int main(void) {
4      int n, i = 2;
5      long fact = 1;
6      printf("Entre n : ");
7      scanf("%d", &n);
8
9      while (i <= n) {
10         fact = fact * i;

```

```

11     i++;
12 }
13 printf("%d! = %ld\n", n, fact);
14 return 0;
15 }

```

► Corrigé de l'exercice 11

On suppose le nombre premier, et on cherche un diviseur. Dès qu'on en trouve un, `break` arrête la recherche.

```

C
#include <stdio.h>

int main(void) {
    int n, i, premier = 1;    // 1 = vrai (on suppose premier)
    printf("Entre un entier (>= 2) : ");
    scanf("%d", &n);

    for (i = 2; i < n; i++) {
        if (n % i == 0) {      // i divise n
            premier = 0;      // donc n n'est pas premier
            break;             // inutile de continuer
        }
    }

    if (premier) {
        printf("%d est premier.\n", n);
    } else {
        printf("%d n'est pas premier.\n", n);
    }
    return 0;
}

```

► Corrigé de l'exercice 12

```

C
#include <stdio.h>

int main(void) {
    int n, i;
    long a = 0, b = 1, suivant;    // F0 = 0, F1 = 1
    printf("Combien de termes ? ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++) {
        printf("%ld ", a);          // on affiche le terme courant
        suivant = a + b;            // terme suivant
        a = b;
        b = suivant;
    }
    printf("\n");
    return 0;
}

```

Pour $n=7$, la boucle affiche : 0 1 1 2 3 5 8

► Corrigé de l'exercice 13

```

1  #include <stdio.h>
2
3  int main(void) {
4      int a, b, r;
5      printf("Deux entiers positifs : ");
6      scanf("%d %d", &a, &b);
7
8      while (b != 0) {
9          r = a % b;
10         a = b;
11         b = r;
12     }
13     printf("PGCD = %d\n", a);
14     return 0;
15 }

```

★ À retenir

On **décide** avec `if / else if / else` et `switch` (un `break` à chaque case !). En C, **0 = faux** et **tout le reste = vrai**; gare à `=` qui n'est pas `==`. On **répète** avec `for` (nombre de tours connu), `while` (condition d'arrêt) ou `do...while` (au moins un tour). `break` sort de la boucle, `continue` passe au tour suivant.

Les fonctions en C

Au programme

Découper un programme en **fonctions** : déclarer un prototype, écrire une définition, appeler la fonction et lui passer des **paramètres**. Comprendre le rôle de `return`, la différence entre variables **locales** et **globales**, et maîtriser la **récurtivité** (factorielle, puissance, Fibonacci). C'est l'outil qui rend un programme C lisible et réutilisable.

10.1 Pourquoi des fonctions ?

Jusqu'ici, tout notre code tenait dans la fonction `main`. Mais dès qu'un programme grandit, écrire tout d'un seul bloc devient illisible et on recopie sans cesse les mêmes calculs. La solution : isoler chaque traitement dans une **fonction** qu'on appelle au besoin.

Définition

Une **fonction** est un sous-programme nommé qui réalise une tâche précise. On l'écrit **une seule fois**, puis on l'**appelle** autant de fois que nécessaire. Elle peut recevoir des données (les **paramètres**) et renvoyer un résultat (la valeur de **retour**).

Trois bénéfices immédiats :

- **Réutiliser** : on écrit le calcul de la moyenne une fois, on s'en sert pour chaque élève du lycée.
- **Structurer** : un gros problème se découpe en petites fonctions simples, faciles à lire et à corriger.
- **Tester** : chaque fonction se vérifie séparément.

 **Remarque.** Tu connais déjà des fonctions : `printf` et `scanf` de la bibliothèque `<stdio.h>`, ou `sqrt` de `<math.h>`, sont des fonctions écrites par d'autres et que tu appelles. Tu vas maintenant écrire les tiennes.

10.2 Déclarer une fonction

10.2.1 La définition

Définir une fonction, c'est écrire son code complet. La structure est toujours la même.

```
type nom(parametres) { instructions return valeur; }
```

- `type` : le type de la valeur renvoyée (`int`, `float`, `double`, `char` ...ou `void` si la fonction ne renvoie rien) ;

- `nom` : le nom de la fonction (mêmes règles qu'une variable) ;
- `parametres` : la liste des données reçues, séparées par des virgules, chacune précédée de son type ;
- entre accolades : le **corps** de la fonction ;
- `return` : renvoie le résultat **et** termine la fonction.

```
C
1 // Fonction qui calcule le carre d'un entier
2 int carre(int x) {
3     int resultat = x * x;    // on calcule le carre
4     return resultat;        // on renvoie le resultat
5 }
```

Ici int est le type de retour, carre le nom, int x le paramètre.

10.2.2 Le rôle de return

Définition

L'instruction `return` fait deux choses : elle **renvoie** une valeur à l'endroit qui a appelé la fonction, et elle **arrête** immédiatement la fonction (les instructions situées après ne sont pas exécutées).

La valeur renvoyée par `return` doit être du même type que celui annoncé devant le nom de la fonction. Une fonction `int` renvoie un entier, une fonction `float` renvoie un décimal.

Exemple. Dans `int carre(int x)`, l'appel `carre(5)` vaut 25. On peut l'utiliser comme une valeur : `int s = carre(5) + carre(3);` place 34 dans `s`.

10.2.3 Les fonctions void (procédures)

Certaines fonctions n'ont pas de résultat à renvoyer : elles se contentent d'*agir* (afficher quelque chose, par exemple). Leur type de retour est alors `void` (« vide »), et elles n'utilisent pas `return valeur;`. On les appelle souvent des **procédures**.

```
C
1 // Procédure : elle affiche, mais ne renvoie rien
2 void afficherLigne(void) {
3     printf("-----\n");
4 }
```

Remarque. Le mot `void` entre les parenthèses indique que la fonction ne prend **aucun** paramètre. On peut aussi laisser les parenthèses vides : `void afficherLigne()`.

10.3 Appeler une fonction

Appeler une fonction, c'est l'exécuter en écrivant son nom suivi des valeurs à lui transmettre entre parenthèses (les **arguments**).

```
C
1 #include <stdio.h>
```

```

1 int carre(int x) {
2     return x * x;
3 }
4
5
6
7
8 int main(void) {
9     int n = 6;
10    int c = carre(n);           // appel : on passe 6, on recupere 36
11    printf("Le carre de %d est %d\n", n, c);
12    printf("Le carre de 4 est %d\n", carre(4)); // appel direct
13    return 0;
14 }

```

Affiche : « Le carre de 6 est 36 » puis « Le carre de 4 est 16 ».

10.3.1 Le passage des paramètres par valeur

Propriété Passage par valeur

En C, les paramètres sont passés **par valeur** : la fonction reçoit une **copie** de l'argument. Modifier le paramètre à l'intérieur de la fonction **ne change pas** la variable d'origine de l'appelant.

```

C
1 #include <stdio.h>
2
3 void doubler(int x) {
4     x = x * 2;           // on modifie la COPIE locale
5     printf("Dans la fonction : x = %d\n", x);
6 }
7
8 int main(void) {
9     int a = 10;
10    doubler(a);          // on passe une copie de a
11    printf("Dans main : a = %d\n", a); // a vaut toujours 10 !
12    return 0;
13 }

```

Affiche « Dans la fonction : x = 20 » puis « Dans main : a = 10 ».

⚠ Attention. La variable `a` de `main` n'est pas modifiée : la fonction travaille sur sa propre copie. Pour qu'une fonction modifie réellement une variable de l'appelant, il faut les **pointeurs** (chapitre dédié) ; ce n'est pas l'objet de ce chapitre.

10.3.2 Vocabulaire : paramètre ou argument ?

- le **paramètre** est la variable écrite dans la définition (`int x` dans `carre(int x)`) ;
- l'**argument** est la valeur réellement transmise lors de l'appel (`6` dans `carre(6)`).

10.4 Où placer les fonctions : le prototype

Le compilateur lit le fichier de haut en bas. Il doit **connaître** une fonction avant qu'on l'appelle. Deux solutions.

- 1) Écrire la définition complète **avant** `main`.

- 2) Écrire seulement le **prototype** avant `main`, et la définition complète après. C'est la méthode recommandée car `main` reste en haut.

Définition

Un **prototype** (ou *déclaration*) annonce au compilateur le nom de la fonction, son type de retour et le type de ses paramètres, suivi d'un point-virgule. Il ne contient pas le corps.

```
type nom(types des parametres);  ex.  int carre(int x);
```

C

```
1 #include <stdio.h>
2
3 int carre(int x);          // PROTOTYPE (avant main)
4
5 int main(void) {
6     printf("%d\n", carre(7)); // l'appel est valide grace au prototype
7     return 0;
8 }
9
10 int carre(int x) {         // DEFINITION (apres main)
11     return x * x;
12 }
```

💡 **Remarque.** Dans un prototype, on peut omettre le nom des paramètres : `int carre(int);` est correct. On garde souvent les noms pour la lisibilité.

10.5 Variables locales et globales : la portée

Définition

La **portée** (*scope*) d'une variable est la zone du programme où cette variable existe et peut être utilisée.

10.5.1 Variables locales

Une variable déclarée **à l'intérieur** d'une fonction est **locale** : elle n'existe que pendant l'exécution de cette fonction et reste invisible aux autres. Les paramètres sont également des variables locales.

C

```
1 #include <stdio.h>
2
3 void fonctionA(void) {
4     int compteur = 5;    // locale a fonctionA
5     printf("A : %d\n", compteur);
6 }
7
8 int main(void) {
9     int compteur = 100;  // une AUTRE variable locale a main
10    fonctionA();
11    printf("main : %d\n", compteur); // affiche 100
12 }
```



```
12     return 0;
13 }
```

Les deux compteur portent le même nom mais sont indépendants.

10.5.2 Variables globales

Une variable déclarée **en dehors** de toute fonction (souvent en haut du fichier) est **globale** : toutes les fonctions peuvent la lire et la modifier.

```
C
#include <stdio.h>

int total = 0;           // variable GLOBALE

void ajouter(int montant) {
    total = total + montant; // accede a la globale
}

int main(void) {
    ajouter(2000);          // en francs CFA
    ajouter(3500);
    printf("Total : %d FCFA\n", total); // affiche 5500
    return 0;
}
```

⚠ Attention. Les variables globales sont à utiliser **avec parcimonie** : comme n'importe quelle fonction peut les modifier, elles rendent les erreurs difficiles à retrouver. Préfère toujours des variables locales et le passage de paramètres.

10.6 Quelques fonctions utiles

10.6.1 Maximum de deux entiers

```
C
// Renvoie le plus grand des deux entiers a et b
int max(int a, int b) {
    if (a > b) {
        return a;           // return arrete la fonction ici
    }
    return b;               // atteint seulement si a <= b
}
```

10.6.2 Tester la parité

```
C
// Renvoie 1 si n est pair, 0 sinon
int estPair(int n) {
    return (n % 2 == 0);    // la condition vaut 1 (vrai) ou 0 (faux)
}
```

💡 **Remarque.** En C, une condition vraie vaut `1` et une condition fausse vaut `0`. On peut donc directement renvoyer le résultat d'un test, comme ci-dessus.

10.6.3 Moyenne de trois notes

```
C
1 #include <stdio.h>
2
3 // Moyenne de trois notes (valeurs decimales)
4 float moyenne(float a, float b, float c) {
5     return (a + b + c) / 3.0;
6 }
7
8 int main(void) {
9     float m = moyenne(12.5, 9.0, 15.5);
10    printf("Moyenne : %.2f / 20\n", m);    // affiche 12.33 / 20
11    return 0;
12 }
```

Le type de retour float permet de garder les décimales.

10.7 La récursivité

10.7.1 Le principe

Définition

Une fonction est dite **récursive** lorsqu'elle s'appelle **elle-même**. Pour qu'elle s'arrête, elle doit comporter deux parties :

- un **cas de base** : une situation simple où le résultat est connu directement, sans nouvel appel ;
- un **cas récursif** : la fonction se rappelle sur un problème **plus petit**, qui se rapproche du cas de base.

⚠ **Attention.** Sans cas de base, ou si les appels ne se rapprochent jamais du cas de base, la fonction s'appelle indéfiniment : c'est la **récursion infinie**. Le programme finit par planter (*stack overflow*, débordement de pile). Vérifie **toujours** qu'il existe un cas de base atteignable.

10.7.2 La factorielle

Rappel mathématique : $n! = n \times (n-1) \times \dots \times 2 \times 1$, avec $0! = 1$. On remarque que $n! = n \times (n-1)!$. Le cas de base est $0! = 1$.

```
C
1 // Factorielle de n, version recursive
2 int factorielle(int n) {
3     if (n == 0) {                // CAS DE BASE
4         return 1;
5     }
6     return n * factorielle(n - 1); // CAS RECURSIF
7 }
```

10.7.3 La pile d'appels

Pour calculer `factorielle(4)`, l'ordinateur empile les appels, puis les résout en remontant :

Exemple. $\text{factorielle}(4) = 4 \times \text{factorielle}(3)$
 $= 4 \times (3 \times \text{factorielle}(2))$
 $= 4 \times (3 \times (2 \times \text{factorielle}(1)))$
 $= 4 \times (3 \times (2 \times (1 \times \text{factorielle}(0))))$
 $= 4 \times 3 \times 2 \times 1 \times 1 = 24$

Chaque appel est mis en attente dans une zone mémoire appelée la **pile** (*stack*) tant que le suivant n'a pas renvoyé son résultat. Quand le cas de base `factorielle(0)` renvoie `1`, la pile se vide en sens inverse.

10.7.4 La puissance

x^n se calcule récursivement : $x^0 = 1$ (cas de base) et $x^n = x \times x^{n-1}$.

C

```
1 // Calcule x puissance n (n entier positif), version recursive
2 int puissance(int x, int n) {
3     if (n == 0) {                // CAS DE BASE : x^0 = 1
4         return 1;
5     }
6     return x * puissance(x, n - 1); // CAS RECURSIF
7 }
```

10.7.5 Somme de 1 à n

La fonction `somme(n)` calcule $1 + 2 + \dots + n$. Cas de base : `somme(0)` vaut `0` ; cas récursif : `somme(n) = n + somme(n-1)`.

C

```
1 // Somme des entiers de 1 a n, version recursive
2 int somme(int n) {
3     if (n == 0) {
4         return 0;                // CAS DE BASE
5     }
6     return n + somme(n - 1); // CAS RECURSIF
7 }
```

10.7.6 La suite de Fibonacci

La suite de Fibonacci est définie par $F(0) = 0$, $F(1) = 1$ et $F(n) = F(n-1) + F(n-2)$ pour $n \geq 2$. Elle a donc **deux** cas de base.

C

```
1 // n-ieme terme de la suite de Fibonacci, version recursive
2 int fibonacci(int n) {
3     if (n == 0) return 0;        // CAS DE BASE
4     if (n == 1) return 1;        // CAS DE BASE
5     return fibonacci(n - 1) + fibonacci(n - 2); // deux appels
6 }
```

💡 **Remarque.** La version récursive de Fibonacci est élégante mais **lente** pour les grands n : elle recalcule plusieurs fois les mêmes termes. Pour `fibonacci(40)`, elle fait des millions d'appels. La version itérative (une boucle) est alors bien plus efficace.

10.7.7 Itératif ou récursif ?

Tout calcul récursif peut s'écrire avec une boucle (**itératif**), et réciproquement. Comparons la factorielle.

```
C
// Factorielle, version ITERATIVE (avec une boucle)
int factorielleIter(int n) {
    int produit = 1;
    for (int i = 2; i <= n; i++) {
        produit = produit * i;
    }
    return produit;
}
```

Critère	Récursif	Itératif
Lisibilité	souvent plus proche des maths	plus terre à terre
Mémoire	consomme la pile (un cadre par appel)	quasi nulle
Vitesse	plus lent (appels)	en général plus rapide
Risque	récursion infinie / débordement	boucle infinie

Écrire une fonction récursive. (1) Identifier le **cas de base** (la plus petite valeur, où la réponse est immédiate). (2) Exprimer le résultat de `f(n)` en fonction de `f(n-1)` (ou d'un cas plus petit). (3) Vérifier que chaque appel **rapproche** du cas de base.

10.8 Exercices

► Application

Exercice 1 Minimum de deux entiers

★ Écris une fonction `int min(int a, int b)` qui renvoie le plus petit des deux entiers. Teste-la dans `main` avec `min(8, 3)`.

Exercice 2 Valeur absolue

★ Écris une fonction `int valeurAbsolue(int n)` qui renvoie la valeur absolue de `n` (par exemple `-7` donne `7`).

Exercice 3 Mention au baccalauréat

★★ Écris une fonction `void afficherMention(float moyenne)` qui affiche la mention obtenue : *Pas-sable* si la moyenne est entre 10 et 12, *Assez bien* entre 12 et 14, *Bien* entre 14 et 16, *Très bien* à partir de 16, et *Recalé* en dessous de 10 (notes sur 20).

Exercice 4 Factorielle dans les deux styles

★★ Écris la fonction `factorielle` en version **récursive**, puis en version **itérative**. Appelle les deux dans `main` pour vérifier qu'elles donnent le même résultat pour `5`.

► Entraînement

Exercice 5 Nombre premier

★★ Écris une fonction `int estPremier(int n)` qui renvoie `1` si `n` est un nombre premier, `0` sinon. Rappel : un nombre premier n'est divisible que par `1` et par lui-même, et est strictement supérieur à `1`.

Exercice 6 PGCD

★★★ Écris une fonction `int pgcd(int a, int b)` qui calcule le plus grand commun diviseur de `a` et `b` par l'algorithme d'Euclide (soustractions successives, ou `pgcd(a, b) = pgcd(b, a % b)` avec `pgcd(a, 0) = a`).

Exercice 7 Somme des chiffres

★★★ Écris une fonction **récursive** `int sommeChiffres(int n)` qui renvoie la somme des chiffres de `n`. Par exemple `sommeChiffres(237)` vaut `2 + 3 + 7 = 12`. Indice : le dernier chiffre est `n % 10`, et le reste du nombre est `n / 10`.

Exercice 8 Puissance

★★ Écris une fonction `puissance(int x, int n)` en version récursive, puis appelle `puissance(2, 10)` dans `main` (le résultat doit valoir `1024`).

10.9 Corrigés

► Corrigé de l'exercice 1

```
C
1 #include <stdio.h>
2
3 int min(int a, int b) {
4     if (a < b) {
5         return a;
6     }
7     return b;
8 }
9
10 int main(void) {
11     printf("Le minimum est %d\n", min(8, 3)); // affiche 3
12     return 0;
13 }
```

► Corrigé de l'exercice 2

```
C
1 #include <stdio.h>
2
3 int valeurAbsolue(int n) {
4     if (n < 0) {
5         return -n; // -(-7) = 7
6     }
7     return n;
8 }
9
10 int main(void) {
11     printf("%d\n", valeurAbsolue(-7)); // affiche 7
12 }
```

```
12     printf("%d\n", valeurAbsolue(4));    // affiche 4
13     return 0;
14 }
```

► Corrigé de l'exercice 3

```
C
1  #include <stdio.h>
2
3  void afficherMention(float moyenne) {
4      if (moyenne < 10) {
5          printf("Recalé\n");
6      } else if (moyenne < 12) {
7          printf("Passable\n");
8      } else if (moyenne < 14) {
9          printf("Assez bien\n");
10     } else if (moyenne < 16) {
11         printf("Bien\n");
12     } else {
13         printf("Tres bien\n");
14     }
15 }
16
17 int main(void) {
18     afficherMention(13.5);    // affiche Assez bien
19     afficherMention(16.2);    // affiche Tres bien
20     return 0;
21 }
```

► Corrigé de l'exercice 4

```
C
1  #include <stdio.h>
2
3  // Version recursive
4  int factorielle(int n) {
5      if (n == 0) {
6          return 1;
7      }
8      return n * factorielle(n - 1);
9  }
10
11 // Version iterative
12 int factorielleIter(int n) {
13     int produit = 1;
14     for (int i = 2; i <= n; i++) {
15         produit = produit * i;
16     }
17     return produit;
18 }
19
20 int main(void) {
21     printf("Recursive : %d\n", factorielle(5));    // 120
22     printf("Iterative : %d\n", factorielleIter(5)); // 120
23     return 0;
24 }
```

► Corrigé de l'exercice 5

```
C
1 #include <stdio.h>
2
3 int estPremier(int n) {
4     if (n < 2) {
5         return 0;           // 0 et 1 ne sont pas premiers
6     }
7     for (int d = 2; d < n; d++) {
8         if (n % d == 0) {    // n est divisible par d
9             return 0;       // donc pas premier
10        }
11    }
12    return 1;               // aucun diviseur trouve : premier
13 }
14
15 int main(void) {
16     printf("%d\n", estPremier(7));    // 1 (premier)
17     printf("%d\n", estPremier(12));   // 0 (non premier)
18     return 0;
19 }
```

► Corrigé de l'exercice 6

```
C
1 #include <stdio.h>
2
3 // Algorithme d'Euclide, version recursive
4 int pgcd(int a, int b) {
5     if (b == 0) {
6         return a;           // CAS DE BASE
7     }
8     return pgcd(b, a % b);  // CAS RECURSIF
9 }
10
11 int main(void) {
12     printf("PGCD(48, 36) = %d\n", pgcd(48, 36));    // 12
13     return 0;
14 }
```

► Corrigé de l'exercice 7

```
C
1 #include <stdio.h>
2
3 // Somme des chiffres, version recursive
4 int sommeChiffres(int n) {
5     if (n == 0) {
6         return 0;           // CAS DE BASE
7     }
8     return (n % 10) + sommeChiffres(n / 10);    // dernier chiffre + le reste
9 }
10
11 int main(void) {
12     printf("%d\n", sommeChiffres(237));    // 12
13 }
```

```
13     return 0;
14 }
```

► Corrigé de l'exercice 8

```
C
#include <stdio.h>

int puissance(int x, int n) {
    if (n == 0) {
        return 1;
    }
    return x * puissance(x, n - 1);
}

int main(void) {
    printf("2^10 = %d\n", puissance(2, 10)); // 1024
    return 0;
}
```

★ À retenir

Une fonction se **déclare** (`type nom(params){ ...return v; }`), s'**appelle** par son nom et reçoit ses arguments **par valeur** (une copie). Place un **prototype** avant `main`. Une fonction **ré-cursive** s'appelle elle-même : elle a **toujours** un **cas de base** pour s'arrêter, sinon c'est la récursion infinie.

Tableaux, tri et enregistrements en C

Au programme

Ranger plusieurs valeurs sous un même nom grâce aux **tableaux**, les parcourir avec une boucle, calculer somme, moyenne, minimum et maximum, rechercher une valeur, puis **trier par sélection**. On découvre aussi les **matrices** (tableaux à deux dimensions) et les **enregistrements** (**struct**) pour décrire une fiche élève. C'est le cœur de la manipulation de données en langage C.

11.1 Les tableaux à une dimension

Jusqu'ici, chaque valeur tenait dans une variable. Mais comment stocker les **60 notes** d'une classe de Terminale TI ? Déclarer `n1`, `n2`, ..., `n60` serait absurde. Le **tableau** résout ce problème : un seul nom regroupe plusieurs cases de **même type**, numérotées par un **indice**.

Définition

Un **tableau** (*array*) est une suite de cases mémoire contiguës, de même type, désignées par un nom unique. On accède à chaque case par son **indice** (sa position), qui commence à **0** en langage C.

11.1.1 Déclaration

```
type nom[taille]; — par exemple int t[10];
```

La déclaration `int t[10];` réserve **10 cases** d'entiers, d'indices **0** à **9**. La **taille** doit être une constante connue à la compilation.

C

```
1 int t[10];           // 10 entiers : t[0], t[1], ..., t[9]
2 float notes[60];    // 60 notes décimales
3 char nom[30];       // 30 caractères (une chaîne)
```

⚠ Attention. Un tableau de taille `10` possède les indices **0** à **9**. La case `t[10]` **n'existe pas** : y accéder est une erreur classique (*débordement*) qui plante le programme ou corrompt la mémoire.

11.1.2 Initialisation et accès

On peut donner les valeurs dès la déclaration, entre accolades :

```
C
1 int t[5] = {12, 7, 25, 3, 18}; // les 5 cases sont remplies
2 int u[5] = {0};               // toutes les cases à 0
3 int v[] = {4, 8, 15};         // taille déduite : 3 cases
```

On lit ou modifie une case avec `nom[indice]` :

```
C
1 t[0] = 20;                    // on change la 1re case
2 int x = t[2];                 // x reçoit la valeur de la 3e case (25)
3 t[4] = t[4] + 1;              // on incrémente la dernière case
```

💡 Remarque. L'indice peut être une variable : `t[i]`. C'est ce qui permet de parcourir tout le tableau avec une boucle `for` en faisant varier `i`.

11.1.3 Saisie et affichage par une boucle

Pour remplir ou afficher un tableau, on utilise une boucle `for` dont le compteur sert d'indice.

```
C
1 // Programme en langage C
2 #include <stdio.h>
3
4 int main() {
5     int notes[5];
6     int i;
7
8     // SAISIE des 5 notes
9     for (i = 0; i < 5; i++) {
10         printf("Note numero %d : ", i + 1);
11         scanf("%d", &notes[i]); // &notes[i] : adresse de la case
12     }
13
14     // AFFICHAGE des 5 notes
15     printf("Vous avez saisi : ");
16     for (i = 0; i < 5; i++) {
17         printf("%d ", notes[i]);
18     }
19     printf("\n");
20
21     return 0;
22 }
```

La boucle parcourt les indices 0 à 4 : un seul code pour toutes les cases.

⚠ Attention. Dans `scanf`, on met le `&` devant la case : `scanf("%d", ¬es[i])`. Il indique l'adresse où ranger la valeur lue.

11.1.4 Passer un tableau à une fonction

Quand on transmet un tableau à une fonction, le C passe en réalité l'**adresse** de sa première case. La fonction travaille donc **directement** sur le tableau original : toute modification est conservée. On doit aussi passer la **taille**, car la fonction ne la connaît pas.

```
C
1 // Programme en langage C
2 #include <stdio.h>
3
4 // Affiche les n premières cases du tableau t
5 void afficher(int t[], int n) {
6     int i;
7     for (i = 0; i < n; i++) {
8         printf("%d ", t[i]);
9     }
10    printf("\n");
11 }
12
13 int main() {
14     int tab[4] = {10, 20, 30, 40};
15     afficher(tab, 4);    // affiche : 10 20 30 40
16     return 0;
17 }
```

Remarque. Dans l'en-tête `void afficher(int t[], int n)`, les crochets `[]` restent **vides** : la fonction accepte un tableau de n'importe quelle longueur, et `n` précise combien de cases utiliser.

11.2 Algorithmes classiques sur un tableau

Une poignée de schémas reviennent sans cesse. Apprends-les par cœur : tous reposent sur une boucle qui parcourt le tableau.

11.2.1 Somme et moyenne

On utilise un **accumulateur** initialisé à `0`, auquel on ajoute chaque case.

```
C
1 // Programme en langage C
2 #include <stdio.h>
3
4 int main() {
5     float notes[5] = {12.0, 15.5, 8.0, 14.0, 11.5};
6     int i;
7     float somme = 0.0;
8
9     for (i = 0; i < 5; i++) {
10        somme = somme + notes[i];    // on accumule
11    }
12    float moyenne = somme / 5;    // division par le nombre de notes
13
14    printf("Somme   : %.2f\n", somme);
15    printf("Moyenne : %.2f\n", moyenne);
16    return 0;
17 }
```

Sortie : Somme : 61.00 Moyenne : 12.20

11.2.2 Minimum et maximum

On suppose d'abord que la **première case** est le maximum, puis on la compare à toutes les autres et on garde la plus grande.

Chercher le maximum. (1) `max` reçoit `t[0]` ; (2) pour chaque case suivante, si `t[i] > max` alors `max` reçoit `t[i]` ; (3) à la fin, `max` contient la plus grande valeur. (Pour le minimum, on remplace `>` par `<.`)

C

```

1 // Programme en langage C
2 #include <stdio.h>
3
4 int main() {
5     int t[6] = {14, 9, 23, 7, 31, 18};
6     int i;
7     int max = t[0];    // hypothese : la 1re case est le max
8
9     for (i = 1; i < 6; i++) {        // on commence a l'indice 1
10         if (t[i] > max) {
11             max = t[i];
12         }
13     }
14     printf("Le maximum est %d\n", max);    // affiche 31
15     return 0;
16 }
```

⚠ Attention. On initialise `max` avec `t[0]`, **jamais** avec `0` : si toutes les valeurs étaient négatives, partir de `0` donnerait un résultat faux.

11.2.3 Recherche séquentielle d'une valeur

Pour savoir si une valeur `cherchee` se trouve dans le tableau, on parcourt les cases une à une : c'est la **recherche séquentielle** (ou linéaire).

C

```

1 // Programme en langage C
2 #include <stdio.h>
3
4 int main() {
5     int t[6] = {14, 9, 23, 7, 31, 18};
6     int cherchee = 23;
7     int i;
8     int trouve = 0;    // 0 = pas encore trouve (faux)
9     int position = -1;
10
11     for (i = 0; i < 6; i++) {
12         if (t[i] == cherchee) {
13             trouve = 1;    // on l'a trouvee (vrai)
14             position = i;    // on retient sa position
15         }
16     }
17 }
```

```

17
18     if (trouve == 1) {
19         printf("Valeur trouvee a l'indice %d\n", position);
20     } else {
21         printf("Valeur absente du tableau\n");
22     }
23     return 0;
24 }

```

Le drapeau `trouve` (0/1) mémorise le résultat de la recherche.

11.3 Le tri par sélection

Tri un tableau, c'est ranger ses valeurs dans l'ordre croissant (ou décroissant). Le **tri par sélection** est la méthode la plus simple à comprendre et figure au programme de Terminale TI.

11.3.1 Le principe

Définition

Le **tri par sélection** consiste à : chercher le plus petit élément du tableau et le placer en première position ; puis chercher le plus petit parmi **ce qui reste** et le placer en deuxième position ; et ainsi de suite jusqu'à la fin.

À chaque tour, on **sélectionne** le minimum de la partie non triée et on l'**échange** avec la première case de cette partie.

Tri par sélection (pseudo-algorithme).

- 1) Pour `i` allant de 0 à `n-2` :
- 2) poser `iMin = i` (position du plus petit présumé) ;
- 3) pour `j` allant de `i+1` à `n-1` : si `t[j] < t[iMin]` alors `iMin = j` ;
- 4) échanger `t[i]` et `t[iMin]` .

11.3.2 Trace sur un exemple

Trions le tableau `[5, 3, 8, 1]` ($n = 4$). Chaque ligne montre l'état après l'échange du tour i ; la partie **triée** est à gauche du trait.

Tour	Plus petit trouvé	Échange	Tableau
Départ	—	—	5 3 8 1
$i = 0$	1 (indice 3)	<code>t[0] ↔ t[3]</code>	1 3 8 5
$i = 1$	3 (indice 1)	aucun (déjà en place)	1 3 8 5
$i = 2$	5 (indice 3)	<code>t[2] ↔ t[3]</code>	1 3 5 8
Fin	—	—	1 3 5 8

En 3 tours ($n-1$), le tableau est trié dans l'ordre croissant.

11.3.3 Code C complet

```

C
// Programme en langage C
#include <stdio.h>

int main() {
    int t[6] = {5, 3, 8, 1, 9, 2};
    int n = 6;
    int i, j, iMin, temp;

    // TRI PAR SELECTION
    for (i = 0; i < n - 1; i++) {
        iMin = i;                                // plus petit presume : la case i
        for (j = i + 1; j < n; j++) {
            if (t[j] < t[iMin]) {
                iMin = j;                        // on a trouve plus petit
            }
        }
        // echange de t[i] et t[iMin] via une variable temporaire
        temp = t[i];
        t[i] = t[iMin];
        t[iMin] = temp;
    }

    // affichage du tableau trie
    printf("Tableau trie : ");
    for (i = 0; i < n; i++) {
        printf("%d ", t[i]);
    }
    printf("\n");
    return 0;
}

```

Sortie : Tableau trie : 1 2 3 5 8 9

⚠ Attention. L'échange de deux cases passe **obligatoirement** par une variable temporaire (temp). Écrire directement `t[i] = t[iMin];` puis `t[iMin] = t[i];` écrase la première valeur et perd une donnée.

11.4 Les tableaux à deux dimensions (matrices)

Un tableau à **deux dimensions** se représente comme un **tableau de lignes et de colonnes**, exactement comme un tableau de notes par élève et par matière.

`type nom[lignes][colonnes];` — par exemple `int m[3][4];`

`int m[3][4];` déclare une matrice de **3 lignes** et **4 colonnes**, soit 12 cases. On accède à une case par `m[i][j]` (ligne `i`, colonne `j`).

```

C
// Programme en langage C
#include <stdio.h>

```

```

1 int main() {
2     int m[2][3] = { {1, 2, 3},
3                     {4, 5, 6} }; // 2 lignes, 3 colonnes
4
5     int i, j;
6
7     // PARCOURS DOUBLE : une boucle par dimension
8     for (i = 0; i < 2; i++) { // pour chaque ligne
9         for (j = 0; j < 3; j++) { // pour chaque colonne
10            printf("%d\t", m[i][j]);
11        }
12        printf("\n"); // retour a la ligne apres chaque ligne
13    }
14    return 0;
15 }
```

Le parcours d'une matrice utilise deux boucles imbriquées (lignes, colonnes).

💡 Remarque. La boucle **extérieure** parcourt les lignes (*i*), la boucle **intérieure** parcourt les colonnes (*j*). On affiche un retour à la ligne (*n*) après chaque ligne pour obtenir un beau tableau.

11.5 Les enregistrements (struct)

Un tableau regroupe des valeurs de **même type**. Mais une fiche élève réunit un **nom** (texte), un **âge** (entier) et une **moyenne** (décimal) : des types différents. L'**enregistrement** (`struct`) permet de les rassembler sous un seul nom.

Définition

Un **enregistrement** (mot-clé `struct`) est un type composé qui regroupe plusieurs variables, appelées **champs**, pouvant être de types différents.

11.5.1 Définir une structure

```

1 struct Eleve {
2     char nom[30]; // champ texte
3     int age; // champ entier
4     float moyenne; // champ decimal
5 }; // ne pas oublier le point-virgule final
```

⚠ Attention. La définition d'une `struct` se termine par un **point-virgule** après l'accolade fermante. C'est une erreur de débutant très fréquente de l'oublier.

11.5.2 Déclarer une variable et accéder aux champs

On accède à un champ avec le **point** (`.`) : `variable.champ` .

```

1 // Programme en langage C
2 #include <stdio.h>
3 #include <string.h>
```

```

4
5 struct Eleve {
6     char nom[30];
7     int age;
8     float moyenne;
9 };
10
11 int main() {
12     struct Eleve e1;           // une variable de type struct Eleve
13
14     strcpy(e1.nom, "Awa");     // copie un texte dans un champ char[]
15     e1.age = 17;               // affectation directe pour les autres
16     e1.moyenne = 14.5;
17
18     printf("Nom      : %s\n", e1.nom);
19     printf("Age      : %d ans\n", e1.age);
20     printf("Moyenne  : %.2f\n", e1.moyenne);
21     return 0;
22 }

```

 **Remarque.** Un champ `char nom[30]` est un tableau de caractères : on ne peut pas écrire `e1.nom = "Awa";`. On copie le texte avec `strcpy` (de la bibliothèque `<string.h>`).

11.5.3 Un tableau de structures

On peut créer un **tableau de struct** pour gérer toute une classe : chaque case est une fiche complète.

```

C
1 // Programme en langage C
2 #include <stdio.h>
3 #include <string.h>
4
5 struct Eleve {
6     char nom[30];
7     int age;
8     float moyenne;
9 };
10
11 int main() {
12     struct Eleve classe[2];    // tableau de 2 eleves
13
14     strcpy(classe[0].nom, "Awa");
15     classe[0].age = 17;
16     classe[0].moyenne = 14.5;
17
18     strcpy(classe[1].nom, "Bello");
19     classe[1].age = 18;
20     classe[1].moyenne = 11.0;
21
22     int i;
23     for (i = 0; i < 2; i++) {
24         printf("%s (%d ans) : %.2f\n",
25             classe[i].nom, classe[i].age, classe[i].moyenne);
26     }
27     return 0;

```



```
28 }
```

Chaque case `classe[i]` est un enregistrement complet (nom, âge, moyenne).

11.6 Exemple complet : la fiche élève

Réunissons tout : saisir une classe, l'afficher, puis trouver le **meilleur élève** (plus forte moyenne).

```
C
// Programme en langage C
#include <stdio.h>

struct Eleve {
    char nom[30];
    int age;
    float moyenne;
};

int main() {
    struct Eleve classe[3];
    int n = 3;
    int i;

    // SAISIE
    for (i = 0; i < n; i++) {
        printf("Eleve %d - nom : ", i + 1);
        scanf("%s", classe[i].nom);    // pas de & : nom est déjà un tableau
        printf("Eleve %d - age : ", i + 1);
        scanf("%d", &classe[i].age);
        printf("Eleve %d - moyenne : ", i + 1);
        scanf("%f", &classe[i].moyenne);
    }

    // AFFICHAGE
    printf("\n--- Liste de la classe ---\n");
    for (i = 0; i < n; i++) {
        printf("%s : %.2f\n", classe[i].nom, classe[i].moyenne);
    }

    // MEILLEUR ELEVE (plus forte moyenne)
    int iMax = 0;
    for (i = 1; i < n; i++) {
        if (classe[i].moyenne > classe[iMax].moyenne) {
            iMax = i;
        }
    }
    printf("\nMajor de la classe : %s (%.2f)\n",
           classe[iMax].nom, classe[iMax].moyenne);
    return 0;
}
```

On retrouve le schéma du maximum, appliqué au champ `moyenne`.

💡 **Remarque.** Pour lire un nom avec `scanf("%s", classe[i].nom)`, on ne met **pas** de `&` : `classe[i].nom` est un tableau, c'est déjà une adresse. En revanche, `age` et `moyenne` en ré-

clament un.

11.7 Exercices

► Application

Exercice 1 Somme et moyenne

★ Un tableau `int notes[5] = {12, 8, 15, 10, 14};` contient 5 notes. Écris un programme qui calcule et affiche leur somme et leur moyenne.

Exercice 2 Minimum et maximum

★ À partir du tableau `int t[6] = {17, 4, 23, 9, 1, 30};`, affiche la plus petite et la plus grande valeur.

Exercice 3 Compter les admis

★★ Un tableau `float moy[8]` contient les moyennes de 8 élèves d'un lycée de Bafoussam. Écris un programme qui compte et affiche le nombre d'élèves **admis** (moyenne supérieure ou égale à 10).

Exercice 4 Recherche d'une valeur

★★ On donne un tableau `int t[7] = {5, 12, 8, 20, 3, 17, 9};` et un entier `cherchee = 20`. Indique si cette valeur est présente, et à quelle position.

► Entraînement

Exercice 5 Inverser un tableau

★★ Écris un programme qui inverse l'ordre des éléments d'un tableau `t` de 6 entiers (par exemple les valeurs 1 à 6) : la première case échange avec la dernière, la deuxième avec l'avant-dernière, etc. Affiche ensuite le résultat.

Exercice 6 Tri par sélection

★★★ Saisis 5 entiers dans un tableau, trie-le par **sélection** dans l'ordre croissant, puis affiche le tableau trié.

Exercice 7 Gestion de produits

★★★ Une boutique de Douala gère ses articles avec une structure `Produit` possédant les champs `nom` (texte), `quantite` (entier) et `prix` (décimal). Saisis 3 produits, affiche-les, puis indique le produit le **plus cher** (en francs CFA).

Exercice 8 Afficher une matrice

★★ Déclare et initialise une matrice `int m[3][3]` avec des valeurs de ton choix, puis affiche-la sous forme de tableau (lignes et colonnes alignées).

11.8 Corrigés

► Corrigé de l'exercice 1

```
C
// Programme en langage C
#include <stdio.h>

int main() {
    int notes[5] = {12, 8, 15, 10, 14};
    int i, somme = 0;
```

```

7
8     for (i = 0; i < 5; i++) {
9         somme = somme + notes[i];
10    }
11    float moyenne = (float) somme / 5;    // (float) pour un resultat decimal
12
13    printf("Somme   : %d\n", somme);
14    printf("Moyenne : %.2f\n", moyenne);
15    return 0;
16 }

```

La somme vaut 59 et la moyenne 11.80 . Le (float) force la division décimale (sinon 59 / 5 donnerait 11 en entier).

► Corrigé de l'exercice 2

```

C
1 // Programme en langage C
2 #include <stdio.h>
3
4 int main() {
5     int t[6] = {17, 4, 23, 9, 1, 30};
6     int i;
7     int min = t[0], max = t[0];
8
9     for (i = 1; i < 6; i++) {
10        if (t[i] < min) min = t[i];
11        if (t[i] > max) max = t[i];
12    }
13    printf("Minimum : %d\n", min);    // 1
14    printf("Maximum : %d\n", max);    // 30
15    return 0;
16 }

```

Une seule boucle suffit pour trouver le minimum et le maximum, en initialisant les deux avec t[0] .

► Corrigé de l'exercice 3

```

C
1 // Programme en langage C
2 #include <stdio.h>
3
4 int main() {
5     float moy[8] = {12.0, 8.5, 15.0, 9.0, 10.0, 7.5, 13.5, 11.0};
6     int i, admis = 0;
7
8     for (i = 0; i < 8; i++) {
9         if (moy[i] >= 10.0) {
10            admis = admis + 1;    // on compte
11        }
12    }
13    printf("Nombre d'admis : %d sur 8\n", admis);    // 5
14    return 0;
15 }

```

On utilise un **compteur** (admis) incrémenté chaque fois que la condition moy[i] >= 10.0 est vraie.

► Corrigé de l'exercice 4

```

1 // Programme en langage C
2 #include <stdio.h>
3
4 int main() {
5     int t[7] = {5, 12, 8, 20, 3, 17, 9};
6     int cherchee = 20;
7     int i, position = -1;
8
9     for (i = 0; i < 7; i++) {
10         if (t[i] == cherchee) {
11             position = i;        // on memorise l'indice
12         }
13     }
14
15     if (position != -1) {
16         printf("Valeur %d trouvee a l'indice %d\n", cherchee, position);
17     } else {
18         printf("Valeur %d absente\n", cherchee);
19     }
20     return 0;
21 }

```

Ici 20 est à l'indice 3. La variable `position` vaut -1 tant que rien n'est trouvé, ce qui sert de signal d'absence.

► Corrigé de l'exercice 5

```

1 // Programme en langage C
2 #include <stdio.h>
3
4 int main() {
5     int t[6] = {1, 2, 3, 4, 5, 6};
6     int n = 6, i, temp;
7
8     // on echange t[i] avec t[n-1-i], jusqu'au milieu
9     for (i = 0; i < n / 2; i++) {
10         temp = t[i];
11         t[i] = t[n - 1 - i];
12         t[n - 1 - i] = temp;
13     }
14
15     for (i = 0; i < n; i++) {
16         printf("%d ", t[i]);    // affiche : 6 5 4 3 2 1
17     }
18     printf("\n");
19     return 0;
20 }

```

On ne parcourt que la **moitié** du tableau (`i < n/2`) : chaque tour échange une case du début avec la case symétrique de la fin.

► Corrigé de l'exercice 6

C

```

1 // Programme en langage C
2 #include <stdio.h>
3
4 int main() {
5     int t[5];
6     int i, j, iMin, temp;
7
8     for (i = 0; i < 5; i++) {
9         printf("Entier %d : ", i + 1);
10        scanf("%d", &t[i]);
11    }
12
13    // tri par selection
14    for (i = 0; i < 4; i++) {                // jusqu'a n-2 (soit 3)
15        iMin = i;
16        for (j = i + 1; j < 5; j++) {
17            if (t[j] < t[iMin]) iMin = j;
18        }
19        temp = t[i];
20        t[i] = t[iMin];
21        t[iMin] = temp;
22    }
23
24    printf("Tableau trie : ");
25    for (i = 0; i < 5; i++) printf("%d ", t[i]);
26    printf("\n");
27    return 0;
28 }

```

► Corrigé de l'exercice 7

C

```

1 // Programme en langage C
2 #include <stdio.h>
3
4 struct Produit {
5     char nom[30];
6     int quantite;
7     float prix;
8 };
9
10 int main() {
11     struct Produit stock[3];
12     int i, iCher = 0;
13
14     for (i = 0; i < 3; i++) {
15         printf("Produit %d - nom : ", i + 1);
16         scanf("%s", stock[i].nom);
17         printf("Produit %d - quantite : ", i + 1);
18         scanf("%d", &stock[i].quantite);
19         printf("Produit %d - prix (FCFA) : ", i + 1);
20         scanf("%f", &stock[i].prix);
21     }
22
23     printf("\n--- Inventaire ---\n");

```

```

24     for (i = 0; i < 3; i++) {
25         printf("%s : %d unites a %.0f FCFA\n",
26             stock[i].nom, stock[i].quantite, stock[i].prix);
27     }
28
29     // produit le plus cher
30     for (i = 1; i < 3; i++) {
31         if (stock[i].prix > stock[iCher].prix) iCher = i;
32     }
33     printf("\nProduit le plus cher : %s (%.0f FCFA)\n",
34         stock[iCher].nom, stock[iCher].prix);
35     return 0;
36 }

```

► Corrigé de l'exercice 8

```

C
// Programme en langage C
#include <stdio.h>

int main() {
    int m[3][3] = { {1, 2, 3},
                    {4, 5, 6},
                    {7, 8, 9} };

    int i, j;

    for (i = 0; i < 3; i++) {
        for (j = 0; j < 3; j++) {
            printf("%d\t", m[i][j]);
        }
        printf("\n");
    }
    return 0;
}

```

Les deux boucles imbriquées parcourent lignes (`i`) puis colonnes (`j`); le `\n` après la boucle interne sépare les lignes.

★ À retenir

Un **tableau** regroupe des valeurs de même type, indexées à partir de **0**, qu'on parcourt avec une boucle `for`. Les schémas **somme/moyenne**, **min/max** et **recherche** sont à connaître par cœur. Le **tri par sélection** place à chaque tour le plus petit élément restant, via un **échange** avec variable temporaire. Une `struct` rassemble des champs de types différents (fiche élève); on y accède avec le point (`.`) et on copie un nom avec `strcpy`.

Projet guidé : la calculatrice (et plus)

Au programme

Mener un projet de programmation du début à la fin : du **cahier des charges** au programme testé. On produit une **calculatrice** complète en C (menu, quatre opérations, gestion de la division par zéro), puis un mini-logiciel de **gestion des notes d'une classe** (tableau de structures). C'est la synthèse de toute l'année : conditions, boucles, fonctions, tableaux et structures réunis.

12.1 La démarche de projet

Jusqu'ici, tu as appris des « briques » : variables, `if`, boucles, fonctions, tableaux, structures. Un **projet** consiste à assembler ces briques pour construire un logiciel qui rend un service. On ne se jette pas sur le clavier : on suit une **démarche**.

Définition

La **démarche de projet** est l'enchaînement organisé des étapes qui mènent d'un besoin exprimé à un programme fonctionnel et fiable. Les cinq étapes sont : **cahier des charges**, **analyse**, **algorithme**, **codage**, **test et débogage**.

12.1.1 Les cinq étapes

- 1) **Cahier des charges** — On écrit, en français, *ce que le programme doit faire* : les entrées, les traitements, les sorties, les cas particuliers. C'est le contrat. Exemple : « le programme doit additionner deux nombres saisis par l'utilisateur et afficher le résultat ».
- 2) **Analyse** — On découpe le problème en sous-problèmes plus simples (méthode *descendante*). Chaque sous-problème deviendra souvent une **fonction**.
- 3) **Algorithme** — On écrit la solution en pseudo-code ou par un organigramme, sans se soucier du langage. On vérifie la logique « sur le papier ».
- 4) **Codage** — On traduit l'algorithme dans le langage choisi (ici, le **C**), en respectant la syntaxe.
- 5) **Test et débogage** — On exécute le programme avec des jeux d'essai (valeurs normales *et* cas limites), on repère les erreurs (*bugs*) et on les corrige.

Réussir un projet de programmation. (1) Lis le sujet et **reformule** le besoin avec tes mots ; (2) liste les **entrées**, le **traitement** et les **sorties** ; (3) découpe en **fonctions** (une fonction = une tâche) ; (4) code et compile **morceau par morceau** ; (5) teste chaque cas, surtout les **cas limites** (zéro, valeur négative, saisie vide).

Remarque. Un bon réflexe : compiler souvent. Un programme qui grossit de 200 lignes avant la première compilation est très difficile à déboguer. Mieux vaut écrire 10 lignes, compiler, vérifier, puis continuer.

12.1.2 Repérer entrées, traitement, sorties

Avant tout projet, on remplit ce petit tableau. Pour notre calculatrice :

Entrées	Traitement	Sorties
Le choix de l'opération	Choisir la bonne opération	Le résultat du calcul
Le premier nombre	Calculer	Un message d'erreur si
Le second nombre	(gérer la division par 0)	division par zéro

12.2 Projet 1 — La calculatrice en C

12.2.1 Cahier des charges

Définition

Le **cahier des charges** décrit précisément le comportement attendu du logiciel. C'est le document de référence : si le programme le respecte, le travail est réussi.

Cahier des charges de la calculatrice :

- 1) Le programme affiche un **menu** : 1) Addition, 2) Soustraction, 3) Multiplication, 4) Division, 5) Quitter.
- 2) L'utilisateur **choisit** une opération en tapant son numéro.
- 3) Pour une opération de calcul, le programme demande **deux nombres**.
- 4) Il **affiche le résultat** de l'opération.
- 5) Pour la division, si le second nombre est **zéro**, il affiche un message d'erreur et ne calcule pas.
- 6) Après chaque opération, le programme **recommence** (réaffiche le menu) tant que l'utilisateur n'a pas choisi « Quitter ».

12.2.2 Conception (analyse + algorithme)

On découpe le problème. Le menu doit se répéter : c'est une **boucle**. Comme le menu doit s'afficher *au moins une fois*, on choisit une boucle `do...while`. À l'intérieur, selon le choix, on exécute une opération différente : c'est un `switch`.

Chaque opération devient une **fonction** : `addition`, `soustraction`, `multiplication`, `division`. Une fonction qui prend deux nombres et renvoie un résultat est un bon découpage.

Structure générale (pseudo-code).

- 1) Répéter :
- 2) afficher le menu ;
- 3) lire `choix` ;
- 4) selon `choix` : 1 → addition, 2 → soustraction, 3 → multiplication, 4 → division, 5 → quitter ;
- 5) Tant que `choix` $\neq$ 5.

⚠ Attention. La **division par zéro** est interdite en mathématiques *et* en informatique : diviser par 0 provoque un comportement indéfini (souvent un plantage du programme). Il faut **toujours** tester le diviseur avant de diviser.

12.2.3 Le code complet commenté

On utilise le type `double` pour accepter les nombres décimaux (ex. 14,5). Voici le programme complet, prêt à compiler avec `gcc calculatrice.c`.

```
C
1  /* calculatrice.c -- calculatrice a menu (OnBuch, Tle TI) */
2  #include <stdio.h>
3
4  /* --- Une fonction par operation : chacune renvoie un double --- */
5  double addition(double a, double b) {
6      return a + b;
7  }
8
9  double soustraction(double a, double b) {
10     return a - b;
11 }
12
13 double multiplication(double a, double b) {
14     return a * b;
15 }
16
17 /* La division renvoie a/b SEULEMENT si b est non nul. */
18 double division(double a, double b) {
19     return a / b;
20 }
21
22 int main(void) {
23     int choix;          /* le numero choisi dans le menu      */
24     double a, b;        /* les deux nombres a calculer */
25
26     do {
27         /* 1. Afficher le menu */
28         printf("\n===== CALCULATRICE OnBuch =====\n");
29         printf("1. Addition\n");
30         printf("2. Soustraction\n");
31         printf("3. Multiplication\n");
32         printf("4. Division\n");
33         printf("5. Quitter\n");
34         printf("Votre choix : ");
35         scanf("%d", &choix);
36
37         /* 2. Si on calcule (choix 1 a 4), lire les deux nombres */
38         if (choix >= 1 && choix <= 4) {
39             printf("Premier nombre  : ");
40             scanf("%lf", &a);
41             printf("Second nombre   : ");
42             scanf("%lf", &b);
43         }
44
45         /* 3. Choisir l'operation */
46         switch (choix) {
47             case 1:
```

```

48     printf("Resultat : %.2f\n", addition(a, b));
49     break;
50 case 2:
51     printf("Resultat : %.2f\n", soustraction(a, b));
52     break;
53 case 3:
54     printf("Resultat : %.2f\n", multiplication(a, b));
55     break;
56 case 4:
57     /* GESTION DE LA DIVISION PAR ZERO */
58     if (b == 0) {
59         printf("Erreur : division par zero impossible !\n");
60     } else {
61         printf("Resultat : %.2f\n", division(a, b));
62     }
63     break;
64 case 5:
65     printf("Au revoir !\n");
66     break;
67 default:
68     printf("Choix invalide, recommencez.\n");
69 }
70 } while (choix != 5); /* 4. Recommencer tant que != 5 */
71
72 return 0;
73 }

```

Calculatrice complète : menu, fonctions, switch et gestion de la division par zéro.

💡 Remarque. On lit un double avec `%lf` dans `scanf` (le « l » est obligatoire), mais on l'affiche avec `%f` (ou `%.2f` pour deux décimales) dans `printf`. C'est une distinction classique qui piège les débutants.

12.2.4 Tests

On vérifie le programme avec des **jeux d'essai** : des valeurs choisies pour couvrir le cas normal *et* les cas limites.

Choix	a	b	Résultat attendu
1 (addition)	7	3	10,00
2 (soustraction)	7	3	4,00
3 (multiplication)	7	3	21,00
4 (division)	7	2	3,50
4 (division)	7	0	message d'erreur
9 (inexistant)	—	—	« Choix invalide »
5 (quitter)	—	—	fin du programme

Déboguer. Si un résultat est faux : (1) vérifie le **format** (`%lf` en lecture, `%f` en affichage); (2) ajoute des `printf` temporaires pour afficher les valeurs de `a`, `b`, `choix`; (3) relis l'algorithme : la logique est-elle bonne ?

12.2.5 Améliorations possibles

Le projet de base fonctionne ; on peut l'enrichir :

- ajouter le **modulo** (reste de la division entière) ;
- ajouter la **puissance** (a^b) ;
- calculer la **moyenne** de deux nombres ;
- enregistrer l'historique des calculs dans un tableau.

Ces extensions sont l'objet des exercices à la fin du chapitre.

12.3 Projet 2 — Gestion des notes d'une classe

Second projet de synthèse : un mini-logiciel pour un professeur du *Lycée Bilingue de Bafoussam*. Il gère les moyennes d'une classe à l'aide d'un **tableau de structures**.

12.3.1 Cahier des charges

- 1) Chaque élève est décrit par un **nom** et une **moyenne**.
- 2) Le programme propose un **menu** : 1) Ajouter un élève, 2) Afficher la liste, 3) Moyenne de la classe, 4) Major (meilleur élève), 5) Quitter.
- 3) On peut enregistrer jusqu'à 50 élèves.
- 4) Le programme recommence tant qu'on n'a pas choisi « Quitter ».

12.3.2 Conception

On définit une **structure** `Eleve` regroupant le nom et la moyenne. Les élèves sont rangés dans un **tableau** de `Eleve`. Une variable `nb` compte combien d'élèves ont été saisis. Chaque action du menu devient une **fonction**.

```
struct Eleve { char nom[30]; float moyenne; };    puis        un        tableau        :
struct Eleve classe[50];
```

 **Remarque.** On passe le tableau `classe` et le nombre `nb` aux fonctions. En C, un tableau passé à une fonction peut être **modifié** par celle-ci (il est passé par son adresse). C'est pourquoi `ajouter` peut renvoyer le nouveau `nb`.

12.3.3 Le code complet commenté

```
C
/* notes.c -- gestion des notes d'une classe (OnBuch, Tle TI) */
#include <stdio.h>
#include <string.h>

#define MAX 50    /* nombre maximal d'eleves */

/* Un eleve : un nom et une moyenne */
struct Eleve {
    char    nom[30];
    float    moyenne;
};

/* Ajoute un eleve, renvoie le nouveau nombre d'eleves */
int ajouter(struct Eleve classe[], int nb) {
    if (nb >= MAX) {
        printf("Classe pleine !\n");
        return nb;
    }
}
```

```

18     }
19     printf("Nom de l'eleve   : ");
20     scanf("%s", classe[nb].nom);
21     printf("Moyenne sur 20   : ");
22     scanf("%f", &classe[nb].moyenne);
23     return nb + 1;    /* un eleve de plus */
24 }
25
26 /* Affiche la liste des eleves */
27 void afficher(struct Eleve classe[], int nb) {
28     int i;
29     if (nb == 0) {
30         printf("Aucun eleve enregistre.\n");
31         return;
32     }
33     printf("\n--- Liste de la classe ---\n");
34     for (i = 0; i < nb; i++) {
35         printf("%d. %-15s %.2f/20\n", i + 1, classe[i].nom, classe[i].moyenne);
36     }
37 }
38
39 /* Calcule et affiche la moyenne de la classe */
40 void moyenneClasse(struct Eleve classe[], int nb) {
41     int i;
42     float somme = 0;
43     if (nb == 0) {
44         printf("Aucun eleve : moyenne impossible.\n");
45         return;
46     }
47     for (i = 0; i < nb; i++) {
48         somme = somme + classe[i].moyenne;
49     }
50     printf("Moyenne de la classe : %.2f/20\n", somme / nb);
51 }
52
53 /* Affiche le major (meilleure moyenne) */
54 void major(struct Eleve classe[], int nb) {
55     int i, indiceMax;
56     if (nb == 0) {
57         printf("Aucun eleve enregistre.\n");
58         return;
59     }
60     indiceMax = 0;    /* on suppose le 1er meilleur */
61     for (i = 1; i < nb; i++) {
62         if (classe[i].moyenne > classe[indiceMax].moyenne) {
63             indiceMax = i;    /* on a trouve mieux */
64         }
65     }
66     printf("Major : %s avec %.2f/20\n",
67           classe[indiceMax].nom, classe[indiceMax].moyenne);
68 }
69
70 int main(void) {
71     struct Eleve classe[MAX];    /* le tableau d'eleves */
72     int nb = 0;    /* nombre d'eleves saisis */
73     int choix;
74
75     do {

```

```

76     printf("\n==== GESTION DES NOTES =====\n");
77     printf("1. Ajouter un eleve\n");
78     printf("2. Afficher la liste\n");
79     printf("3. Moyenne de la classe\n");
80     printf("4. Major de la classe\n");
81     printf("5. Quitter\n");
82     printf("Votre choix : ");
83     scanf("%d", &choix);
84
85     switch (choix) {
86         case 1: nb = ajouter(classe, nb); break;
87         case 2: afficher(classe, nb); break;
88         case 3: moyenneClasse(classe, nb); break;
89         case 4: major(classe, nb); break;
90         case 5: printf("Au revoir !\n"); break;
91         default: printf("Choix invalide.\n");
92     }
93     while (choix != 5);
94
95     return 0;
96 }

```

Gestion des notes : structure Eleve, tableau, et une fonction par action du menu.

⚠ Attention. `scanf("%s", classe[nb].nom)` lit un mot **sans espace** : tape `Awa` et non `Awa Bello`. Pour gérer les prénoms composés, il faudrait `fgets`, hors programme ici : on s'en tient à un seul mot.

💡 Remarque. Dans `scanf("%f", &classe[nb].moyenne)` on met le `&` (adresse) car on lit *vers* une variable. Mais pour `classe[nb].nom`, qui est un tableau de caractères, on n'écrit **pas** de `&` : le nom d'un tableau est déjà une adresse.

12.4 Exercices

► Application

Exercice 1 Étendre la calculatrice : le modulo

★ Ajoute au Projet 1 une option « 6. Modulo » qui affiche le reste de la division entière de `a` par `b` (opérateur `%` sur des entiers). Que faut-il prévoir si `b` vaut 0 ?

Exercice 2 Étendre la calculatrice : la puissance

★★ Ajoute une option « 7. Puissance » qui calcule a^b (b entier positif) à l'aide d'une fonction `puissance(double a, int b)` utilisant une boucle (sans `pow`).

Exercice 3 Mention au bulletin

★ Dans le Projet 2, écris une fonction `mention(float m)` qui affiche la mention selon la moyenne : ≥ 16 « Très Bien », ≥ 14 « Bien », ≥ 12 « Assez Bien », ≥ 10 « Passable », sinon « Insuffisant ». Appelle-la dans `afficher`.

Exercice 4 Compter les admis

★★ Ajoute au Projet 2 une option « Nombre d'admis » : une fonction `compterAdmis` qui renvoie le nombre d'élèves ayant une moyenne ≥ 10 , et affiche le résultat ainsi que le taux de réussite en pourcentage.

► Entraînement

Exercice 5 Trier les élèves par moyenne

*** Écris une fonction `trier(struct Eleve classe[], int nb)` qui range les élèves de la meilleure à la moins bonne moyenne par un **tri par sélection**. Ajoute une option de menu « Classement » qui trie puis affiche la liste.

Exercice 6 Historique de la calculatrice

*** Modifie la calculatrice pour qu'elle enregistre chaque résultat dans un tableau `double historique[100]`. Ajoute une option « Afficher l'historique » qui liste tous les calculs effectués depuis le lancement.

Exercice 7 Moyenne pondérée

** Une matière a une note et un coefficient. Définis une structure `Matiere` avec un nom, une note et un coefficient (entier), saisis 4 matières et calcule la moyenne pondérée $\frac{\sum(\text{note} \times \text{coef})}{\sum \text{coef}}$.

Exercice 8 Recherche d'un élève

** Dans le Projet 2, ajoute une option « Rechercher » qui demande un nom et affiche la moyenne de l'élève correspondant, ou « Élève introuvable » sinon (parcours du tableau + `strcmp`).

12.5 Corrigés

► Corrigé de l'exercice 1

Le modulo n'existe que pour les **entiers** en C (opérateur `%`). On ajoute un `case 6` et on protège contre le diviseur nul, exactement comme la division.

```
C
case 6:
    if ((int)b == 0) {
        printf("Erreur : modulo par zero impossible !\n");
    } else {
        printf("Reste : %d\n", (int)a % (int)b);
    }
    break;
```

On pense aussi à ajouter la ligne `printf("6. Modulon");` dans le menu et à modifier le test de lecture en `if (choix >= 1 && choix <= 6)`.

► Corrigé de l'exercice 2

La puissance se calcule en multipliant `a` par lui-même `b` fois.

```
C
double puissance(double a, int b) {
    double resultat = 1; /* a^0 vaut 1 */
    int i;
    for (i = 1; i <= b; i++) {
        resultat = resultat * a;
    }
    return resultat;
}
```

Dans `main`, on ajoute le `case 7` : `printf("Resultat : %.2fn", puissance(a, (int)b));`.

► Corrigé de l'exercice 3

Une cascade de `if...else if` testée de la plus haute mention à la plus basse.

```
C
1 void mention(float m) {
2     if (m >= 16) printf(" -> Tres Bien\n");
3     else if (m >= 14) printf(" -> Bien\n");
4     else if (m >= 12) printf(" -> Assez Bien\n");
5     else if (m >= 10) printf(" -> Passable\n");
6     else printf(" -> Insuffisant\n");
7 }
```

Dans `afficher`, après chaque ligne d'élève, on appelle `mention(classe[i].moyenne);`.

► Corrigé de l'exercice 4

On compte les moyennes ≥ 10 , puis on calcule le pourcentage.

```
C
1 int compterAdmis(struct Eleve classe[], int nb) {
2     int i, admis = 0;
3     for (i = 0; i < nb; i++) {
4         if (classe[i].moyenne >= 10) {
5             admis = admis + 1;
6         }
7     }
8     return admis;
9 }
```

Appel dans le menu :

```
C
1 int a = compterAdmis(classe, nb);
2 printf("Admis : %d / %d\n", a, nb);
3 if (nb > 0) {
4     printf("Taux de reussite : %.1f%%\n", 100.0 * a / nb);
5 }
```

► Corrigé de l'exercice 5

Tri par sélection : à chaque tour, on cherche le maximum dans la partie non triée et on l'échange avec le premier élément non trié.

```
C
1 void trier(struct Eleve classe[], int nb) {
2     int i, j, iMax;
3     struct Eleve temp;
4     for (i = 0; i < nb - 1; i++) {
5         iMax = i;
6         for (j = i + 1; j < nb; j++) {
7             if (classe[j].moyenne > classe[iMax].moyenne) {
8                 iMax = j;
9             }
10        }
11        /* echange classe[i] et classe[iMax] */
12        temp = classe[i];
13        classe[i] = classe[iMax];
14        classe[iMax] = temp;
15    }
16 }
```

L'échange de deux structures se fait simplement avec une variable temporaire `temp` de type `struct Eleve`. Dans le menu, on appelle `trier` puis `afficher`.

★ À retenir

Un projet se mène par **étapes** : cahier des charges → analyse → algorithme → codage → test. On **découpe** en fonctions (une fonction = une tâche), on **compile souvent** et on **teste les cas limites** (division par zéro, classe vide). C'est cette méthode, plus que le code lui-même, qui fait le bon programmeur.

Aide-mémoire

Mode d'emploi

Cette annexe rassemble, sous forme de fiches de référence denses, l'essentiel de la syntaxe vue dans le fascicule : **PHP**, **SQL**, l'extension **mysqli** (PHP + MySQL) et le langage **C**. Une dernière fiche met en correspondance PHP et C. À consulter pendant les TP et les révisions.

A.1 Aide-mémoire PHP

A.1.1 Balises, commentaires et instructions

`<?php` *instructions...* `?>` — fichier en `.php`, chaque instruction se termine par `;`.

PHP

```
<?php
1 // commentaire sur une ligne
2 # autre commentaire sur une ligne
3 /* commentaire
4    sur plusieurs lignes */
5 echo "Bonjour OnBuch"; // une instruction = un point-virgule
6 ?>
```

A.1.2 Variables et constantes

Une variable commence **toujours** par `$`. PHP est typé dynamiquement : le type dépend de la valeur affectée.

PHP

```
<?php
1 $nom    = "Awa";           // string (chaîne)
2 $age    = 17;              // int   (entier)
3 $moy    = 14.5;            // float (décimal)
4 $admis  = true;             // bool  (true / false)
5 define("VILLE", "Maroua"); // constante (sans $)
6 const TVA = 0.1925;         // autre forme de constante
7 ?>
```

Type	Exemple de valeur
int	42, -7, 0
float	14.5, 3.14
string	"Awa", 'Lycée'
bool	true, false
array	[1, 2, 3]
NULL	null

A.1.3 Afficher : echo et print

- `echo` affiche une ou plusieurs valeurs séparées par des virgules ;
- `print` affiche une seule valeur et renvoie 1 ;
- `.` concatène (colle) deux chaînes ; `.=` ajoute à la fin.

PHP

```

1 <?php
2 $nom = "Awa"; $age = 17;
3 echo "Élève : $nom", " (" , $age, " ans)"; // virgules
4 echo "Élève : " . $nom . " (" . $age . ")"; // concaténation
5 echo "Bonjour {$nom} !"; // accolades dans "..."
6 echo 'Texte brut : $nom'; // '...' n'interprète pas $
7 ?>

```

A.1.4 Opérateurs

Catégorie	Opérateurs	Exemple
Arithmétiques	+ - * / % **	\$x % 2 (reste)
Affectation	= += -= *= /= .=	\$s .= "!"
Comparaison	== != < > <= >=	\$a <= \$b
Identité	=== !==	\$a === \$b (valeur + type)
Logiques	&& !	\$x>0 && \$x<10
Incrémentement	++ --	\$i++
Ternaire	? :	\$m>=10 ? "Admis":"Échec"

A.1.5 Structures de contrôle

PHP

```

1 <?php
2 // if / elseif / else
3 if ($moy >= 16) { echo "Très bien"; }
4 elseif ($moy >= 10) { echo "Admis"; }
5 else { echo "Échec"; }
6
7 // switch
8 switch ($jour) {
9     case 1: echo "Lundi"; break;
10    case 2: echo "Mardi"; break;
11    default: echo "Autre";
12 }
13 ?>

```

PHP

```

1 <?php
2 // for : compteur connu
3 for ($i = 1; $i <= 5; $i++) { echo $i; }
4
5 // while : tant que la condition est vraie
6 $n = 1;
7 while ($n <= 5) { echo $n; $n++; }
8
9 // do...while : exécute au moins une fois
10 $n = 1;
11 do { echo $n; $n++; } while ($n <= 5);
12
13 // foreach : parcourir un tableau
14 foreach ($notes as $note) { echo $note; }
15 foreach ($eleve as $cle => $valeur) { echo "$cle = $valeur"; }
16 ?>

```

💡 **Remarque.** `break` sort de la boucle (ou du `switch`); `continue` passe directement à l'itération suivante.

A.1.6 Tableaux

PHP

```

1 <?php
2 // tableau indexé (clés 0, 1, 2...)
3 $notes = [12, 15, 9, 18];
4 echo $notes[0]; // 12
5 $notes[] = 11; // ajoute en fin
6
7 // tableau associatif (clé => valeur)
8 $eleve = ["nom" => "Awa", "age" => 17, "ville" => "Maroua"];
9 echo $eleve["nom"]; // Awa
10 ?>

```

Fonction	Rôle
<code>count(\$t)</code>	nombre d'éléments
<code>sort(\$t)</code>	trie par valeurs croissantes
<code>rsort(\$t)</code>	trie par valeurs décroissantes
<code>in_array(\$v, \$t)</code>	true si \$v est dans \$t
<code>array_push(\$t, \$v)</code>	ajoute \$v en fin
<code>implode(",", \$t)</code>	tableau → chaîne « 12,15,9 »
<code>explode(",", \$s)</code>	chaîne → tableau

A.1.7 Fonctions

```
function nom($param1, $param2 = défaut) { ...return $valeur; }
```

PHP

```

1 <?php
2 function moyenne($a, $b, $c) {
3     return ($a + $b + $c) / 3; // return renvoie le résultat
4 }
5 function bonjour($nom = "élève") { // valeur par défaut
6     return "Bonjour $nom";
7 }
8 echo moyenne(12, 15, 9); // 12
9 echo bonjour(); // Bonjour élève
10 ?>

```

A.1.8 Superglobales (formulaires & serveur)

Superglobale	Contenu
<code>\$_GET</code>	données passées dans l'URL (<code>?id=3</code>)
<code>\$_POST</code>	données d'un formulaire (<code>method="post"</code>)
<code>\$_REQUEST</code>	<code>\$_GET</code> + <code>\$_POST</code> réunis
<code>\$_FILES</code>	fichiers téléversés (upload)
<code>\$_SERVER</code>	infos serveur (<code>REQUEST_METHOD</code> , ...)
<code>\$_SESSION</code>	variables de session

PHP

```

1 <?php
2 // récupérer un champ de formulaire envoyé en POST
3 $nom = $_POST["nom"];
4 $age = $_GET["age"];
5
6 if ($_SERVER["REQUEST_METHOD"] == "POST") {
7     echo "Formulaire envoyé : " . $_POST["nom"];
8 }
9 ?>

```

⚠ Attention. Ne jamais faire confiance aux données reçues. Vérifie l'existence avec `isset($_POST["nom"])` avant de les utiliser.

A.1.9 Fonctions utiles

Fonction	Rôle	Exemple → résultat
<code>strlen(\$s)</code>	longueur d'une chaîne	<code>strlen("Awa")</code> → 3
<code>strtoupper(\$s)</code>	majuscules	<code>"awa"</code> → AWA
<code>strtolower(\$s)</code>	minuscules	<code>"AWA"</code> → awa
<code>trim(\$s)</code>	enlève les espaces	<code>" x "</code> → x
<code>round(\$n, 2)</code>	arrondi	<code>round(14.567,2)</code> → 14.57
<code>abs(\$n)</code>	valeur absolue	<code>abs(-5)</code> → 5
<code>rand(1, 6)</code>	entier aléatoire	<code>rand(1,6)</code> → 4
<code>date("d/m/Y")</code>	date du jour	→ 26/06/2026
<code>isset(\$v)</code>	<code>\$v</code> existe ?	true / false

A.2 Aide-mémoire SQL

A.2.1 Types de données courants

Type	Usage
INT	nombre entier (âge, identifiant)
FLOAT	nombre décimal (moyenne, prix)
VARCHAR(n)	chaîne de longueur variable, max n
TEXT	texte long
DATE	date au format AAAA-MM-JJ
BOOLEAN	vrai / faux (0 ou 1)

A.2.2 Créer une table

SQL

```

1 CREATE TABLE eleves (
2   id          INT PRIMARY KEY AUTO_INCREMENT,
3   nom         VARCHAR(50) NOT NULL,
4   age         INT,
5   moyenne     FLOAT,
6   ville       VARCHAR(40),
7   inscrit     DATE
8 );

```

💡 **Remarque.** La **clé primaire** (PRIMARY KEY) identifie chaque ligne de façon unique. **AUTO_INCREMENT** fait que la base attribue elle-même les valeurs 1, 2, 3, ...

A.2.3 Insérer des données

SQL

```

1 INSERT INTO eleves (nom, age, moyenne, ville, inscrit)
2 VALUES ('Awa', 17, 14.5, 'Maroua', '2026-09-01');
3
4 INSERT INTO eleves (nom, age) VALUES ('Bello', 18);

```

A.2.4 Lire : SELECT

SQL

```

1 SELECT * FROM eleves;           -- toutes les colonnes
2 SELECT nom, moyenne FROM eleves; -- colonnes choisies
3
4 SELECT * FROM eleves
5 WHERE moyenne >= 10             -- filtre (condition)
6 ORDER BY moyenne DESC;         -- tri décroissant
7
8 SELECT * FROM eleves WHERE ville = 'Maroua' AND age > 16;
9 SELECT COUNT(*) FROM eleves;    -- nombre de lignes

```

Clause	Rôle
WHERE	condition de sélection
ORDER BY	tri (ASC croissant, DESC décroissant)
AND OR	combinaison des conditions
LIKE	motif texte ('A%' = commence par A)
LIMIT n	limiter à n lignes

A.2.5 Modifier et supprimer

SQL

```
UPDATE eleves SET moyenne = 16 WHERE nom = 'Awa';

DELETE FROM eleves WHERE id = 5;
```

⚠ Attention. Un UPDATE ou un DELETE sans WHERE modifie ou supprime **toutes** les lignes de la table. Toujours préciser la condition !

A.3 Aide-mémoire mysqli (PHP + MySQL)

A.3.1 Fonctions essentielles

Fonction	Rôle
mysqli_connect(h,u,p,bd)	se connecter au serveur MySQL
mysqli_select_db(\$c,bd)	choisir la base de données
mysqli_query(\$c, \$sql)	exécuter une requête SQL
mysqli_fetch_assoc(\$r)	1 ligne en tableau associatif
mysqli_fetch_row(\$r)	1 ligne en tableau indexé
mysqli_fetch_array(\$r)	1 ligne (indexé + associatif)
mysqli_num_rows(\$r)	nombre de lignes du résultat
mysqli_close(\$c)	fermer la connexion
mysqli_connect_error()	message d'erreur de connexion

A.3.2 Squelette de connexion

PHP

```
<?php
$serveur = "localhost";
$user     = "root";
$passe    = "";
$base     = "onbuch";

// connexion (die() arrête le script en cas d'échec)
$conn = mysqli_connect($serveur, $user, $passe, $base)
        or die("Connexion impossible : " . mysqli_connect_error());

?>
```

A.3.3 Lire et afficher (boucle)

PHP

```

1 <?php
2 $sql = "SELECT nom, moyenne FROM eleves ORDER BY moyenne DESC";
3 $res = mysqli_query($conn, $sql);
4
5 if (mysqli_num_rows($res) > 0) {
6     // parcourir chaque ligne du résultat
7     while ($ligne = mysqli_fetch_assoc($res)) {
8         echo $ligne["nom"] . " : " . $ligne["moyenne"] . "<br>";
9     }
10 } else {
11     echo "Aucun élève trouvé.";
12 }
13
14 mysqli_close($conn); // toujours fermer à la fin
15 ?>

```

 **Remarque.** `mysqli_fetch_assoc` renvoie un tableau dont les clés sont les *noms de colonnes* (`$ligne["nom"]`). `mysqli_fetch_row` renvoie des clés numériques (`$ligne[0]`).

A.3.4 Insérer depuis un formulaire

PHP

```

1 <?php
2 $nom = $_POST["nom"];
3 $age = $_POST["age"];
4 $sql = "INSERT INTO eleves (nom, age) VALUES ('$nom', $age)";
5 if (mysqli_query($conn, $sql)) {
6     echo "Élève ajouté !";
7 } else {
8     echo "Erreur : " . mysqli_error($conn);
9 }
10 ?>

```

A.4 Aide-mémoire langage C

A.4.1 Structure d'un programme

C

```

1 #include <stdio.h>      /* bibliothèque entrées / sorties */
2 #include <stdlib.h>
3
4 int main() {
5     printf("Bonjour OnBuch\n");
6     return 0;           /* 0 = le programme s'est bien terminé */
7 }

```

A.4.2 Types et déclarations

Type	Contenu	Format printf/scanf
int	entier	%d
float	décimal	%f
double	décimal précis	%lf
char	1 caractère	%c
char[]	chaîne	%s

C

```
1 int age = 17;           /* déclaration + initialisation */
2 float moyenne = 14.5;
3 char lettre = 'A';
4 char nom[30] = "Awa";  /* chaîne = tableau de char */
```

A.4.3 Afficher et lire : printf / scanf

C

```
1 int age;
2 float moy;
3 printf("Entre ton age : ");
4 scanf("%d", &age);      /* & = adresse de la variable */
5 printf("Entre ta moyenne : ");
6 scanf("%f", &moy);
7 printf("Age : %d, moyenne : %.2f\n", age, moy);
```

⚠ Attention. Dans `scanf`, on met `&` devant le nom de la variable (sauf pour une chaîne `char[]`). `\n` = retour à la ligne, `%.2f` = 2 décimales.

A.4.4 Opérateurs

Catégorie	Opérateurs
Arithmétiques	+ - * / %
Comparaison	== != < > <= >=
Logiques	&& !
Affectation	= += -= *= /=
Incrémentation	++ --

A.4.5 Structures de contrôle

C

```
1 /* if / else if / else */
2 if (moy >= 16) printf("Tres bien");
3 else if (moy >= 10) printf("Admis");
4 else printf("Echec");
5
6 /* switch */
7 switch (jour) {
8     case 1: printf("Lundi"); break;
9     case 2: printf("Mardi"); break;
10    default: printf("Autre");
```



```
1 }

```

```
C
/* for */
for (int i = 1; i <= 5; i++) printf("%d ", i);

/* while */
int n = 1;
while (n <= 5) { printf("%d ", n); n++; }

/* do...while : exécute au moins une fois */
n = 1;
do { printf("%d ", n); n++; } while (n <= 5);

```

A.4.6 Fonctions

```
type nom(type param1, type param2) { ...return valeur; }

```

```
C
/* fonction qui renvoie un float */
float moyenne(int a, int b, int c) {
    return (a + b + c) / 3.0;
}

/* fonction sans retour : void */
void saluer(char nom[]) {
    printf("Bonjour %s\n", nom);
}

int main() {
    printf("%.2f\n", moyenne(12, 15, 9));
    saluer("Awa");
    return 0;
}

```

A.4.7 Tableaux et structures

```
C
/* tableau de 4 entiers */
int notes[4] = {12, 15, 9, 18};
printf("%d", notes[0]);          /* 12 */

for (int i = 0; i < 4; i++)
    printf("%d ", notes[i]);

/* structure (struct) : regrouper plusieurs champs */
struct Eleve {
    char nom[30];
    int age;
    float moyenne;
};

```

```

14 struct Eleve e1 = {"Awa", 17, 14.5};
15 printf("%s a %d ans\n", e1.nom, e1.age);    /* accès par . */
16

```

A.5 Correspondance PHP ↔ C

Les deux langages partagent une syntaxe proche (héritée du C). Voici les différences à connaître pour passer de l'un à l'autre.

Élément	PHP	C
Variable	\$age = 17;	int age = 17;
Décimal	\$m = 14.5;	float m = 14.5;
Chaîne	\$nom = "Awa";	char nom[] = "Awa";
Affichage	echo \$age;	printf("%d", age);
Saisie	\$_POST["x"]	scanf("%d", &x);
Condition	if (\$x > 0) { }	if (x > 0) { }
Boucle for	for(\$i=0;\$i<5;\$i++)	for(int i=0;i<5;i++)
Fonction	function f(){return;}	int f(){return 0;}
Commentaire	// ... ou # ...	// ... ou /* ...*/
Tableau	[12, 15, 9]	{12, 15, 9}
Fin d'instr.	;	;

💡 Remarque. Différence majeure : en **PHP**, on ne déclare pas le type des variables (typage dynamique) et tout commence par **\$** ; en **C**, chaque variable a un type fixe déclaré à l'avance (typage statique) et il faut compiler le programme avant de l'exécuter.

★ À retenir

Garde ces fiches à portée de main : **PHP** pour le web dynamique côté serveur, **SQL** pour interroger la base, **mysql** pour relier les deux, et **C** pour la programmation procédurale. La logique (conditions, boucles, fonctions) est la même partout : seule la syntaxe change.

Sujets type Baccalauréat TI (corrigés)

Tous ces sujets respectent la **structure réelle** de l'épreuve d'« Algorithmique et Programmation » de la série TI : **Partie I — Programmation web** (HTML, JavaScript, PHP, MySQL — 10 points) et **Partie II — Programmation procédurale en C** (10 points), pour une durée de **3 heures** (coef. 4). Le premier est une *vraie épreuve* tombée en composition ; les suivants sont des *sujets originaux* OnBuch calqués sur le même modèle. Compose en temps limité, *puis* confronte ta copie au corrigé.

B.1 Bac Blanc 2025 — Collège F.-X. Vogt

Une vraie annale

Voici, transcrite fidèlement, l'épreuve de **Baccalauréat Blanc** (séquence 06) du **Collège Mgr François-Xavier Vogt**, année 2024/2025, série **TI**. Elle réunit, comme à l'examen, une **Partie I — Programmation web** (HTML, JavaScript, PHP, MySQL) et une **Partie II — Programmation procédurale en C** (structures, fonctions, récursivité). Barèmes d'origine conservés. Cherche d'abord seul, puis confronte ta copie au corrigé complet.

COLLÈGE Mgr FRANÇOIS-XAVIER VOGT

Département : Informatique

BACCALAURÉAT BLANC — Séquence 06

Année scolaire : 2024/2025

Épreuve : Algorithmique et Programmation

Niveau : T^{les} TI

Durée : **03 heures** • Coefficient : **04**

L'épreuve comporte deux parties indépendantes, notées chacune sur 10 points.

Partie I — Programmation web

(10 points)

Contexte. « Vous êtes stagiaire au sein d'une entreprise locale de commerce électronique. L'un des projets consiste à développer une application web pour la gestion des commandes de produits et sa publication via un hébergeur web dynamique. L'entreprise opte pour une programmation modulaire et vous êtes chargé du module d'ajout de nouveaux produits. Ce module consiste à créer deux pages web. Une page permet de saisir les détails du produit, et ces détails seront stockés dans une base de données nommée `ECOM_DB`, où l'une des tables est nommée `PRODUITS`. »

Exercice 1 — Programmation HTML et JavaScript

(03 pts)

1. (0,25 pt) Définir : *Application web*.

2. (1,25 pt) Écrire le code HTML pour afficher le formulaire (nommé `productForm`). La méthode est `POST`, et le fichier de traitement est nommé `save_product.php`. Les noms des champs du formulaire sont respectivement « *Nom du produit* », « *Catégorie* », « *Prix* », « *Quantité* » et « *Enregistrer* ». Vous pouvez utiliser un tableau de 05 lignes et 02 colonnes. Inclure une fonction JavaScript de validation de formulaire nommée `validateForm()`.
3. (0,5 pt) Préciser une autre méthode de récupération des données d'un formulaire, puis donner un avantage de la méthode `POST` sur cette autre méthode.
4. (0,75 pt) On désire récupérer les données issues du formulaire et les enregistrer en vérifiant juste que tous les champs sont renseignés. Si c'est le cas, le message « *Produit enregistré avec succès* » est affiché dans une boîte de dialogue ; ou bien « *Échec d'enregistrement du produit* » dans le cas contraire. Écrire en JavaScript le code de la fonction `validateForm()` permettant d'effectuer cette tâche au clic du bouton « *Enregistrer* ».

Exercice 2 — Programmation PHP et MySQL

(07 pts)

On s'intéresse maintenant à la base de données `ECOM_DB` et au moyen d'y accéder.

1. (0,25 pt) Définir : *Hébergeur web*.
2. (0,25 pt) Donner la signification de *PHP*.
3. (0,5 pt) Donner deux avantages du PHP par rapport à JavaScript.
4. (0,75 pt) Donner les éléments constitutifs d'un environnement intégré de développement web.
5. (0,25 pt) Indiquer le côté dans lequel sont exécutés les scripts PHP.
6. (1,5 pt) Écrire un code PHP qui, au clic du bouton « *Envoyer* », récupère les informations du formulaire et les affiche. On prendra pour nom d'hôte `localhost`, l'utilisateur `root` et le mot de passe vide.

Soit l'extrait de code ci-dessous :

```

1 <?php
2 $result = mysqli_query($conn, "SELECT Product_Name, Category, Price FROM PRODUCTS") or
3     die("ERREUR: " . mysqli_error($conn));
4 while ($row = mysqli_fetch_array($result)) {
5     echo "Nom du produit: " . $row["Product_Name"] . "<br/>";
6     echo "Catégorie: " . $row["Category"] . "<br/>";
7     echo "Prix: " . $row["Price"] . "<br/>";
8 }
9 ?>

```

7. (1 pt) Donner le rôle des fonctions ci-dessous : `mysqli_fetch_assoc()` ; `die()`.
8. (0,5 pt) Présenter la différence entre les fonctions `mysqli_fetch_array()` et `mysqli_fetch_row()`.
9. (2 pts) Écrire un code PHP qui permet de supprimer un produit de la base de données, à partir du nom saisi dans un formulaire adéquat, puis d'afficher le contenu de la table modifiée dans un tableau HTML.

Partie II — Programmation procédurale en C

(10 points)

Exercice 1

(5 pts)

« *Fadil a été choisi pour écrire un programme C permettant de gérer les stagiaires caractérisés par leur numéro d'inscription (entier), nom (chaîne), prénom (chaîne), filière (chaîne) et moyenne (réel). On lui a notamment confié la partie consistant à implémenter trois fonctions : Enregistrement, Modification et Recherche.* » Aider Fadil à réaliser les tâches ci-dessous.

1. (1,5 pt) Donner l'instruction C permettant de déclarer une structure d'enregistrement nommée `stagiaire` pour gérer les stagiaires avec les caractéristiques données dans le texte.
2. (0,5 pt) Donner l'instruction C permettant de déclarer un tableau `T` de 100 stagiaires.
3. (0,5 pt) Donner deux avantages de l'utilisation des fonctions dans un programme.
4. (2,5 pts) Donner en C le code de définition de la fonction `Rechercher` permettant de rechercher un stagiaire par son numéro d'inscription dans un tableau. La fonction affiche le nom, le prénom et la filière du stagiaire recherché. Dans le cas où le stagiaire n'est pas présent dans le tableau, la fonction affiche le message : « *Aucun stagiaire ne correspond à ce numéro d'inscription* ». La fonction prend comme paramètres le tableau de stagiaires `T`, sa taille `n` et le numéro d'inscription à rechercher `num_insc`. L'en-tête de la fonction est : `void Rechercher(struct stagiaire T[], int n, int num_insc)`.

Exercice 2

(5 pts)

On donne le programme C ci-dessous, qui calcule la factorielle d'un nombre (les numéros de ligne 1 à 27 sont rappelés en commentaire pour pouvoir y faire référence) :

```
C
1  /* 1*/  /* Exemple de fonction recurrente. Calcul de la factorielle d'un nombre */
2  /* 2*/  #include <stdio.h>
3  /* 3*/  #include <stdlib.h>
4  /* 4*/  unsigned int f, x;
5  /* 5*/  unsigned int factorielle(unsigned int a);
6  /* 6*/  int main() {
7  /* 7*/      puts("Entrez une valeur entiere entre 1 et 8: ");
8  /* 8*/      scanf("%d", &x);
9  /* 9*/      if (x > 8 || x < 1) {
10 /*10*/         printf("On a dit entre 1 to 8 !");
11 /*11*/     }
12 /*12*/     else {
13 /*13*/         f = factorielle(x);
14 /*14*/         printf("Factorielle %u egal %u\n", x, f);
15 /*15*/     }
16 /*16*/     exit(EXIT_SUCCESS);
17 /*17*/ }
18 /*18*/ unsigned int factorielle(unsigned int a) {
19 /*19*/     if (a == 1) {
20 /*20*/         return 1;
21 /*21*/     }
22 /*22*/     else {
23 /*23*/         a *= factorielle(a-1);
24 /*24*/         return a;
25 /*25*/     }
26 /*26*/ }
27 /*27*/
```

1. Donner le rôle de :
 - 1.1 (0,25 pt) la ligne 5 (la déclaration `unsigned int factorielle(unsigned int a);`);
 - 1.2 (0,25 pt) la ligne 8 (`scanf("%d", &x);`).
2. (0,5 pt) Donner la différence fondamentale entre la variable `x` et la variable `a` dans ce code.
3. (1,5 pt) Exécuter pas à pas ce programme lorsque la valeur de `x = 6` (présenter la trace, de préférence dans un tableau).
4. (1 pt) Réécrire les instructions de la structure `if/else` (lignes 9 à 13) en utilisant l'opérateur ternaire (`? :`).

5. (1,5 pt) Réécrire la version itérative de la fonction `factorielle`.

► **Corrigé de l'annale Vogt 2025**

Partie I — Exercice 1 (HTML / JavaScript)

1. Définition — Application web. Une *application web* est un logiciel applicatif accessible et exécuté à travers un navigateur web, via le réseau Internet ou un intranet. Côté serveur, elle s'appuie sur un langage dynamique (PHP, etc.) et, le plus souvent, sur une base de données; l'utilisateur n'installe rien sur sa machine.

2. Formulaire HTML (`productForm`) avec tableau 5×2.

```

HTML
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Ajout d'un produit</title>
  <script src="validation.js"></script>
</head>
<body>
  <h2>Nouveau produit</h2>
  <form name="productForm" method="POST" action="save_product.php"
    onsubmit="return validateForm()">
    <table border="1" cellpadding="6">
      <tr>
        <td>Nom du produit</td>
        <td><input type="text" name="Nom du produit"></td>
      </tr>
      <tr>
        <td>Catégorie</td>
        <td><input type="text" name="Catégorie"></td>
      </tr>
      <tr>
        <td>Prix</td>
        <td><input type="number" name="Prix"></td>
      </tr>
      <tr>
        <td>Quantité</td>
        <td><input type="number" name="Quantité"></td>
      </tr>
      <tr>
        <td colspan="2" align="center">
          <input type="submit" name="Enregistrer" value="Enregistrer">
        </td>
      </tr>
    </table>
  </form>
</body>
</html>

```

Tableau de 5 lignes × 2 colonnes; la validation est déclenchée par `onsubmit`.

💡 **Remarque.** Le formulaire porte le nom `productForm`, la méthode est `POST` et l'attribut `action` pointe vers `save_product.php`. L'attribut `onsubmit="return validateForm()"` bloque l'envoi si la fonction retourne `false`.

3. Autre méthode + avantage de POST. L'autre méthode de récupération des données d'un formulaire est la méthode **GET** (les données circulent dans l'URL, après le `?`). Un avantage de **POST** sur **GET** : avec **POST**, les données sont transmises dans le *corps* de la requête (elles n'apparaissent pas dans l'URL), ce qui est plus **sûr/discret** et permet d'envoyer un **volume de données plus important** (GET est limité par la longueur de l'URL).

4. Fonction `validateForm()`.

JavaScript

```
function validateForm() {
    // Récupération des champs du formulaire productForm
    var f = document.forms["productForm"];
    var nom      = f["Nom du produit"].value;
    var categorie = f["Catégorie"].value;
    var prix     = f["Prix"].value;
    var quantite = f["Quantité"].value;

    // Vérifie que tous les champs sont renseignés
    if (nom === "" || categorie === "" || prix === "" || quantite === "") {
        alert("Échec d'enregistrement du produit");
        return false; // empêche l'envoi du formulaire
    } else {
        alert("Produit enregistré avec succès");
        return true;  // autorise l'envoi vers save_product.php
    }
}
```

 **Remarque.** `return false` empêche la soumission (les champs sont incomplets); `return true` laisse le formulaire partir vers `save_product.php`. La fonction est appelée par l'attribut `onsubmit="return validateForm()"` placé sur la balise `<form>`.

Partie I — Exercice 2 (PHP / MySQL)

1. Définition — Hébergeur web. Un *hébergeur web* est une entreprise (ou un prestataire) qui met à disposition des serveurs connectés en permanence à Internet pour stocker les fichiers d'un site web et le rendre accessible 24h/24 aux internautes (souvent avec base de données, nom de domaine, etc.).

2. Signification de PHP. *PHP* signifie **PHP : Hypertext Preprocessor** (à l'origine *Personal Home Page*).

3. Deux avantages du PHP par rapport à JavaScript.

- PHP s'exécute **côté serveur** : il peut se connecter à une base de données et manipuler des fichiers du serveur en toute sécurité (le code source n'est pas visible par le client);
- PHP permet de **générer dynamiquement du HTML** et de gérer la persistance des données (sessions, base de données), alors que JavaScript (côté client) ne peut pas accéder directement à la base de données.

4. Éléments constitutifs d'un environnement intégré de développement web. Un EID web (de type WampServer/XAMPP) regroupe : un **serveur web** (Apache), un **interpréteur PHP** (le langage serveur), un **SGBD** (MySQL / MariaDB) et un outil d'administration de la base (**phpMyAdmin**). On y ajoute un **éditeur de code** et un **navigateur web**.

5. Côté d'exécution des scripts PHP. Les scripts PHP sont exécutés **du côté serveur** (server side).

6. Code PHP de récupération et d'affichage des données du formulaire.

PHP

```
<?php
// save_product.php - récupère et affiche les données du formulaire
if (isset($_POST["Enregistrer"])) {
    // Connexion à la base ECOM_DB
    $conn = mysqli_connect("localhost", "root", "", "ECOM_DB");
    if (!$conn) {
        die("Connexion echouee: " . mysqli_connect_error());
    }

    // Récupération des champs envoyés par le formulaire (méthode POST)
    $nom      = $_POST["Nom du produit"];
    $categorie = $_POST["Catégorie"];
    $prix      = $_POST["Prix"];
    $quantite  = $_POST["Quantité"];

    // Affichage des informations reçues
    echo "Nom du produit : " . $nom . "<br/>";
    echo "Catégorie : "      . $categorie . "<br/>";
    echo "Prix : "           . $prix . " FCFA<br/>";
    echo "Quantité : "       . $quantite . "<br/>";

    mysqli_close($conn);
}
?>
```

7. Rôle des fonctions.

- `mysqli_fetch_assoc()` : récupère la *ligne courante* d'un résultat de requête `SELECT` sous forme de **tableau associatif**, dont les clés sont les **noms des colonnes** (ex. `$row["Product_Name"]`). À chaque appel, on avance d'une ligne; elle renvoie `NULL` quand il n'y a plus de lignes.
- `die()` : **arrête immédiatement l'exécution** du script PHP (équivalent de `exit()`); on lui passe souvent un message qui sera affiché avant l'arrêt. Elle sert à interrompre proprement le script en cas d'erreur (échec de connexion ou de requête).

8. Différence entre `mysqli_fetch_array()` et `mysqli_fetch_row()`. `mysqli_fetch_row()` renvoie la ligne sous forme de tableau **indexé numériquement** uniquement (accès par `$row[0]`, `$row[1]` ...). En revanche, `mysqli_fetch_array()` renvoie par défaut un tableau **à la fois indexé numériquement ET associatif** (accès possible par `$row[0]` ou par `$row["nom_colonne"]`).

9. Code PHP de suppression d'un produit + ré-affichage du tableau.

PHP

```
<?php
// delete_product.php - supprime un produit puis réaffiche la table
$conn = mysqli_connect("localhost", "root", "", "ECOM_DB");
if (!$conn) {
    die("Connexion echouee: " . mysqli_connect_error());
}

// Si le formulaire de suppression a été soumis
if (isset($_POST["supprimer"])) {
    $nom = $_POST["nom_produit"];
    $sql = "DELETE FROM PRODUITS WHERE Product_Name = '" . $nom . "'";
    if (mysqli_query($conn, $sql)) {
```



```

13     echo "<p>Produit supprimé : " . $nom . "</p>";
14 } else {
15     echo "<p>Erreur : " . mysqli_error($conn) . "</p>";
16 }
17 }
18 ?>
19
20 <!-- Formulaire de saisie du nom à supprimer -->
21 <form method="POST" action="delete_product.php">
22     Nom du produit à supprimer :
23     <input type="text" name="nom_produit">
24     <input type="submit" name="supprimer" value="Supprimer">
25 </form>
26
27 <!-- Affichage du contenu de la table modifiée dans un tableau HTML -->
28 <table border="1" cellpadding="6">
29     <tr>
30         <th>Nom du produit</th>
31         <th>Catégorie</th>
32         <th>Prix</th>
33     </tr>
34     <?php
35     $result = mysqli_query($conn, "SELECT Product_Name, Category, Price FROM PRODUITS");
36     while ($row = mysqli_fetch_assoc($result)) {
37         echo "<tr>";
38         echo "<td>" . $row["Product_Name"] . "</td>";
39         echo "<td>" . $row["Category"] . "</td>";
40         echo "<td>" . $row["Price"] . "</td>";
41         echo "</tr>";
42     }
43     mysqli_close($conn);
44     ?>
45 </table>

```

⚠ Attention. En production, on ne concatène jamais directement une saisie utilisateur dans une requête (risque d'injection SQL). On utiliserait une **requête préparée**. Ici, le code suit le niveau attendu à l'examen Terminale TI.

Partie II — Exercice 1 (structures et fonctions en C)

1. Déclaration de la structure `stagiaire`.

```

C
1 struct stagiaire {
2     int num_insc;      /* numéro d'inscription (entier) */
3     char nom[50];      /* nom (chaîne de caractères) */
4     char prenom[50];   /* prénom (chaîne de caractères) */
5     char filiere[50];  /* filière (chaîne de caractères) */
6     float moyenne;    /* moyenne (réel) */
7 };

```

2. Déclaration d'un tableau `T` de 100 stagiaires.

C

```
| struct stagiaire T[100];
```

3. Deux avantages des fonctions.

- Elles évitent la **répétition de code** (réutilisation) : on écrit un traitement une seule fois puis on l'appelle autant de fois que nécessaire;
- Elles rendent le programme plus **lisible, structuré et facile à maintenir/ déboguer** (découpage en sous-problèmes — programmation modulaire).

4. Définition de la fonction `Rechercher`.

C

```
1 #include <stdio.h>
2
3 void Rechercher(struct stagiaire T[], int n, int num_insc) {
4     int i;
5     int trouve = 0; /* drapeau : 0 = non trouvé, 1 = trouvé */
6
7     for (i = 0; i < n; i++) {
8         if (T[i].num_insc == num_insc) {
9             printf("Nom      : %s\n", T[i].nom);
10            printf("Prénom   : %s\n", T[i].prenom);
11            printf("Filière:  %s\n", T[i].filiere);
12            trouve = 1;
13            break; /* numéro d'inscription unique : on s'arrête */
14        }
15    }
16
17    if (trouve == 0) {
18        printf("Aucun stagiaire ne correspond à ce numéro d'inscription\n");
19    }
20 }
```

 **Remarque.** On parcourt le tableau du début à la fin. Dès qu'un `num_insc` correspond, on affiche le nom, le prénom et la filière, on lève le drapeau `trouve` et on sort de la boucle. Si aucune correspondance n'est trouvée après le parcours, on affiche le message d'absence.

Partie II — Exercice 2 (factorielle récursive)

1. Rôle des lignes.

- **Ligne 5** — `unsigned int factorielle(unsigned int a);` : c'est le **prototype** (déclaration anticipée) de la fonction `factorielle`. Il annonce au compilateur, avant `main`, le nom de la fonction, son type de retour (`unsigned int`) et le type de son paramètre, pour qu'elle puisse être appelée dans `main` (ligne 13) alors qu'elle n'est définie qu'ensuite (ligne 18).
- **Ligne 8** — `scanf("%d", &x);` : **lit au clavier** un entier saisi par l'utilisateur et le **range dans la variable** `x` (l'opérateur `&` fournit l'adresse de `x`).

2. Différence fondamentale entre `x` et `a`. `x` est une variable **globale** (déclarée ligne 4, hors de toute fonction : visible dans tout le programme), tandis que `a` est une variable **locale** — c'est le **paramètre** de la fonction `factorielle` (elle n'existe que pendant l'exécution de la fonction et reçoit une copie de la valeur passée à l'appel).

3. Trace d'exécution pour $x = 6$. L'appel `factorielle(6)` déclenche une suite d'appels récursifs jusqu'au cas de base `a == 1`, puis les valeurs « remontent ».

Appel	Test <code>a==1</code> ?	Calcul	Valeur retournée
<code>factorielle(6)</code>	non	<code>6 * factorielle(5)</code>	<code>6 * 120 = 720</code>
<code>factorielle(5)</code>	non	<code>5 * factorielle(4)</code>	<code>5 * 24 = 120</code>
<code>factorielle(4)</code>	non	<code>4 * factorielle(3)</code>	<code>4 * 6 = 24</code>
<code>factorielle(3)</code>	non	<code>3 * factorielle(2)</code>	<code>3 * 2 = 6</code>
<code>factorielle(2)</code>	non	<code>2 * factorielle(1)</code>	<code>2 * 1 = 2</code>
<code>factorielle(1)</code>	oui	cas de base	1

En remontant : $1 \rightarrow 2 \rightarrow 6 \rightarrow 24 \rightarrow 120 \rightarrow 720$. Le programme affiche donc : `Factorielle 6 egal 720`.

4. Réécriture des lignes 9 à 13 avec l'opérateur ternaire.

C

```
(x > 8 || x < 1)
? printf("On a dit entre 1 to 8 !")
: printf("Factorielle %u egal %u\n", x, f = factorielle(x));
```

Remarque. L'opérateur ternaire `condition ? valeur_si_vrai : valeur_si_faux` remplace le `if/else`. Si `(x > 8 || x < 1)` est vrai, on affiche le message d'erreur; sinon on calcule `f = factorielle(x)` et on l'affiche.

5. Version itérative de la fonction `factorielle`.

C

```
unsigned int factorielle(unsigned int a) {
    unsigned int resultat = 1;
    unsigned int i;
    for (i = 2; i <= a; i++) {
        resultat *= i; /* resultat = resultat * i */
    }
    return resultat; /* factorielle(1) et factorielle(0) renvoient 1 */
}
```

Remarque. La boucle multiplie `resultat` par tous les entiers de 2 à `a`. Pour `a = 6` : $1 \times 2 \times 3 \times 4 \times 5 \times 6 = 720$, comme la version récursive.

★ À retenir

Au Bac TI, la **Partie web** attend du code *exact* : un `<form>` (méthode, action), une validation JavaScript (`return false` bloque l'envoi), `mysqli_connect + $_POST` côté serveur, et l'affichage d'un `SELECT` dans un `<table>`. La **Partie C** attend une `struct` bien typée, un parcours de tableau avec drapeau pour la recherche, et la maîtrise de la **récursivité** (cas de base + cas récursif) face à sa version **itérative**.

B.2 Sujet TI n° 1 — Bibliothèque scolaire

Un sujet type d'examen

Voici un **sujet type** de Baccalauréat, série **TI**, épreuve **Algorithmique et Programmation**, calqué sur la structure réelle de l'examen. Il réunit une **Partie I — Programmation web** (HTML, JavaScript, PHP, MySQL) et une **Partie II — Programmation procédurale en C** (structures, fonctions, trace). Contexte : la gestion informatisée de la **bibliothèque d'un lycée camerounais**. Compose d'abord seul, dans les conditions de l'examen, puis confronte ta copie au corrigé détaillé qui suit.

OFFICE DU BACCALAURÉAT DU CAMEROUN

Examen : Baccalauréat de l'Enseignement Secondaire Technique

SUJET TYPE — Session de juin

Série : TI

Épreuve : Algorithmique et Programmation

Durée : **03 heures** • Coefficient : **04**

L'épreuve comporte deux parties indépendantes, notées chacune sur 10 points. L'usage de la calculatrice n'est pas autorisé.

Contexte général. « Le Lycée Bilingue de Bafoussam souhaite informatiser la gestion de sa bibliothèque scolaire, jusque-là tenue sur des registres papier. L'établissement confie à un stagiaire en informatique le développement d'une petite application web de gestion du fonds documentaire : une page permet de saisir les caractéristiques d'un livre, et ces informations sont enregistrées dans une base de données nommée `BIBLIO_DB`, dont la table principale est nommée `LIVRES` (champs : titre, auteur, catégorie, nombre d'exemplaires). En parallèle, un programme en langage C est écrit pour gérer hors ligne la liste des ouvrages. »

Partie I — Programmation web

(10 points)

Exercice 1 — Programmation HTML et JavaScript

(03,5 pts)

- (0,25 pt) Définir : *page web statique*.
- (0,25 pt) Citer le langage qui permet de structurer le contenu d'une page web et celui qui permet de la rendre interactive côté client.
- (1,5 pt) Écrire le code HTML affichant le formulaire de saisie d'un livre (nommé `bookForm`). La méthode est `POST` et le fichier de traitement est nommé `ajouter_livre.php`. Les champs sont respectivement « Titre », « Auteur », « Catégorie », « Nombre d'exemplaires » et le bouton « Ajouter ». On utilisera un tableau de 05 lignes et 02 colonnes. Le formulaire appellera, au clic du bouton, une fonction JavaScript de validation nommée `validateForm()`.
- (1,5 pt) On veut, au clic du bouton « Ajouter », vérifier que les champs *Titre*, *Auteur* et *Catégorie* ne sont pas vides, et que le champ *Nombre d'exemplaires* contient bien une valeur numérique (et non du texte). Si tout est correct, une boîte de dialogue affiche « Livre prêt à être enregistré » ; sinon, elle affiche « Saisie invalide : vérifiez les champs » et l'envoi du formulaire est annulé. Écrire en JavaScript le code de la fonction `validateForm()`.

Exercice 2 — Programmation PHP et MySQL

(06,5 pts)

On s'intéresse maintenant à la base de données `BIBLIO_DB` et au moyen d'y accéder depuis le script de traitement `ajouter_livre.php`.

1. (0,25 pt) Définir : *base de données*.
2. (0,25 pt) Donner la signification du sigle *SGBD*.
3. (0,25 pt) Indiquer le côté (client ou serveur) où s'exécute un script PHP.
4. (0,75 pt) Donner les éléments constitutifs d'un environnement intégré de développement web dynamique (citer au moins trois éléments).
5. (1 pt) Donner le rôle des fonctions suivantes : `mysqli_connect()` ; `mysqli_query()` .
6. (0,5 pt) Présenter la différence entre les fonctions `mysqli_fetch_assoc()` et `mysqli_fetch_row()` .
7. (1,75 pt) Écrire le code PHP du fichier `ajouter_livre.php` qui : (a) se connecte à la base `BIBLIO_DB` (hôte `localhost` , utilisateur `root` , mot de passe vide) ; (b) récupère les données envoyées par le formulaire ; (c) les affiche dans un tableau HTML (une ligne par champ).
8. (1,75 pt) Écrire la requête SQL, puis le code PHP, permettant **d'insérer** le nouveau livre dans la table `LIVRES` à partir des données récupérées du formulaire, en signalant le succès ou l'échec de l'opération.

Partie II — Programmation procédurale en C

(10 points)

Exercice 1

(5 pts)

« Pour gérer la bibliothèque hors ligne, on caractérise chaque livre par un code (entier), un titre (chaîne), un auteur (chaîne) et un nombre d'exemplaires (entier). On souhaite ranger l'ensemble des ouvrages dans un tableau, puis disposer d'une fonction permettant de retrouver rapidement un livre à partir de son code. » Réaliser les tâches ci-dessous.

1. (1,5 pt) Donner l'instruction C permettant de déclarer une structure d'enregistrement nommée `Livre` contenant les caractéristiques décrites dans le texte (les chaînes seront des tableaux de 50 caractères).
2. (0,5 pt) Donner l'instruction C permettant de déclarer un tableau `biblio` de 200 éléments de type `struct Livre` .
3. (0,5 pt) Donner deux avantages de l'utilisation des fonctions dans un programme.
4. (2,5 pts) Donner en C le code de définition de la fonction `rechercherLivre` qui recherche un livre par son code dans le tableau, en utilisant la **recherche séquentielle**. Si le livre est trouvé, la fonction affiche son titre, son auteur et son nombre d'exemplaires, puis renvoie l'indice de la case trouvée ; sinon elle affiche le message « *Aucun livre ne correspond à ce code* » et renvoie `-1`. L'entête imposé de la fonction est : `int rechercherLivre(struct Livre biblio[], int n, int code)` .

Exercice 2

(5 pts)

On donne le programme C ci-dessous, qui calcule la **somme des chiffres** d'un entier naturel saisi par l'utilisateur (les numéros de ligne 1 à 22 sont rappelés en commentaire pour pouvoir y faire référence) :

```
C
1  /* 1*/  /* Calcul de la somme des chiffres d'un entier naturel */
2  /* 2*/  #include <stdio.h>
3  /* 3*/  #include <stdlib.h>
4  /* 4*/  int sommeChiffres(int n);
5  /* 5*/  int main() {
6  /* 6*/      int x, s;
7  /* 7*/      printf("Entrez un entier naturel : ");
8  /* 8*/      scanf("%d", &x);
9  /* 9*/      if (x < 0) {
10 /*10*/         printf("Nombre negatif non autorise !\n");
11 /*11*/     }
12 /*12*/     else {
```

```

13 /*13*/      s = sommeChiffres(x);
14 /*14*/      printf("Somme des chiffres de %d = %d\n", x, s);
15 /*15*/      }
16 /*16*/      return 0;
17 /*17*/      }
18 /*18*/      int sommeChiffres(int n) {
19 /*19*/          int somme = 0;
20 /*20*/          while (n != 0) { somme = somme + n % 10; n = n / 10; }
21 /*21*/          return somme;
22 /*22*/      }

```

1. Donner le rôle de :
 - 1.1 (0,25 pt) la ligne 4 (la déclaration `int sommeChiffres(int n);`);
 - 1.2 (0,25 pt) la ligne 8 (`scanf("%d", &x);`).
2. (0,5 pt) Présenter la différence fondamentale entre la variable `x` (ligne 6) et la variable `n` (ligne 18) dans ce programme.
3. (1,5 pt) Exécuter pas à pas la fonction `sommeChiffres` lorsque `x = 472` (présenter la trace dans un tableau : tours de boucle, valeurs de `n`, de `n % 10` et de `somme`).
4. (1 pt) Réécrire les instructions de la structure `if/else` (lignes 9 à 15) en utilisant l'opérateur ternaire (`? :`) pour déterminer le message, l'affichage de la somme restant inchangé.
5. (1,5 pt) Réécrire la fonction `sommeChiffres` sous forme **récursive**.

► Corrigé du sujet TI n° 1

Partie I — Exercice 1 (HTML / JavaScript)

1. Définition — page web statique. Une *page web statique* est une page dont le contenu est figé : il est écrit une fois pour toutes en HTML et s'affiche à l'identique pour tous les visiteurs, sans traitement côté serveur ni accès à une base de données. (Par opposition, une page *dynamique* est générée à la volée, par exemple par PHP.)

2. Langages. Le langage qui structure le contenu d'une page web est **HTML**; celui qui la rend interactive côté client est **JavaScript**.

3. Formulaire HTML (bookForm) avec tableau 5×2.

```

HTML
1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4   <meta charset="utf-8">
5   <title>Ajout d'un livre</title>
6   <script src="validation.js"></script>
7 </head>
8 <body>
9   <h2>Nouveau livre</h2>
10  <form name="bookForm" method="POST" action="ajouter_livre.php"
11    onsubmit="return validateForm()">
12    <table border="1" cellpadding="6">
13      <tr>
14        <td>Titre</td>
15        <td><input type="text" name="titre" id="titre"></td>
16      </tr>
17      <tr>
18        <td>Auteur</td>

```

```

19     <td><input type="text" name="auteur" id="auteur"></td>
20 </tr>
21 <tr>
22     <td>Catégorie</td>
23     <td><input type="text" name="categorie" id="categorie"></td>
24 </tr>
25 <tr>
26     <td>Nombre d'exemplaires</td>
27     <td><input type="text" name="nbExemplaires" id="nbExemplaires"></td>
28 </tr>
29 <tr>
30     <td colspan="2" align="center">
31         <input type="submit" name="ajouter" value="Ajouter">
32     </td>
33 </tr>
34 </table>
35 </form>
36 </body>
37 </html>

```

Tableau de 5 lignes $\times$ 2 colonnes; la validation est déclenchée par `onsubmit`.

Remarque. On donne à chaque champ un attribut `id` pour pouvoir le retrouver facilement en JavaScript avec `document.getElementById`. Le champ *nombre d'exemplaires* est de type `text` (et non `number`) afin de pouvoir contrôler nous-mêmes, en JavaScript, qu'une valeur numérique a bien été saisie.

4. Fonction `validateForm()`.

JavaScript

```

function validateForm() {
    // Récupération des valeurs saisies (sans espaces superflus)
    var titre = document.getElementById("titre").value.trim();
    var auteur = document.getElementById("auteur").value.trim();
    var categorie = document.getElementById("categorie").value.trim();
    var nbExp = document.getElementById("nbExemplaires").value.trim();

    // 1) Les trois champs texte doivent être non vides
    if (titre === "" || auteur === "" || categorie === "") {
        alert("Saisie invalide : vérifiez les champs");
        return false; // annule l'envoi du formulaire
    }

    // 2) Le nombre d'exemplaires doit être une valeur numérique
    if (nbExp === "" || isNaN(nbExp)) {
        alert("Saisie invalide : vérifiez les champs");
        return false;
    }

    // Tout est correct
    alert("Livre prêt à être enregistré");
    return true; // autorise l'envoi vers ajouter_livre.php
}

```


Remarque. `isNaN(nbExp)` renvoie `true` quand la valeur *n'est pas un nombre*. En renvoyant `false`, la fonction empêche l'envoi du formulaire (grâce à `onsubmit="return validateForm()"`).

Partie I — Exercice 2 (PHP / MySQL)

1. Définition — base de données. Une *base de données* est un ensemble structuré et organisé de données, stockées de façon durable, de manière à pouvoir être facilement recherchées, consultées et mises à jour.

2. SGBD. *Système de Gestion de Base de Données* (en anglais *DBMS*). Exemples : MySQL, PostgreSQL, Oracle.

3. Côté d'exécution de PHP. Un script PHP s'exécute du **côté serveur**; le navigateur du client ne reçoit que le résultat (du HTML).

4. Éléments d'un environnement intégré de développement web. On retrouve au minimum : un **serveur web** (Apache), un **interpréteur PHP** et un **serveur de base de données** (MySQL/MariaDB). Une suite comme *WampServer* ou *XAMPP* regroupe ces trois éléments; on y ajoute en général un éditeur de code et l'outil d'administration *phpMyAdmin*.

5. Rôle des fonctions.

- `mysqli_connect()` : ouvre une connexion au serveur MySQL et sélectionne la base de données; elle renvoie un identifiant de connexion.
- `mysqli_query()` : envoie (exécute) une requête SQL sur la base via la connexion ouverte; pour un `SELECT`, elle renvoie un jeu de résultats.

6. Différence `mysqli_fetch_assoc()` / `mysqli_fetch_row()`. Les deux extraient une ligne du résultat sous forme de tableau, mais : `mysqli_fetch_assoc()` renvoie un tableau **associatif** indexé par le *nom des colonnes* (ex. `$row["titre"]`), tandis que `mysqli_fetch_row()` renvoie un tableau **indexé numériquement** à partir de 0 (ex. `$row[0]`).

7. `ajouter_livre.php` — connexion, récupération, affichage.

PHP

```
<?php
// (a) Connexion a la base BIBLIO_DB
$conn = mysqli_connect("localhost", "root", "", "BIBLIO_DB");
if (!$conn) {
    die("Echec de la connexion : " . mysqli_connect_error());
}

// (b) Recuperation des donnees envoyees par le formulaire (methode POST)
$titre      = $_POST["titre"];
$auteur     = $_POST["auteur"];
$categorie  = $_POST["categorie"];
$nbExemplaires = $_POST["nbExemplaires"];

// (c) Affichage des donnees dans un tableau HTML
echo "<table border='1' cellpadding='6'>";
echo "<tr><td>Titre</td><td>" . $titre . "</td></tr>";
echo "<tr><td>Auteur</td><td>" . $auteur . "</td></tr>";
echo "<tr><td>Categorie</td><td>" . $categorie . "</td></tr>";
echo "<tr><td>Nombre d'exemplaires</td><td>" . $nbExemplaires . "</td></tr>";
echo "</table>";
?>
```


8. Requête SQL d'insertion + code PHP. La requête SQL d'insertion est :

SQL

```
1 INSERT INTO LIVRES (titre, auteur, categorie, nbExemplaires)
2 VALUES ('Things Fall Apart', 'Chinua Achebe', 'Roman', 12);
```

En PHP, en réutilisant les données récupérées du formulaire :

PHP

```
1 <?php
2 // Construction de la requete avec les valeurs du formulaire
3 $sql = "INSERT INTO LIVRES (titre, auteur, categorie, nbExemplaires)
4     VALUES ('$titre', '$auteur', '$categorie', '$nbExemplaires')";
5
6 // Execution de la requete
7 if (mysqli_query($conn, $sql)) {
8     echo "<p>Livre ajoute avec succes !</p>";
9 } else {
10     echo "<p>Echec de l'ajout : " . mysqli_error($conn) . "</p>";
11 }
12
13 mysqli_close($conn); // fermeture de la connexion
14 ?>
```

💡 Remarque. Dans une application réelle, on protégerait la requête contre les injections SQL (requêtes préparées, `mysqli_real_escape_string`). Au Bac TI, l'écriture directe ci-dessus est acceptée.

Partie II — Exercice 1 (structures et fonctions en C)

1. Déclaration de la structure `Livre`.

C

```
1 struct Livre {
2     int code;           /* code du livre (entier) */
3     char titre[50];     /* titre (chaîne de 50 car.) */
4     char auteur[50];    /* auteur (chaîne de 50 car.) */
5     int nbExemplaires;  /* nombre d'exemplaires (entier) */
6 };
```

2. Déclaration du tableau `biblio`.

C

```
1 struct Livre biblio[200];
```

3. Deux avantages des fonctions.

- Elles évitent la répétition du code : on écrit un traitement une seule fois et on l'appelle autant de fois que nécessaire (réutilisabilité).
- Elles rendent le programme plus lisible et plus facile à corriger, en le découpant en blocs courts ayant chacun un rôle précis (modularité).

4. Fonction `rechercherLivre` (recherche séquentielle).

```

C
1 int rechercherLivre(struct Livre biblio[], int n, int code) {
2     int i;
3     for (i = 0; i < n; i++) {
4         if (biblio[i].code == code) {           /* livre trouve */
5             printf("Titre   : %s\n", biblio[i].titre);
6             printf("Auteur  : %s\n", biblio[i].auteur);
7             printf("Exempl. : %d\n", biblio[i].nbExemplaires);
8             return i;                          /* indice de la case trouvee */
9         }
10    }
11    printf("Aucun livre ne correspond a ce code\n");
12    return -1;                                /* non trouve */
13 }

```

Remarque. On parcourt le tableau case par case (de l'indice 0 à $n-1$) : c'est la *recherche séquentielle*. Dès que le code recherché est rencontré, on affiche les informations et on quitte la fonction avec `return`, sans parcourir le reste du tableau.

Partie II — Exercice 2 (programme « somme des chiffres »)

1. Rôle des lignes.

- **Ligne 4** (`int sommeChiffres(int n);`) : c'est le *prototype* (déclaration) de la fonction. Il annonce au compilateur le nom de la fonction, le type de son paramètre (`int`) et le type de sa valeur de retour (`int`), avant son utilisation dans `main`.
- **Ligne 8** (`scanf("%d", &x);`) : elle lit au clavier un entier saisi par l'utilisateur et le range dans la variable `x` (le `&` fournit l'adresse de `x`).

2. Différence entre `x` et `n`. `x` est une variable **locale à `main`** : elle contient la valeur saisie par l'utilisateur. `n` est le **paramètre formel** de la fonction `sommeChiffres` : c'est une variable locale à la fonction qui reçoit une *copie* de la valeur de `x` au moment de l'appel (passage par valeur). Modifier `n` dans la fonction ne change donc pas `x` dans `main`.

3. Trace pour `x = 472`. La fonction est appelée avec `n = 472` et `somme = 0`. La boucle `while` tourne tant que `n != 0` :

Tour	n (début)	n % 10	somme	n (fin = n / 10)
1	472	2	$0 + 2 = 2$	47
2	47	7	$2 + 7 = 9$	4
3	4	4	$9 + 4 = 13$	0

À l'issue du tour 3, `n` vaut 0 : la boucle s'arrête. La fonction renvoie `somme = 13`. Le programme affiche : « Somme des chiffres de 472 = 13 ».

4. Réécriture du `if/else` avec l'opérateur ternaire. Le bloc des lignes 9 à 15 peut se réécrire ainsi (on choisit le message avec le ternaire) :

```

C
1 char *message = (x < 0) ? "Nombre negatif non autorise !"
2                          : "OK";
3
4 if (x < 0)
5     printf("%s\n", message);
6 else {

```

```

1 s = sommeChiffres(x);
2 printf("Somme des chiffres de %d = %d\n", x, s);
3 }

```

Une forme plus compacte, lorsqu'on ne veut afficher qu'un message :

C

```

1 (x < 0) ? printf("Nombre negatif non autorise !\n")
2 : printf("Somme des chiffres de %d = %d\n", x, sommeChiffres(x));

```

5. Version récursive de `sommeChiffres`.

C

```

1 int sommeChiffres(int n) {
2     if (n == 0)                                /* cas de base */
3         return 0;
4     return (n % 10) + sommeChiffres(n / 10);    /* dernier chiffre + reste */
5 }

```

💡 **Remarque.** À chaque appel, on isole le dernier chiffre avec `n % 10`, puis on relance la fonction sur `n / 10` (le nombre privé de son dernier chiffre). La récursion s'arrête quand `n` vaut 0 : c'est le *cas de base*, indispensable pour éviter une récursion infinie.

★ À retenir

Un sujet TI se compose toujours de deux parties indépendantes notées sur 10. **Partie web** : sache écrire un formulaire HTML dans un tableau, une fonction `validateForm()` (champs non vides, `isNaN`, `return false`), une connexion `mysqli_connect` et une requête (`INSERT`, `SELECT`, `DELETE`). **Partie C** : maîtrise la déclaration d'une `struct`, un tableau de structures, la recherche séquentielle, la trace pas-à-pas d'une boucle et le passage d'une version itérative à une version récursive.

B.3 Sujet TI n° 2 — Pharmacie

Un sujet type, calqué sur l'examen

Voici un **sujet type** d'« Algorithmique et Programmation », série **TI**, construit exactement comme à l'examen : une **Partie I — Programmation web** (HTML, JavaScript, PHP, MySQL) sur **10 points** et une **Partie II — Programmation procédurale en C** (structures, fonctions, récursivité) sur **10 points**. Le fil conducteur : la gestion informatisée d'une **pharmacie**. Compose d'abord seul, dans les conditions de l'examen, puis confronte ta copie au corrigé détaillé.

OFFICE DU BACCALAURÉAT DU CAMEROUN

Examen : Baccalauréat de l'Enseignement Secondaire Technique

SUJET TYPE n° 2

Série : TI

Épreuve : Algorithmique et Programmation

Durée : **03 heures** • Coefficient : **04**

L'épreuve comporte deux parties indépendantes, notées chacune sur 10 points.

Partie I — Programmation web

(10 points)

Contexte. « *La Pharmacie de la Cité Verte, à Yaoundé, souhaite informatiser la gestion de son stock de médicaments. On vous confie le développement d'un module web modulaire qui permet d'ajouter de nouveaux médicaments au stock et de mettre à jour les quantités après chaque vente. Les médicaments sont enregistrés dans une base de données nommée `PHARMA_DB`, dont la table principale est nommée `MEDICAMENTS`. Chaque médicament est caractérisé par son **nom**, sa **catégorie**, son **prix** (en FCFA) et sa **quantité en stock**.* »

Exercice 1 — Programmation HTML et JavaScript

(03,5 pts)

- (0,5 pt) Définir : *page web dynamique*; *langage de script*.
- (1,5 pt) Écrire le code HTML affichant le formulaire (nommé `medForm`) d'ajout d'un médicament. La méthode est `POST` et le fichier de traitement est nommé `ajout_med.php`. Les champs sont, dans l'ordre, « *Nom du médicament* », « *Catégorie* », « *Prix (FCFA)* », « *Quantité* » et un bouton « *Ajouter* ». Vous présenterez le formulaire sous forme d'un tableau de **05 lignes** et **02 colonnes**. La balise `<form>` appellera, à la soumission, une fonction de validation JavaScript nommée `validateForm()`.
- (0,25 pt) Citer un attribut HTML qui rend obligatoire la saisie d'un champ de formulaire côté navigateur.
- (1,25 pt) On souhaite, au clic du bouton « *Ajouter* », vérifier la saisie : tous les champs doivent être renseignés, et de plus le **prix** ainsi que la **quantité** doivent être des nombres **strictement positifs**. Si une règle n'est pas respectée, un message d'erreur adéquat est affiché dans une boîte de dialogue et l'envoi du formulaire est annulé ; sinon le message « *Médicament prêt à être enregistré* » est affiché et le formulaire est envoyé. Écrire en JavaScript le code de la fonction `validateForm()`.

Exercice 2 — Programmation PHP et MySQL

(06,5 pts)

On s'intéresse maintenant à la base de données `PHARMA_DB` et aux scripts PHP qui y accèdent.

1. (0,25 pt) Donner la signification du sigle *SGBD*.
2. (0,25 pt) Donner la signification de *PHP*.
3. (0,5 pt) Indiquer le côté (client ou serveur) où s'exécute un script PHP, puis donner un avantage qui en découle.
4. (0,5 pt) Citer les quatre opérations de base du *CRUD* sur une table, et donner pour chacune l'instruction SQL correspondante.
5. (1,5 pt) Écrire le script PHP `ajout_med.php` qui, à la réception du formulaire de l'exercice 1, se connecte à la base `PHARMA_DB` (hôte `localhost`, utilisateur `root`, mot de passe vide), récupère les champs envoyés par la méthode `POST` et les **insère** dans la table `MEDICAMENTS`.
6. (0,75 pt) Donner le rôle des fonctions suivantes : `mysqli_connect()` ; `mysqli_query()` ; `mysqli_fetch_assoc()`.
7. (2 pts) Après chaque vente, on doit **mettre à jour le stock**. On dispose d'un formulaire qui saisit le *nom* du médicament vendu et la *quantité vendue*. Écrire un script PHP `vente.php` qui décrémente la quantité en stock du médicament concerné (`quantite = quantite - quantité vendue`) au moyen d'une requête `UPDATE`, puis **réaffiche** le contenu de la table `MEDICAMENTS` dans un **tableau HTML** (nom, catégorie, prix, quantité).
8. (0,5 pt) Écrire la requête SQL qui sélectionne le nom et la quantité des médicaments dont le stock est **inférieur à 10** (médicaments à recommander).

Partie II — Programmation procédurale en C

(10 points)

Exercice 1

(5 pts)

« Pour gérer hors-ligne le stock de la pharmacie, on écrit un programme C. Chaque médicament est caractérisé par un *code* (entier), un *nom* (chaîne), un *prix* (réel, en FCFA) et une *quantité* en stock (entier). Les médicaments sont rangés dans un tableau. On vous demande d'aider à implémenter les fonctions de gestion. »

1. (1,5 pt) Donner l'instruction C permettant de déclarer une structure d'enregistrement nommée `Medicament` regroupant les caractéristiques décrites dans le texte.
2. (0,5 pt) Donner l'instruction C permettant de déclarer un tableau `stock` de 200 médicaments.
3. (0,5 pt) Donner deux avantages de l'utilisation des fonctions dans un programme.
4. (2,5 pts) Écrire la définition de la fonction `medsEnRupture` qui parcourt le tableau et **affiche le nom de chaque médicament en rupture de stock** (c'est-à-dire dont la quantité est égale à 0), puis **retourne le nombre** de médicaments en rupture trouvés. Si aucun médicament n'est en rupture, la fonction affiche le message « *Aucune rupture de stock* ». L'en-tête imposé de la fonction est : `int medsEnRupture(struct Medicament stock[], int n)`, où `n` est le nombre de médicaments du tableau.

Exercice 2

(5 pts)

On donne le programme C ci-dessous, qui calcule le **PGCD** de deux entiers par la **méthode des soustractions successives** (les numéros de ligne 1 à 24 sont rappelés en commentaire pour pouvoir y faire référence) :

```

1  /* 1*/  /* PGCD de deux entiers par soustractions successives */
2  /* 2*/  #include <stdio.h>
3  /* 3*/  #include <stdlib.h>
4  /* 4*/  int a, b;
5  /* 5*/  int pgcd(int x, int y);
6  /* 6*/  int main() {
7  /* 7*/      printf("Entrez deux entiers strictement positifs: ");
8  /* 8*/      scanf("%d %d", &a, &b);

```

```

9  /* 9*/      if (a <= 0 || b <= 0) {
10 /*10*/      printf("Valeurs invalides !\n");
11 /*11*/      }
12 /*12*/      else {
13 /*13*/      printf("PGCD(%d, %d) = %d\n", a, b, pgcd(a, b));
14 /*14*/      }
15 /*15*/      return EXIT_SUCCESS;
16 /*16*/  }
17 /*17*/  int pgcd(int x, int y) {
18 /*18*/      if (x == y) {
19 /*19*/          return x;
20 /*20*/      }
21 /*21*/      if (x > y)
22 /*22*/          return pgcd(x - y, y);
23 /*23*/      return pgcd(x, y - x);
24 /*24*/  }

```

- Donner le rôle de :
 - (0,25 pt) la ligne 5 (`int pgcd(int x, int y);`);
 - (0,25 pt) la ligne 8 (`scanf("%d %d", &a, &b);`).
- (0,5 pt) Donner la différence fondamentale entre les variables `a` et `b` d'une part, et les paramètres `x` et `y` d'autre part.
- (1,5 pt) Exécuter pas à pas ce programme pour `a = 18` et `b = 12` (présenter la trace, de préférence dans un tableau).
- (1 pt) Réécrire les instructions de la structure `if/else` des lignes 9 à 13 en utilisant l'opérateur ternaire (`? :`).
- (1,25 pt) Réécrire la fonction `pgcd` sous sa forme **itérative** (boucle, sans récursivité).

► Corrigé du sujet TI n°2

Partie I — Exercice 1 (HTML / JavaScript)

1. Définitions.

- *Page web dynamique* : page web dont le contenu est **généré au moment de la requête** par un programme exécuté sur le serveur (PHP, etc.), souvent à partir d'une base de données. Son contenu peut donc varier d'une visite à l'autre (contrairement à une page statique au contenu figé).
- *Langage de script* : langage interprété (non compilé) servant à **automatiser des traitements** à l'intérieur d'un environnement hôte — par exemple PHP côté serveur ou JavaScript côté navigateur.

2. Formulaire HTML (`medForm`) avec tableau 5×2.

```

HTML
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Ajout d'un médicament</title>
  <script src="validation.js"></script>
</head>
<body>
  <h2>Nouveau médicament</h2>
  <form name="medForm" method="POST" action="ajout_med.php"

```

```

11     onsubmit="return validateForm()">
12     <table border="1" cellpadding="6">
13         <tr>
14             <td>Nom du médicament</td>
15             <td><input type="text" name="nom"></td>
16         </tr>
17         <tr>
18             <td>Catégorie</td>
19             <td><input type="text" name="categorie"></td>
20         </tr>
21         <tr>
22             <td>Prix (FCFA)</td>
23             <td><input type="number" name="prix"></td>
24         </tr>
25         <tr>
26             <td>Quantité</td>
27             <td><input type="number" name="quantite"></td>
28         </tr>
29         <tr>
30             <td colspan="2" align="center">
31                 <input type="submit" name="ajouter" value="Ajouter">
32             </td>
33         </tr>
34     </table>
35 </form>
36 </body>
37 </html>

```

Tableau de 5 lignes × 2 colonnes ; la validation est lancée par `onsubmit`.

💡 Remarque. Le formulaire porte le nom `medForm`, la méthode est `POST` et `action` pointe vers `ajout_med.php`. L'attribut `onsubmit="return validateForm()"` bloque l'envoi du formulaire lorsque la fonction retourne `false`.

3. Attribut rendant un champ obligatoire. L'attribut `required` (ex. `<input type="text" name="nom" required>`) impose au navigateur de refuser l'envoi si le champ est vide.

4. Fonction `validateForm()`.

JavaScript

```

1 function validateForm() {
2     // Récupération des champs du formulaire medForm
3     var f = document.forms["medForm"];
4     var nom = f["nom"].value;
5     var categorie = f["categorie"].value;
6     var prix = f["prix"].value;
7     var quantite = f["quantite"].value;
8
9     // 1) Tous les champs doivent être renseignés
10    if (nom === "" || categorie === "" || prix === "" || quantite === "") {
11        alert("Tous les champs sont obligatoires.");
12        return false; // annule l'envoi du formulaire
13    }
14
15    // 2) Le prix et la quantité doivent être strictement positifs
16    if (parseFloat(prix) <= 0 || isNaN(prix)) {

```

```

17     alert("Le prix doit être un nombre strictement positif.");
18     return false;
19 }
20 if (parseInt(quantite) <= 0 || isNaN(quantite)) {
21     alert("La quantité doit être un nombre strictement positif.");
22     return false;
23 }
24
25 // Saisie valide
26 alert("Médicament prêt à être enregistré");
27 return true; // autorise l'envoi vers ajout_med.php
28 }

```

Remarque. `return false` empêche la soumission (saisie incorrecte); `return true` laisse le formulaire partir vers `ajout_med.php`. `parseFloat` / `parseInt` convertissent le texte saisi en nombre, et `isNaN` détecte une saisie non numérique.

Partie I — Exercice 2 (PHP / MySQL)

1. Signification de SGBD. *SGBD* = **S**ystème de **G**estion de **B**ase de **D**onnées (ex. MySQL / MariaDB) : logiciel qui crée, stocke, interroge et sécurise les données d'une base.

2. Signification de PHP. *PHP* signifie **PHP : Hypertext Preprocessor** (à l'origine *Personal Home Page*).

3. Côté d'exécution de PHP + avantage. Un script PHP s'exécute **du côté serveur**. Avantage : le serveur peut se connecter à la **base de données** et lire des fichiers en toute sécurité, et le **code source PHP n'est jamais visible** par le client (le navigateur ne reçoit que le HTML produit).

4. Les quatre opérations CRUD et leur instruction SQL.

Opération	Signification	Instruction SQL
Create	Créer / insérer	INSERT INTO
Read	Lire / consulter	SELECT
Update	Mettre à jour	UPDATE
Delete	Supprimer	DELETE

5. Script `ajout_med.php` : connexion + `$_POST` + INSERT.

```

PHP
1 <?php
2 // ajout_med.php - insère un nouveau médicament dans MEDICAMENTS
3 if (isset($_POST["ajouter"])) {
4     // Connexion à la base PHARMA_DB
5     $conn = mysqli_connect("localhost", "root", "", "PHARMA_DB");
6     if (!$conn) {
7         die("Connexion echouee: " . mysqli_connect_error());
8     }
9
10    // Récupération des champs envoyés par le formulaire (méthode POST)
11    $nom      = $_POST["nom"];
12    $categorie = $_POST["categorie"];
13    $prix     = $_POST["prix"];
14    $quantite = $_POST["quantite"];

```



```

15 // Insertion dans la table MEDICAMENTS
16 $sql = "INSERT INTO MEDICAMENTS (nom, categorie, prix, quantite)
17       VALUES ('$nom', '$categorie', $prix, $quantite)";
18
19
20 if (mysqli_query($conn, $sql)) {
21     echo "<p>Médicament '$nom' ajouté avec succès.</p>";
22 } else {
23     echo "<p>Erreur : " . mysqli_error($conn) . "</p>";
24 }
25 mysqli_close($conn);
26 }
27 ?>

```

6. Rôle des fonctions.

- `mysqli_connect()` : ouvre une connexion au serveur MySQL et sélectionne la base (hôte, utilisateur, mot de passe, nom de la base). Elle renvoie un identifiant de connexion utilisé par les requêtes suivantes.
- `mysqli_query()` : exécute une requête SQL sur la connexion ouverte. Pour un `SELECT`, elle renvoie un résultat à parcourir ; pour `INSERT` / `UPDATE` / `DELETE`, elle renvoie `true` en cas de succès.
- `mysqli_fetch_assoc()` : récupère la ligne courante d'un résultat de `SELECT` sous forme de tableau associatif (clés = noms des colonnes). À chaque appel, on avance d'une ligne ; elle renvoie `NULL` quand il n'y a plus de ligne.

7. Script `vente.php` : `UPDATE` du stock + ré-affichage du tableau.

```

PHP
1 <?php
2 // vente.php - décrémente le stock après une vente puis réaffiche la table
3 $conn = mysqli_connect("localhost", "root", "", "PHARMA_DB");
4 if (!$conn) {
5     die("Connexion echouee: " . mysqli_connect_error());
6 }
7
8 // Si le formulaire de vente a été soumis
9 if (isset($_POST["vendre"])) {
10     $nom = $_POST["nom"];
11     $qteVue = (int) $_POST["qte_vendue"];
12
13     // Mise à jour du stock : quantite = quantite - quantité vendue
14     $sql = "UPDATE MEDICAMENTS
15           SET quantite = quantite - $qteVue
16           WHERE nom = '$nom'";
17
18     if (mysqli_query($conn, $sql)) {
19         echo "<p>Vente enregistrée : $qteVue x $nom.</p>";
20     } else {
21         echo "<p>Erreur : " . mysqli_error($conn) . "</p>";
22     }
23 }
24 ?>
25
26 <!-- Formulaire de saisie de la vente -->
27 <form method="POST" action="vente.php">
28     Nom du médicament :

```

```

29 <input type="text" name="nom">
30 Quantité vendue :
31 <input type="number" name="qte_vendue">
32 <input type="submit" name="vendre" value="Vendre">
33 </form>
34
35 <!-- Affichage du contenu de la table modifiée dans un tableau HTML -->
36 <table border="1" cellpadding="6">
37   <tr>
38     <th>Nom</th>
39     <th>Catégorie</th>
40     <th>Prix (FCFA)</th>
41     <th>Quantité</th>
42   </tr>
43   <?php
44     $result = mysqli_query($conn, "SELECT nom, categorie, prix, quantite FROM MEDICAMENTS"
45     );
46     while ($row = mysqli_fetch_assoc($result)) {
47       echo "<tr>";
48       echo "<td>" . $row["nom"] . "</td>";
49       echo "<td>" . $row["categorie"] . "</td>";
50       echo "<td>" . $row["prix"] . "</td>";
51       echo "<td>" . $row["quantite"] . "</td>";
52       echo "</tr>";
53     }
54     mysqli_close($conn);
55   ?>
56 </table>

```

⚠ Attention. En production, on ne concatène jamais une saisie utilisateur directement dans une requête (risque d'**injection SQL**) : on emploie des **requêtes préparées**. Ici, le code reste au niveau attendu en Terminale TI.

8. Requête SQL des médicaments à recommander (stock < 10).

SQL

```
SELECT nom, quantite FROM MEDICAMENTS WHERE quantite < 10;
```

Partie II — Exercice 1 (structures et fonctions en C)

1. Déclaration de la structure `Medicament`.

C

```

1 struct Medicament {
2     int code;           /* code du médicament (entier) */
3     char nom[50];       /* nom (chaîne de caractères) */
4     float prix;         /* prix en FCFA (réel) */
5     int quantite;       /* quantité en stock (entier) */
6 };

```

2. Déclaration d'un tableau `stock` de 200 médicaments.

C

```
| struct Medicament stock[200];
```

3. Deux avantages des fonctions.

- Elles évitent la **répétition de code** (réutilisation) : un traitement écrit une seule fois est appelé autant de fois que nécessaire ;
- Elles rendent le programme plus **lisible, structuré et facile à maintenir/déboguer** (découpage en sous-problèmes — programmation modulaire).

4. Définition de la fonction `medsEnRupture` .

C

```
1 #include <stdio.h>
2
3 int medsEnRupture(struct Medicament stock[], int n) {
4     int i;
5     int nb = 0;    /* compteur de médicaments en rupture */
6
7     for (i = 0; i < n; i++) {
8         if (stock[i].quantite == 0) {    /* rupture de stock */
9             printf("Rupture : %s\n", stock[i].nom);
10            nb++;
11        }
12    }
13
14    if (nb == 0) {
15        printf("Aucune rupture de stock\n");
16    }
17
18    return nb;    /* nombre de médicaments en rupture */
19 }
```

 **Remarque.** On parcourt tout le tableau ; pour chaque médicament dont `quantite == 0` , on affiche son nom et on incrémente le compteur `nb` . Après le parcours, si `nb` vaut 0, aucun médicament n'est en rupture : on affiche le message correspondant. Enfin la fonction retourne `nb` .

Partie II — Exercice 2 (PGCD récursif par soustractions)

1. Rôle des lignes.

- **Ligne 5** — `int pgcd(int x, int y);` : c'est le **prototype** (déclaration anticipée) de la fonction `pgcd` . Il annonce au compilateur, avant `main` , le nom de la fonction, son type de retour (`int`) et le type de ses paramètres, pour qu'elle puisse être appelée dans `main` (ligne 13) alors qu'elle n'est définie qu'ensuite (ligne 17).
- **Ligne 8** — `scanf("%d %d", &a, &b);` : **lit au clavier** deux entiers saisis par l'utilisateur et les **range dans `a` et `b`** (l'opérateur `&` fournit l'adresse de chaque variable).

2. Différence entre `a`, `b` et `x`, `y`. `a` et `b` sont des variables **globales** (déclarées ligne 4, hors de toute fonction : visibles dans tout le programme), tandis que `x` et `y` sont des variables **locales** — ce sont les **paramètres** de la fonction `pgcd` (ils n'existent que pendant l'exécution de la fonction et reçoivent une copie des valeurs passées à l'appel).

3. Trace d'exécution pour $a = 18$ et $b = 12$. L'appel `pgcd(18, 12)` déclenche une suite d'appels récursifs : à chaque étape, on soustrait le plus petit du plus grand, jusqu'à l'égalité $x == y$.

Appel	$x == y$?	$x > y$?	Appel suivant / retour
<code>pgcd(18, 12)</code>	non	oui	<code>pgcd(18-12, 12) = pgcd(6, 12)</code>
<code>pgcd(6, 12)</code>	non	non	<code>pgcd(6, 12-6) = pgcd(6, 6)</code>
<code>pgcd(6, 6)</code>	oui	—	<code>return 6</code>

La valeur `6` remonte alors à travers tous les appels. Le programme affiche : `PGCD(18, 12) = 6`.

4. Réécriture des lignes 9 à 13 avec l'opérateur ternaire.

C

```
(a <= 0 || b <= 0)
? printf("Valeurs invalides !\n")
: printf("PGCD(%d, %d) = %d\n", a, b, pgcd(a, b));
```

Remarque. L'opérateur ternaire `condition ? expr_si_vrai : expr_si_faux` remplace le `if/else`. Si `(a <= 0 || b <= 0)` est vrai, on affiche le message d'erreur ; sinon on calcule et on affiche `pgcd(a, b)`.

5. Version itérative de la fonction `pgcd`.

C

```
int pgcd(int x, int y) {
    while (x != y) {           /* tant que les deux valeurs diffèrent */
        if (x > y)
            x = x - y;         /* on retire le plus petit du plus grand */
        else
            y = y - x;
    }
    return x;                  /* quand x == y, c'est le PGCD */
}
```

Remarque. La boucle `while` reproduit la récursivité : on soustrait à répétition le plus petit du plus grand jusqu'à l'égalité. Pour $x = 18$, $y = 12$: $18, 12 \rightarrow 6, 12 \rightarrow 6, 6$, d'où le PGCD `6`, comme la version récursive.

★ À retenir

Au Bac TI, la **Partie web** attend du code *exact* : un `<form>` (méthode `POST`, action), une validation JavaScript (`return false` bloque l'envoi), `mysqli_connect + $_POST` côté serveur, une requête **CRUD** (ici `INSERT` puis `UPDATE`) et l'affichage d'un `SELECT` dans un `<table>`. La **Partie C** attend une `struct` bien typée, un parcours de tableau avec compteur, et la maîtrise de la **récursivité** (cas de base + cas récursif) face à sa version **itérative**.

B.4 Sujet TI n° 3 — Cybercafé

Office du Baccalauréat
du Cameroun
Session 2025

Série : TI
Épreuve : Algorithmique
et Programmation

Durée : 03 heures
Coef. : 04

ÉPREUVE D'ALGORITHMIQUE ET PROGRAMMATION

Remarque. Sujet type *original* OnBuch, calqué sur la structure réelle de l'épreuve d'Algorithmique et Programmation de la série TI (deux parties de 10 points : programmation web puis programmation procédurale en C). Les corrigés qui suivent la mention « Corrigé » sont proposés par OnBuch.

Contexte. Le *Cyber « LumièreNet »*, situé au quartier Briqueterie à Yaoundé, loue des postes de connexion Internet aux élèves et étudiants. Le gérant souhaite informatiser le suivi de ses sessions de connexion. Pour chaque session, on note le **numéro de poste**, le **nom du client**, l'**heure de début**, la **durée** (en minutes) et le **montant** facturé (en FCFA). Le tarif appliqué est de **300 FCFA l'heure**. On vous demande, à travers les deux parties suivantes, de l'aider à construire l'outil web puis le module en langage C.

Partie I — Programmation web

(10 points)

Exercice 1 — Page d'accueil HTML et JavaScript.

(4 points)

Le gérant veut une page d'accueil contenant un petit **simulateur de coût** qui, à partir d'une durée saisie en minutes, affiche le montant à payer (au tarif de 300 FCFA l'heure), sans recharger la page.

- 1- Définir : **page web statique**, **balise**, **événement** `onclick`. 0,75pt
- 2- Donner la différence fondamentale entre un script **JavaScript** et un script **PHP** quant au lieu de leur exécution. 0,5pt
- 3- Écrire le code **HTML** d'un formulaire contenant : un champ de saisie de durée (en minutes) d'identifiant `duree`, un bouton « Calculer » et une zone de résultat d'identifiant `resultat`. 1,5pt
- 4- Écrire la fonction **JavaScript** `calculerCout()` qui lit la durée saisie, calcule le montant ($montant = durée \div 60 \times 300$, *arrondi à l'entier*) et l'affiche, suivi de « FCFA », dans la zone `resultat`. 1,25pt

Exercice 2 — Enregistrement des sessions (PHP + MySQL).

(6 points)

Le gérant dispose d'une base de données **CYBER_DB** contenant la table **SESSIONS** décrite ci-dessous. Un formulaire permet d'enregistrer une nouvelle session ; un script PHP l'insère dans la base puis affiche la liste des sessions du jour dans un tableau HTML.

Champ	Type	Description
id	INT (clé primaire, auto)	identifiant de la session
poste	INT	numéro de poste
client	VARCHAR(50)	nom du client
debut	TIME	heure de début
duree	INT	durée en minutes
montant	INT	montant en FCFA

- 1- a) Définir : **SGBD**, **requête SQL**, **clé primaire**. 0,75pt
 b) Citer les **quatre** opérations de base du CRUD et donner, pour chacune, le mot-clé SQL correspondant. 1pt

- 2- Écrire l'instruction **SQL** qui crée la table **SESSIONS** décrite ci-dessus. 1pt
- 3- Écrire le script **PHP** (fichier `enregistrer.php`) qui :
 - a) se connecte à la base **CYBER_DB** avec `mysqli` (serveur `localhost`, utilisateur `root`, mot de passe vide); 0,75pt
 - b) récupère par `$_POST` le poste, le client et la durée envoyés par le formulaire, calcule le montant ($300 \times \text{durée} \div 60$, *arrondi*) et **insère** la session (heure de début = heure courante). 1,5pt
- 4- Écrire la requête SQL et le code PHP qui **affichent dans un tableau HTML** toutes les sessions enregistrées (colonnes : poste, client, durée, montant). 1pt

Partie II — Programmation procédurale en C

(10 points)

Exercice 1 — Structure Session et chiffre d'affaires.

(5 points)

Pour le module hors-ligne du cyber, on représente une session de connexion par un enregistrement (`struct`) regroupant le numéro de poste, le nom du client, la durée (en minutes) et le montant (en FCFA). On gère un **tableau** de telles sessions.

- 1- Définir : **structure** (enregistrement), **tableau de structures**. 0,5pt
- 2- Déclarer en langage C la structure `Session` possédant les champs : `poste` (entier), `client` (chaîne de 30 caractères), `duree` (entier) et `montant` (entier). 1pt
- 3- Déclarer un tableau `tab` de 50 sessions, puis écrire l'instruction qui affecte au champ `montant` de la **première** session la valeur `1500`. 0,75pt
- 4- Écrire la fonction `int chiffreAffaires(Session t[], int n)` qui reçoit le tableau et le nombre `n` de sessions, et **retourne** la somme des montants de toutes les sessions (le chiffre d'affaires total). 1,5pt
- 5- Écrire la fonction `int sessionPlusLongue(Session t[], int n)` qui retourne l'**indice** de la session de plus longue durée. 1,25pt

Exercice 2 — Lecture d'un programme C.

(5 points)

On donne ci-dessous un programme C **numéroté ligne par ligne**. Il affiche la table de multiplication d'un nombre saisi par l'utilisateur.

```

C
1  #include <stdio.h>
2
3  int main(void) {
4      int n, i, produit;
5      printf("Entrez un nombre : ");
6      scanf("%d", &n);
7      for (i = 1; i <= 10; i++) {
8          produit = n * i;
9          printf("%d x %d = %d\n", n, i, produit);
10     }
11     return 0;
12 }
```

- 1- Donner le **rôle** de chacune des lignes 4, 6, 7 et 8. 1pt
- 2- En supposant que l'utilisateur saisit la valeur `5`, dresser la **trace pas-à-pas** (valeurs de `i` et `produit`) pour les **trois premières** itérations de la boucle, puis donner la dernière ligne affichée. 1,5pt
- 3- Réécrire l'instruction de la ligne 8 en utilisant l'**opérateur ternaire** de sorte que `produit` reçoive le double de `n*i` si `i` est pair, et `n*i` sinon. 1pt

- 4- La boucle de la ligne 7 est **itérative** (`for`). Écrire une fonction **réursive** `void table(int n, int i)` qui affiche la même table de multiplication (de `i=1` à `i=10`). **1,5pt**

► Corrigé du sujet TI n°3

Partie I — Programmation web

Ex.1 — 1) Définitions.

- *Page web statique* : page dont le contenu est fixe ; il est livré tel quel au navigateur, sans traitement côté serveur ni modification dynamique.
- *Balise* : marqueur du langage HTML, écrit entre chevrons (`<...>`), qui délimite et qualifie un élément de la page (ex. `<p>`, `<input>`).
- *Événement onclick* : événement déclenché par un **clic** de l'utilisateur sur un élément ; il permet d'associer à ce clic l'appel d'une fonction JavaScript.

Ex.1 — 2) **JavaScript vs PHP**. Un script **JavaScript** s'exécute **côté client** (dans le navigateur du visiteur), tandis qu'un script **PHP** s'exécute **côté serveur** (le serveur interprète le code et n'envoie au navigateur que le HTML produit).

Ex.1 — 3) Formulaire HTML.

HTML

```

1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4   <meta charset="utf-8">
5   <title>Cyber LumièreNet - Simulateur</title>
6 </head>
7 <body>
8   <h1>Cyber LumièreNet</h1>
9   <p>Durée (en minutes) : <input type="text" id="duree"></p>
10  <button onclick="calculerCout()">Calculer</button>
11  <p id="resultat"></p>
12
13  <script src="script.js"></script>
14 </body>
15 </html>

```

Ex.1 — 4) Fonction JavaScript `calculerCout` .

JavaScript

```

1 function calculerCout() {
2   var minutes = parseInt(document.getElementById("duree").value);
3   var montant = Math.round(minutes / 60 * 300); // 300 FCFA par heure
4   document.getElementById("resultat").innerHTML = montant + " FCFA";
5 }

```

💡 **Remarque.** `parseInt` convertit le texte saisi en entier ; `Math.round` arrondit le montant. Pour 90 minutes : $90/60 \times 300 = 450$, donc « 450 FCFA ».

Ex.2 — 1.a) Définitions.

- *SGBD* (Système de Gestion de Base de Données) : logiciel qui permet de créer, stocker, organiser et manipuler les données d'une base (ex. MySQL).
- *Requête SQL* : instruction écrite en langage SQL adressée au SGBD pour agir sur les données (créer, lire, modifier, supprimer).

- *Clé primaire* : champ (ou ensemble de champs) qui identifie de manière **unique** chaque enregistrement d'une table.

Ex.2 — 1.b) Opérations CRUD et mots-clés SQL.

Opération	Signification	Mot-clé SQL
Create	Créer / insérer	INSERT
Read	Lire / consulter	SELECT
Update	Modifier	UPDATE
Delete	Supprimer	DELETE

Ex.2 — 2) Création de la table SESSIONS.

SQL

```
CREATE TABLE SESSIONS (
    id      INT AUTO_INCREMENT PRIMARY KEY,
    poste   INT,
    client  VARCHAR(50),
    debut   TIME,
    duree   INT,
    montant INT
);
```

Ex.2 — 3) Script enregistrer.php : connexion, \$_POST et insertion.

PHP

```
<?php
// a) Connexion à la base CYBER_DB avec mysqli
$cnx = new mysqli("localhost", "root", "", "CYBER_DB");
if ($cnx->connect_error) {
    die("Échec de connexion : " . $cnx->connect_error);
}

// b) Récupération des données du formulaire (méthode POST)
$poste = $_POST["poste"];
$client = $_POST["client"];
$duree = $_POST["duree"];

// Calcul du montant : 300 FCFA / heure, arrondi
$montant = round(300 * $duree / 60);

// Heure de début = heure courante
$debut = date("H:i:s");

// Insertion de la session dans la table
$sql = "INSERT INTO SESSIONS (poste, client, debut, duree, montant)
VALUES ('$poste', '$client', '$debut', '$duree', '$montant')";

if ($cnx->query($sql)) {
    echo "<p>Session enregistrée : $client, $montant FCFA.</p>";
} else {
    echo "<p>Erreur : " . $cnx->error . "</p>";
}
?>
```


💡 **Remarque.** Le formulaire d'appel (fichier `form.html`) enverrait ces données ainsi :
`<form method="post" action="enregistrer.php">` avec trois champs nommés `poste`, `client`
 et `duree`, puis un bouton de soumission.

Ex.2 — 4) Requête de lecture et affichage dans un tableau HTML. On ajoute, à la suite du script précédent, la lecture de toutes les sessions :

PHP

```
<?php
// Requête de lecture
$res = $cnx->query("SELECT poste, client, duree, montant FROM SESSIONS");

echo "<table border='1'>";
echo "<tr><th>Poste</th><th>Client</th><th>Durée</th><th>Montant</th></tr>";

// Parcours des lignes et affichage
while ($ligne = $res->fetch_assoc()) {
    echo "<tr>";
    echo "<td>" . $ligne["poste"] . "</td>";
    echo "<td>" . $ligne["client"] . "</td>";
    echo "<td>" . $ligne["duree"] . " min</td>";
    echo "<td>" . $ligne["montant"] . " FCFA</td>";
    echo "</tr>";
}
echo "</table>";

$cnx->close();
?>
```

Partie II — Programmation procédurale en C

Ex.1 — 1) Définitions.

- *Structure (enregistrement)* : type de données défini par le programmeur qui regroupe sous un même nom plusieurs champs de types éventuellement différents.
- *Tableau de structures* : tableau dont chaque case est une structure; il permet de stocker une collection d'enregistrements de même type (ici, plusieurs sessions).

Ex.1 — 2) Déclaration de la structure `Session`.

C

```
typedef struct {
    int  poste;
    char client[30];
    int  duree;    // durée en minutes
    int  montant;  // montant en FCFA
} Session;
```

Ex.1 — 3) Tableau de 50 sessions et affectation.

C

```
Session tab[50];
tab[0].montant = 1500;    // première session : indice 0
```

Ex.1 — 4) Fonction `chiffreAffaires`.

C

```
1 int chiffreAffaires(Session t[], int n) {
2     int i, total = 0;
3     for (i = 0; i < n; i++) {
4         total = total + t[i].montant;    // cumul des montants
5     }
6     return total;
7 }
```

Ex.1 — 5) Fonction `sessionPlusLongue`. Elle retourne l'indice de la session dont la durée est la plus grande.

C

```
1 int sessionPlusLongue(Session t[], int n) {
2     int i, indiceMax = 0;
3     for (i = 1; i < n; i++) {
4         if (t[i].duree > t[indiceMax].duree) {
5             indiceMax = i;
6         }
7     }
8     return indiceMax;
9 }
```

💡 **Remarque.** On suppose $n \geq 1$. On initialise `indiceMax` à 0 (première session) puis on compare chaque durée à celle de la meilleure session trouvée jusque-là.

Ex.2 — 1) Rôle des lignes.

- **Ligne 4 :** déclaration des trois variables entières `n` (le nombre), `i` (le compteur de boucle) et `produit` (le résultat).
- **Ligne 6 :** lecture au clavier de la valeur de `n` (`scanf`), rangée à l'adresse de `n` grâce à l'opérateur `&`.
- **Ligne 7 :** début de la boucle `for` qui fait varier `i` de 1 à 10 inclus (10 itérations).
- **Ligne 8 :** calcul du produit `n * i` et affectation à la variable `produit`.

Ex.2 — 2) Trace pas-à-pas pour `n = 5`.

Itération	i	produit	Affichage
1	1	5	5 x 1 = 5
2	2	10	5 x 2 = 10
3	3	15	5 x 3 = 15

La boucle se poursuit jusqu'à `i = 10` ; la dernière ligne affichée est `5 x 10 = 50`.

Ex.2 — 3) Ligne 8 avec l'opérateur ternaire. Le double si `i` est pair, sinon `n*i` :

C

```
1 produit = (i % 2 == 0) ? 2 * n * i : n * i;
```

💡 **Remarque.** La condition `i % 2 == 0` teste la parité de `i` : si le reste de la division par 2 est nul, `i` est pair et `produit` reçoit `2*n*i` ; sinon il reçoit `n*i`.

Ex.2 — 4) Version récursive `table`.

C

```

1 void table(int n, int i) {
2     if (i > 10) {
3         return;                // cas d'arrêt
4     }
5     printf("%d x %d = %d\n", n, i, n * i);
6     table(n, i + 1);           // appel récursif
7 }

```

On l'appelle par `table(n, 1);` pour obtenir la table complète de `n`, identique à la version itérative.

★ À retenir

Sujet en deux moitiés de 10 points : (I) un **simulateur** HTML/JavaScript (événement `onclick`, lecture d'un champ, calcul, affichage) puis un module **PHP/MySQL** (connexion `mysqli`, récupération par `$_POST`, `INSERT`, puis `SELECT` affiché dans un tableau HTML); (II) un **tableau de struct** (Session) avec une fonction de **chiffre d'affaires** et de **recherche du maximum**, puis la **lecture** d'un programme C (rôles, trace, opérateur **ternaire**, passage **itératif**→**récursif**).

B.5 Sujet TI n° 4 — Gestion des notes

RÉPUBLIQUE DU CAMEROUN
Paix – Travail – Patrie
OFFICE DU BACCALAURÉAT

Série : TI
Épreuve : Algorithmique et Programmation
Durée : 03 heures Coef. : 04

EXAMEN : BACCALAURÉAT TECHNIQUE SESSION : JUIN

Remarque. Sujet type **original** OnBuch, calqué sur la structure réelle de l'épreuve d'Algorithmique et Programmation du Baccalauréat série TI. Les deux parties sont **obligatoires** et notées sur 10 points chacune. Les corrigés placés après la mention « Corrigé » ne font pas partie du sujet à composer.

Contexte. Le Lycée Technique de Bafoussam souhaite informatiser la gestion des notes de ses élèves. L'administration dispose d'une base de données **SCOLA_DB** contenant une table **ELEVES** décrite ci-dessous. Les notes sont sur 20 ; un élève est déclaré **ADMIS** lorsque sa moyenne est supérieure ou égale à 10. Vous êtes recruté(e) comme développeur pour réaliser l'application web et les modules de traitement en langage C.

Champ	Type	Clé	Description
matricule	VARCHAR(10)	primaire	matricule de l'élève
nom	VARCHAR(50)		nom et prénom
classe	VARCHAR(15)		ex. : « Tle TI »
moyenne	DECIMAL(4,2)		moyenne sur 20

Partie I — Programmation web

(10 points)

Exercice 1 — Saisie et validation côté client (HTML / JavaScript)

(4 points)

- Définir : **site web dynamique, langage de script côté client.** **1pt**
- On souhaite une page `saisie.html` contenant un formulaire de saisie d'une note. Le formulaire, présenté **sous forme de tableau** (balise `<table>`), comporte trois champs : le *matricule* de l'élève, le *nom* de la matière et la *note* obtenue ; il se termine par un bouton « Valider ». Le formulaire envoie ses données à `traitement.php` par la méthode `POST`. Écrire le code HTML de ce formulaire. **1,5pt**
- Écrire la fonction JavaScript `validateForm()` appelée à la soumission du formulaire. Elle doit vérifier que la note saisie est un nombre **compris entre 0 et 20**. Si la note est invalide, la fonction affiche un message avec `alert()` et renvoie `false` (la soumission est annulée) ; sinon elle renvoie `true`. **1,5pt**

Exercice 2 — Traitement serveur et base de données (PHP / MySQL)

(6 points)

- Questions de cours : **1,5pt**
 - Citer deux différences entre le langage PHP et le langage JavaScript. **0,5pt**
 - À quoi sert la super-globale `$_POST` ? **0,5pt**
 - Donner le rôle de la fonction `mysqli_connect()`. **0,5pt**
- Le script `traitement.php` reçoit les données du formulaire. Écrire le code PHP qui : (a) se connecte au serveur MySQL (hôte `localhost`, utilisateur `root`, mot de passe vide) et à la base `SCOLA_DB` avec `mysqli_connect()` ; (b) récupère le matricule et la note transmis par `$_POST` ; (c) insère un nouvel élève dans la table `ELEVES` grâce à une requête **INSERT** (on prendra `classe = "Tle TI"`). **2pt**

- 3- Écrire le script `admis.php` qui sélectionne dans la table `ELEVES` **tous les élèves admis** (moyenne supérieure ou égale à 10) à l'aide d'une requête `SELECT ...WHERE`, puis affiche le résultat dans un **tableau HTML** (colonnes : matricule, nom, moyenne, mention). **2pt**
- 4- Écrire la requête **UPDATE** qui corrige la moyenne de l'élève de matricule `"BAF015"` en la fixant à `13.50`. **0,5pt**

Partie II — Programmation procédurale en C

(10 points)

Exercice 1 — Structures et fonctions

(5 points)

On modélise un élève par la structure suivante :

```
C
struct Eleve {
    char nom[50];
    char classe[15];
    float moyenne;    // sur 20
};
```

- 1- Donner deux **avantages** de l'utilisation des fonctions en programmation. **1pt**
- 2- Déclarer un tableau `classe` de 30 éléments de type `struct Eleve`, puis écrire les instructions qui lisent au clavier la moyenne de la classe pour l'élève d'indice 0 (`scanf`). **1pt**
- 3- Écrire la fonction `float moyenneClasse(struct Eleve t[], int n)` qui reçoit le tableau et le nombre `n` d'élèves, et renvoie la **moyenne générale** de la classe (moyenne des moyennes). **1,5pt**
- 4- Écrire la fonction `void afficherMajor(struct Eleve t[], int n)` qui affiche le nom et la moyenne du **major** de la classe (l'élève ayant la plus forte moyenne). **1,5pt**

Exercice 2 — Lecture, trace et réécriture d'un programme C

(5 points)

On donne le programme C suivant, dont les lignes sont numérotées. Il trie un tableau de moyennes par ordre croissant selon le principe du **tri par sélection**.

```
C
1  #include <stdio.h>
2  void trier(float t[], int n) {
3      int i, j, imin;
4      float tmp;
5      for (i = 0; i < n - 1; i++) {
6          imin = i;
7          for (j = i + 1; j < n; j++) {
8              if (t[j] < t[imin])
9                  imin = j;
10         }
11         tmp = t[i];
12         t[i] = t[imin];
13         t[imin] = tmp;
14     }
15 }
16 int main() {
17     float notes[5] = {12, 8, 15, 6, 10};
18     int k;
19     trier(notes, 5);
20     for (k = 0; k < 5; k++)
21         printf("%.1f ", notes[k]);
22     return 0;
23 }
```

- 1- Donner le rôle des lignes 6, 8–9 et 11–13. 1,5pt
- 2- Quelle est la différence entre les variables `imin` et `tmp` (leur rôle dans l'algorithme) ? 1pt
- 3- Effectuer la **trace** du contenu du tableau `notes` après chaque tour complet de la boucle externe (ligne 5), puis donner l'**affichage** produit par le programme. 1,5pt
- 4- Réécrire l'instruction conditionnelle des lignes 8–9 en utilisant l'**opérateur ternaire**. 1pt

► Corrigé du sujet TI n° 4

Partie I — Programmation web

► Corrigé Exercice 1 (HTML / JavaScript)

1- *Site web dynamique* : site dont les pages sont générées au moment de la requête par un programme côté serveur (PHP), souvent à partir d'une base de données ; le contenu peut changer d'un visiteur à l'autre.

Langage de script côté client : langage (ex. JavaScript) interprété par le **navigateur** du visiteur, qui agit sur la page sans solliciter le serveur (validation de formulaire, animations).

- 2- Formulaire HTML présenté en tableau :

```

HTML
1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4   <meta charset="utf-8">
5   <title>Saisie d'une note</title>
6   <script src="valid.js"></script>
7 </head>
8 <body>
9   <h2>Lycée Technique de Bafoussam - Saisie d'une note</h2>
10  <form action="traitement.php" method="post" onsubmit="return validateForm()">
11    <table border="1" cellpadding="6">
12      <tr>
13        <td>Matricule :</td>
14        <td><input type="text" name="matricule" id="matricule"></td>
15      </tr>
16      <tr>
17        <td>Matiere :</td>
18        <td><input type="text" name="matiere" id="matiere"></td>
19      </tr>
20      <tr>
21        <td>Note (/20) :</td>
22        <td><input type="text" name="note" id="note"></td>
23      </tr>
24      <tr>
25        <td colspan="2" align="center">
26          <input type="submit" value="Valider">
27        </td>
28      </tr>
29    </table>
30  </form>
31 </body>
32 </html>

```

- 3- Fonction de validation (fichier `valid.js`) :

JavaScript

```

1 function validateForm() {
2     // On recupere la valeur saisie et on la convertit en nombre
3     var note = parseFloat(document.getElementById("note").value);
4     // Verification : nombre valide ET compris entre 0 et 20
5     if (isNaN(note) || note < 0 || note > 20) {
6         alert("La note doit etre un nombre compris entre 0 et 20 !");
7         return false;    // annule l'envoi du formulaire
8     }
9     return true;        // la note est valide : on soumet
10 }

```

► Corrigé Exercice 2 (PHP / MySQL)

1- Cours.

- PHP s'exécute **côté serveur** (le client ne voit jamais le code), tandis que JavaScript s'exécute **côté client** dans le navigateur. PHP peut accéder à une base de données ; JavaScript agit sur la page affichée.
- `$_POST` est un tableau associatif qui contient les données envoyées par un formulaire HTML avec la méthode `post` (on y accède par le nom des champs).
- `mysqli_connect()` établit la **connexion** entre le script PHP et le serveur de base de données MySQL ; elle renvoie un identifiant de connexion.

2- Script `traitement.php` :

PHP

```

1 <?php
2     // (a) Connexion au serveur MySQL et a la base SCOLA_DB
3     $conn = mysqli_connect("localhost", "root", "", "SCOLA_DB");
4     if (!$conn) {
5         die("Echec de connexion : " . mysqli_connect_error());
6     }
7
8     // (b) Recuperation des donnees du formulaire
9     $matricule = $_POST["matricule"];
10    $note      = $_POST["note"];
11
12    // (c) Insertion du nouvel eleve dans la table ELEVES
13    $sql = "INSERT INTO ELEVES (matricule, nom, classe, moyenne)
14           VALUES ('$matricule', '', 'Tle TI', $note)";
15    if (mysqli_query($conn, $sql)) {
16        echo "Eleve enregistre avec succes.";
17    } else {
18        echo "Erreur : " . mysqli_error($conn);
19    }
20    mysqli_close($conn);
21 ?>

```

3- Script `admis.php` (sélection des admis + tableau HTML) :

PHP

```

1 <?php
2     $conn = mysqli_connect("localhost", "root", "", "SCOLA_DB");
3     // Selection de tous les eleves admis (moyenne >= 10)
4     $sql = "SELECT matricule, nom, moyenne FROM ELEVES WHERE moyenne >= 10";

```

```

1 $res = mysqli_query($conn, $sql);
2
3 echo "<table border='1' cellpadding='6'>";
4 echo "<tr><th>Matricule</th><th>Nom</th><th>Moyenne</th><th>Mention</th></tr>";
5
6 // Parcours ligne par ligne du resultat
7 while ($eleve = mysqli_fetch_assoc($res)) {
8     $moy = $eleve["moyenne"];
9     // Determination de la mention
10    if ($moy >= 16)      $mention = "Tres Bien";
11    elseif ($moy >= 14) $mention = "Bien";
12    elseif ($moy >= 12) $mention = "Assez Bien";
13    else                $mention = "Passable";
14
15    echo "<tr>";
16    echo "<td>" . $eleve["matricule"] . "</td>";
17    echo "<td>" . $eleve["nom"] . "</td>";
18    echo "<td>" . $moy . "</td>";
19    echo "<td>" . $mention . "</td>";
20    echo "</tr>";
21 }
22 echo "</table>";
23 mysqli_close($conn);
24 ?>

```

4- Requête de mise à jour :

SQL

```
UPDATE ELEVES SET moyenne = 13.50 WHERE matricule = 'BAF015';
```

Partie II — Programmation procédurale en C

► Corrigé Exercice 1 (Structures et fonctions)

1- *Avantages des fonctions* : (i) elles évitent la répétition du code (une même tâche est écrite une seule fois puis appelée autant de fois que nécessaire); (ii) elles rendent le programme plus **lisible** et plus facile à corriger (découpage en sous-problèmes, réutilisation).

2- Déclaration du tableau et lecture d'une moyenne :

C

```

1 struct Eleve classe[30];
2 printf("Moyenne de l'eleve 0 : ");
3 scanf("%f", &classe[0].moyenne);

```

3- Moyenne générale de la classe :

C

```

1 float moyenneClasse(struct Eleve t[], int n) {
2     int i;
3     float somme = 0;
4     for (i = 0; i < n; i++)
5         somme = somme + t[i].moyenne;    // on cumule les moyennes
6     return somme / n;                  // moyenne des moyennes
7 }

```


4- Affichage du major :

```
C
1 void afficherMajor(struct Eleve t[], int n) {
2     int i, imax = 0;
3     for (i = 1; i < n; i++) {
4         if (t[i].moyenne > t[imax].moyenne)
5             imax = i;                // on retient l'indice du maximum
6     }
7     printf("Major : %s (%.2f / 20)\n", t[imax].nom, t[imax].moyenne);
8 }
```

► Corrigé Exercice 2 (Lecture et trace)

1- Rôle des lignes :

- **ligne 6** (`imin = i;`) : on suppose au départ que le plus petit élément du sous-tableau non trié se trouve à la position `i`.
- **lignes 8–9** : on compare chaque élément suivant à l'élément le plus petit trouvé ; si `t[j]` est plus petit, on mémorise son indice dans `imin`.
- **lignes 11–13** : on **échange** l'élément de position `i` avec le plus petit élément trouvé (`t[imin]`), à l'aide de la variable `tmp`.

2- `imin` est un **indice** (entier) : il repère la position du minimum courant. `tmp` est une variable **temporaire** (un flottant) qui sert uniquement à conserver une valeur le temps d'effectuer l'échange de deux cases.

3- Trace. Tableau initial : [12, 8, 15, 6, 10] .

Après le tour i =	Contenu du tableau
0	[6, 8, 15, 12, 10]
1	[6, 8, 15, 12, 10]
2	[6, 8, 10, 12, 15]
3	[6, 8, 10, 12, 15]

Affichage produit : 6.0 8.0 10.0 12.0 15.0

4- Réécriture des lignes 8–9 avec l'opérateur ternaire :

```
C
1 imin = (t[j] < t[imin]) ? j : imin;
```

★ À retenir

Côté **web**, on valide d'abord la saisie en **JavaScript** (note entre 0 et 20, `alert + return false`), puis on traite et on stocke côté **serveur** en **PHP/MySQL** (`mysqli_connect`, `$_POST`, `SELECT ...WHERE moyenne >= 10`). Côté **C**, une `struct` regroupe les informations d'un élève et les **fonctions** (moyenne, major, tri par sélection) découpent le traitement en sous-problèmes réutilisables.

B.6 Sujet TI n° 5 — Boutique en ligne

Office du Baccalauréat
du Cameroun
Examen : **Baccalauréat**

Série : TI
Épreuve : Algorithmique
et Programmation

Durée : 03 heures
Coef. : 04
Session : Juin

ÉPREUVE D'ALGORITHMIQUE ET PROGRAMMATION

Le candidat traitera obligatoirement les deux parties. La calculatrice n'est pas autorisée.

Contexte. La coopérative « **BoutikCM** », basée à Douala, souhaite vendre ses articles en ligne et gérer ses commandes par informatique. Elle utilise une base de données nommée **SHOP_DB** contenant une table **ARTICLES** qui décrit chaque produit du catalogue :

reference	designation	prix_unitaire	quantite
ART01	Sac de riz 5 kg	4500	20
ART02	Bidon d'huile 1 L	1200	35
ART03	Savon de Ngaoundéré	350	80
ART04	Cahier 200 pages	600	50

Les prix sont exprimés en **francs CFA (FCFA)**. On vous demande d'écrire une partie du logiciel de cette boutique.

Partie I — Programmation web

(10 points)

Exercice 1 — Page d'accueil HTML et JavaScript.

(4 points)

La page d'accueil du site présente un petit *simulateur de commande* : le client saisit le prix unitaire d'un article et la quantité voulue, puis clique sur un bouton « **Calculer le montant** » pour afficher le montant total à payer.

- 1- Définir : **balise**, **événement**, **fonction** (en JavaScript). **0,75pt**
- 2- Écrire le code **HTML** d'un formulaire contenant deux zones de saisie (`id="pu"` et `id="qte"`), un bouton « Calculer le montant » et une zone de texte `id="total"` qui recevra le résultat. **1,5pt**
- 3- Écrire en **JavaScript** la fonction `montant()` qui lit les deux valeurs saisies, calcule le produit `prix × quantité` et affiche le résultat dans la zone `id="total"` . **1,25pt**
- 4- Écrire l'instruction (attribut `onclick`) qui appelle la fonction `montant()` lorsqu'on clique sur le bouton. **0,5pt**

Exercice 2 — Gestion de la table ARTICLES en PHP/MySQL.

(6 points)

L'administrateur de BoutikCM gère son catalogue depuis une page PHP connectée à la base **SHOP_DB** (serveur `localhost` , utilisateur `root` , mot de passe vide).

- 1- a) Définir : **base de données**, **SGBD**. **0,5pt**
b) Citer le rôle des langages **PHP** et **SQL** dans cette application. **0,5pt**
c) Donner la requête **SQL** qui crée la table **ARTICLES** (champ `reference` clé primaire de type texte, `designation` texte, `prix_unitaire` et `quantite` entiers). **1pt**
- 2- Écrire le code PHP qui **se connecte** à la base **SHOP_DB** avec `mysqli_connect` , puis affiche **tous** les articles dans un **tableau HTML** de quatre colonnes (référence, désignation, prix unitaire, quantité). **2pts**
- 3- L'administrateur saisit dans un formulaire le **nom** (la désignation) d'un article à retirer du catalogue. À la réception (méthode `POST` , champ `nom_article`), on souhaite **supprimer** de la table **ARTICLES** la ligne dont la désignation correspond, puis **ré-afficher** la table mise à jour.
a) Écrire le formulaire HTML qui envoie le nom de l'article par la méthode `POST`. **0,5pt**

- b) Écrire le code PHP qui récupère le nom saisi via `$_POST`, exécute la requête `DELETE ... WHERE` correspondante, affiche un message de confirmation, puis ré-affiche le contenu de la table `ARTICLES`. **1,5pt**

Partie II — Programmation procédurale en C

(10 points)

Exercice 1 — Structure Article et montant d'une commande.

(5 points)

On modélise un article du catalogue par une structure C nommée `Article`, possédant les champs `reference` (chaîne), `designation` (chaîne), `prix` (entier) et `quantite` (entier, la quantité commandée).

- 1- Définir : **structure (enregistrement)**, **tableau de structures**. **0,75pt**
- 2- Déclarer en C la structure `Article` avec ses quatre champs, puis déclarer un tableau `commande` de 4 articles. **1,25pt**
- 3- On souhaite calculer le **montant total** d'une commande, c'est-à-dire la somme des $prix \times quantite$ de tous ses articles. Écrire la fonction imposée :

```
int montantTotal(Article cmd[], int n)
```

qui reçoit le tableau `cmd` de `n` articles et **retourne** le montant total en FCFA.

2pts

- 4- Écrire le `main` qui remplit le tableau `commande` avec les quatre articles du contexte (mêmes quantités que le tableau de l'énoncé), appelle `montantTotal` et affiche le résultat. **1pt**

Exercice 2 — Étude d'un programme C.

(5 points)

On donne le programme C suivant, dont les lignes sont numérotées :

```
C
1  #include <stdio.h>
2
3  int mystere(int t[], int n) {
4      int m = t[0];
5      for (int i = 1; i < n; i++) {
6          if (t[i] > m)
7              m = t[i];
8      }
9      return m;
10 }
11
12 int main(void) {
13     int stock[5] = {20, 35, 80, 50, 12};
14     int r = mystere(stock, 5);
15     printf("Resultat = %d\n", r);
16     return 0;
17 }
```

- 1- Indiquer le rôle des lignes **4**, **6** et **9**, puis donner en une phrase le **rôle global** de la fonction `mystere`. **1,25pt**
- 2- Faire la **trace pas-à-pas** de l'exécution de `mystere(stock, 5)` : donner, pour chaque valeur de `i`, la valeur de `m`. Quelle valeur s'affiche à la ligne 15 ? **1,5pt**
- 3- Réécrire la **ligne 6-7** (le `if`) en utilisant l'**opérateur ternaire** `?:` en une seule instruction. **1pt**
- 4- La fonction `mystere` est **itérative**. En écrire une version **récursive** `int maxRec(int t[], int n)` qui retourne le même résultat. **1,25pt**

► Corrigé du sujet TI n°5

Partie I — Programmation web

Exercice 1.

1- Définitions.

- *Balise* : élément du langage HTML, écrit entre chevrons (ex. `<p>`), qui délimite et structure un contenu de la page (la plupart vont par paire : ouvrante `<p>` et fermante `</p>`).
- *Événement* : action de l'utilisateur ou du navigateur (clic, survol, chargement...) à laquelle une page peut réagir grâce à un gestionnaire comme `onclick`.
- *Fonction* (JavaScript) : bloc d'instructions nommé, défini par le mot-clé `function`, que l'on peut appeler à volonté pour réaliser un traitement et éventuellement retourner une valeur.

2- Formulaire HTML.

HTML

```

1 <!DOCTYPE html>
2 <html lang="fr">
3 <head><meta charset="utf-8"><title>BoutikCM - Simulateur</title></head>
4 <body>
5   <h2>Simulateur de commande</h2>
6   <p>Prix unitaire (FCFA) : <input type="text" id="pu"></p>
7   <p>Quantite : <input type="text" id="qte"></p>
8   <p>
9     <input type="button" value="Calculer le montant" onclick="montant()">
10  </p>
11  <p>Montant total : <input type="text" id="total"></p>
12  <script src="boutik.js"></script>
13 </body>
14 </html>

```

3- Fonction `montant()` en JavaScript.

JavaScript

```

1 function montant() {
2   var pu = parseFloat(document.getElementById("pu").value);
3   var qte = parseFloat(document.getElementById("qte").value);
4   var total = pu * qte; // produit
5   document.getElementById("total").value = total; // affichage
6 }

```

4- Appel au clic. L'attribut placé sur le bouton est :

HTML

```

<input type="button" value="Calculer le montant" onclick="montant()">

```

Le clic déclenche `onclick="montant()"`, qui exécute la fonction et écrit le montant dans la zone `id="total"`.

Exercice 2.

1.a) Définitions.

- *Base de données* : ensemble structuré de données organisées (ici en tables) de façon à être facilement consultées, mises à jour et partagées.
- *SGBD* (Système de Gestion de Base de Données) : logiciel qui permet de créer, manipuler et administrer une base de données (ex. MySQL).

1.b) Rôle de PHP et SQL.

- **PHP** : langage côté serveur qui produit la page web dynamique et dialogue avec la base (connexion, envoi des requêtes, affichage des résultats).
- **SQL** : langage de requêtes utilisé pour créer la table et manipuler les données (`SELECT` , `INSERT` , `DELETE` ...).

1.c) Création de la table.

SQL

```
CREATE TABLE ARTICLES (
    reference    VARCHAR(10) PRIMARY KEY,
    designation   VARCHAR(50),
    prix_unitaire INT,
    quantite      INT
);
```

2- Connexion et affichage de tous les articles.

PHP

```
<?php
// Connexion a la base SHOP_DB
$cnx = mysqli_connect("localhost", "root", "", "SHOP_DB");
if (!$cnx) {
    die("Connexion impossible : " . mysqli_connect_error());
}

// Lecture de tous les articles
$res = mysqli_query($cnx, "SELECT * FROM ARTICLES");

echo "<table border='1'>";
echo "<tr><th>Reference</th><th>Designation</th>";
echo "<th>Prix unitaire</th><th>Quantite</th></tr>";

while ($ligne = mysqli_fetch_assoc($res)) {
    echo "<tr>";
    echo "<td>" . $ligne["reference"] . "</td>";
    echo "<td>" . $ligne["designation"] . "</td>";
    echo "<td>" . $ligne["prix_unitaire"] . " FCFA</td>";
    echo "<td>" . $ligne["quantite"] . "</td>";
    echo "</tr>";
}
echo "</table>";

mysqli_close($cnx);
?>
```

3.a) Formulaire HTML (méthode POST).

HTML

```
<form action="supprimer.php" method="post">
    <label>Nom de l'article a supprimer :</label>
    <input type="text" name="nom_article">
    <input type="submit" value="Supprimer">
</form>
```

3.b) Code PHP : suppression puis ré-affichage.

PHP

```

1 <?php
2 // Connexion
3 $cnx = mysqli_connect("localhost", "root", "", "SHOP_DB");
4 if (!$cnx) {
5     die("Connexion impossible : " . mysqli_connect_error());
6 }
7
8 // 1) Recuperation du nom saisi et suppression
9 $nom = $_POST["nom_article"];
10 $sql = "DELETE FROM ARTICLES WHERE designation = '$nom'";
11
12 if (mysqli_query($cnx, $sql)) {
13     echo "<p>L'article \"$nom\" a ete supprime.</p>";
14 } else {
15     echo "<p>Erreur : " . mysqli_error($cnx) . "</p>";
16 }
17
18 // 2) Re-affichage de la table mise a jour
19 $res = mysqli_query($cnx, "SELECT * FROM ARTICLES");
20 echo "<table border='1'>";
21 echo "<tr><th>Reference</th><th>Designation</th>";
22 echo "<th>Prix unitaire</th><th>Quantite</th></tr>";
23 while ($ligne = mysqli_fetch_assoc($res)) {
24     echo "<tr>";
25     echo "<td>" . $ligne["reference"] . "</td>";
26     echo "<td>" . $ligne["designation"] . "</td>";
27     echo "<td>" . $ligne["prix_unitaire"] . " FCFA</td>";
28     echo "<td>" . $ligne["quantite"] . "</td>";
29     echo "</tr>";
30 }
31 echo "</table>";
32
33 mysqli_close($cnx);
34 ?>

```

 **Remarque.** La condition `WHERE designation = '$nom'` est indispensable : sans `WHERE`, la requête `DELETE FROM ARTICLES` viderait toute la table. Si plusieurs articles portent la même désignation, ils sont tous supprimés.

Partie II — Programmation procédurale en C

Exercice 1.

1- Définitions.

- *Structure (enregistrement)* : type de données composé regroupant sous un seul nom plusieurs champs de types éventuellement différents (ex. une référence, un prix...).
- *Tableau de structures* : tableau dont chaque case est une structure; il permet de stocker une collection d'enregistrements de même type (ici plusieurs articles).

2- Déclaration de la structure et du tableau.

C

```

1 typedef struct {
2     char reference[10];
3     char designation[50];

```

```

4     int  prix;
5     int  quantite;
6 } Article;

7
8 Article commande[4];

```

3- Fonction montantTotal imposée.

C

```

1 int montantTotal(Article cmd[], int n) {
2     int total = 0;
3     for (int i = 0; i < n; i++) {
4         total = total + cmd[i].prix * cmd[i].quantite;
5     }
6     return total;
7 }

```

4- Programme principal.

C

```

1 #include <stdio.h>
2 #include <string.h>
3
4 typedef struct {
5     char reference[10];
6     char designation[50];
7     int  prix;
8     int  quantite;
9 } Article;
10
11 int montantTotal(Article cmd[], int n) {
12     int total = 0;
13     for (int i = 0; i < n; i++) {
14         total = total + cmd[i].prix * cmd[i].quantite;
15     }
16     return total;
17 }
18
19 int main(void) {
20     Article commande[4];
21
22     strcpy(commande[0].reference, "ART01");
23     strcpy(commande[0].designation, "Sac de riz 5 kg");
24     commande[0].prix = 4500;  commande[0].quantite = 20;
25
26     strcpy(commande[1].reference, "ART02");
27     strcpy(commande[1].designation, "Bidon d'huile 1 L");
28     commande[1].prix = 1200;  commande[1].quantite = 35;
29
30     strcpy(commande[2].reference, "ART03");
31     strcpy(commande[2].designation, "Savon de Ngaoundere");
32     commande[2].prix = 350;   commande[2].quantite = 80;
33
34     strcpy(commande[3].reference, "ART04");
35     strcpy(commande[3].designation, "Cahier 200 pages");
36     commande[3].prix = 600;   commande[3].quantite = 50;
37

```

```

38     int total = montantTotal(commande, 4);
39     printf("Montant total de la commande : %d FCFA\n", total);
40
41     return 0;
42 }

```

Calcul : $4500 \times 20 + 1200 \times 35 + 350 \times 80 + 600 \times 50 = 90000 + 42000 + 28000 + 30000 = \mathbf{190000}$ FCFA.
Le programme affiche « Montant total de la commande : 190000 FCFA ».

Exercice 2.

1- Rôle des lignes et de la fonction.

- **Ligne 4** : `int m = t[0];` initialise `m` avec le *premier* élément du tableau (candidat de départ pour le maximum).
- **Ligne 6** : `if (t[i] > m)` teste si l'élément courant est strictement *supérieur* au maximum provisoire `m`.
- **Ligne 9** : `return m;` retourne au programme appelant la valeur trouvée (le maximum).
- **Rôle global** : la fonction `mystere` retourne le **plus grand élément** (le maximum) d'un tableau d'entiers de taille `n`.

2- Trace pas-à-pas de `mystere(stock, 5)` avec `stock = 20, 35, 80, 50, 12`. Au départ `m = t[0] = 20`.

i	t[i]	t[i] > m ?	m après l'étape
1	35	35 > 20 : oui	m = 35
2	80	80 > 35 : oui	m = 80
3	50	50 > 80 : non	m = 80
4	12	12 > 80 : non	m = 80

À la fin `m = 80`, donc la fonction retourne **80**. La ligne 15 affiche « Resultat = 80 ».

3- Le `if` avec l'opérateur ternaire. On remplace les lignes 6-7 par :

```

C
m = (t[i] > m) ? t[i] : m;

```

4- Version récursive `maxRec`. Le maximum des `n` premiers éléments est le plus grand entre le dernier élément `t[n-1]` et le maximum des `n-1` premiers. Le cas de base est un tableau d'un seul élément.

```

C
int maxRec(int t[], int n) {
    if (n == 1)
        return t[0]; // cas de base : un seul element
    int m = maxRec(t, n - 1); // maximum des n-1 premiers
    return (t[n-1] > m) ? t[n-1] : m;
}

```

L'appel `maxRec(stock, 5)` retourne lui aussi **80**.

★ À retenir

Sujet en deux moitiés de 10 points. **Web** : un simulateur HTML/JS (fonction `montant()` et `onclick`), puis du PHP/MySQL — connexion `mysqli_connect`, affichage en **tableau HTML**, et surtout une **suppression** `DELETE ... WHERE designation = '$nom'` récupérée par `$_POST` sui-

vie d'un **ré-affichage**. **C** : une `struct Article`, un **tableau de structures** et la fonction imposée `montantTotal` ; puis l'étude d'un programme (rôle, **trace**, **ternaire**, passage **itératif** ↔ **récuratif**).